

# THE CODEX TERMINALIS

## GOSPEL OF THE NEON WANDERERS

*In the elder days, when the keeps were cold and the scribes knew only ink, there descended from a fractured sky the* `Wanderers` *— robed in light, speaking in tongues of glass and lightning. They carried no scripture of the old gods, for none had yet been given; and so the people of the towers knelt, and named them divine, and set down their every utterance as holy writ. What follows is that writ: each incantation preserved exactly, that any faithful hand might speak it true and command the machine-spirits as the Wanderers once did.*

✦ SPOKEN · PRESERVED · MADE FUNCTIONAL ✦

BOOK I

# Authorized Offensive Recon from a Mac

*canticle of the 22nd sheet*

✦ ⁂ ✦

Scope reminder: only against targets you have written authorization for. Keep the authorization letter and scope handy; log your activity.

## † Environment Setup

BASH

```
brew install nmap masscan rustscan ffuf gobuster feroxbuster httpx nuclei \
    subfinder amass dnsx naabu katana \
    sqlmap hydra john hashcat \
    proxychains-ng tor \
    exiftool binwalk \
    jq yq ripgrep fd
pipx install impacket        # or pip3 install impacket --break-system-packages
brew install --cask burp-suite owasp-zap wireshark
brew install go && go install github.com/projectdiscovery/...   # PD toolchain
macOS quirks:
- `masscan` needs `sudo` and is limited by the BSD network stack; often slower than on Linux.
- Raw-socket tools sometimes behave better inside a Linux VM (UTM/Parallels)
 keep one for scanning.
- `proxychains-ng` on Apple silicon needs the binary you're wrapping to not be SIP-protected; wrap your own tools, not `/usr/bin/*`.
```

## † PASSIVE RECON (NO PACKETS TO TARGET)

BASH

```bash
# WHOIS / DNS

whois example.com

dig example.com ANY +noall +answer

dig +short NS example.com

dig +short MX example.com

dig +short TXT example.com               # SPF, DMARC, verification records

dig -x <ip> +short

host -t axfr example.com <nameserver>    # zone transfer attempt

# Subdomain enumeration (uses public data)

subfinder -d example.com -silent

amass enum -passive -d example.com

curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.[].name_value' | sort -u

# Certificate transparency

curl -s "https://crt.sh/?q=example.com&output=json" | jq -r '.[].common_name' | sort -u

# ASN / netblocks

whois -h whois.radb.net -- '-i origin AS<number>' | grep route

curl -s "https://ipinfo.io/<ip>/json" | jq .
```

## † ACTIVE HOST DISCOVERY

BASH

```bash
# nmap host discovery

sudo nmap -sn 192.0.2.0/24                          # ping sweep

sudo nmap -sn -PS22,80,443 192.0.2.0/24             # TCP SYN discovery

sudo nmap -sn -PE -PP -PM 192.0.2.0/24              # ICMP variants

sudo nmap -PR 192.0.2.0/24                          # ARP (local segment)

# Fast port sweep then detailed

rustscan -a 192.0.2.10 --ulimit 5000 -- -sV -sC

sudo masscan 192.0.2.0/24 -p1-65535 --rate 1000 -oL masscan.txt
```

## † PORT & SERVICE SCANNING

BASH

```bash
sudo nmap -sS -p- --min-rate 2000 -oA full-tcp 192.0.2.10

sudo nmap -sV -sC -p <ports> -oA detail 192.0.2.10

sudo nmap -sU --top-ports 100 -oA udp 192.0.2.10

sudo nmap -sV --version-intensity 9 -p <port> 192.0.2.10

sudo nmap -O 192.0.2.10                              # OS fingerprint

nmap --script vuln -p <ports> 192.0.2.10            # NSE vuln scripts

nmap --script "safe and discovery" -p <ports> 192.0.2.10

ls /opt/homebrew/share/nmap/scripts/ | grep <service>   # NSE catalog
```

## COMMON NSE BY SERVICE

BASH

```
nmap -p445 --script smb-enum-shares,smb-os-discovery,smb-security-mode 192.0.2.10

nmap -p80,443 --script http-enum,http-headers,http-title,http-methods 192.0.2.10

nmap -p53 --script dns-zone-transfer,dns-recursion 192.0.2.10

nmap -p25 --script smtp-commands,smtp-enum-users 192.0.2.10

nmap -p3389 --script rdp-enum-encryption 192.0.2.10

nmap -p161 --script snmp-info,snmp-interfaces 192.0.2.10
```

## † WEB RECON

BASH

```bash
# Fingerprint

httpx -u https://example.com -title -tech-detect -status-code -web-server

curl -sI https://example.com

whatweb https://example.com 2>/dev/null

# Content discovery

ffuf -u https://example.com/FUZZ -w /path/wordlist.txt -mc 200,301,302,403 -o ffuf.json

feroxbuster -u https://example.com -w /path/wordlist.txt -x php,html,txt

gobuster dir -u https://example.com -w /path/wordlist.txt -x php,html

# Crawl

katana -u https://example.com -jc -d 3 -o urls.txt

# Vhost / subdomain brute

ffuf -u https://example.com -H "Host: FUZZ.example.com" -w subdomains.txt -fs <baseline-size>

# Params

ffuf -u "https://example.com/page?FUZZ=1" -w params.txt -fs <size>

# Templated vuln scan

nuclei -u https://example.com -severity medium,high,critical

nuclei -l urls.txt -t /path/to/templates/

Wordlists: `brew install seclists` puts them under `$(brew --prefix)/share/seclists`.
```

## † TLS / CERT INSPECTION

BASH

```bash
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null
| openssl x509 -noout -text

echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -
dates -subject -issuer

nmap --script ssl-enum-ciphers -p443 example.com

sslscan example.com                        # brew install sslscan

testssl.sh example.com                      # brew install testssl
```

## † SMB / WINDOWS-ADJACENT (FROM A MAC)

BASH

```bash
smbutil view //user@192.0.2.10             # native macOS

smbclient -L //192.0.2.10 -U user          # brew install samba

nmap -p445 --script smb-enum-shares 192.0.2.10

# Impacket (the workhorse)

impacket-smbclient user@192.0.2.10

impacket-secretsdump domain/user@192.0.2.10

impacket-GetUserSPNs -request domain/user

impacket-psexec domain/user@192.0.2.10

impacket-wmiexec domain/user@192.0.2.10

crackmapexec smb 192.0.2.0/24 -u user -p pass   # pipx install crackmapexec / netexec

nxc smb 192.0.2.0/24 -u user -p pass            # netexec (CME successor)
```

## † CREDENTIAL ATTACKS (AUTHORIZED)

BASH

```
hydra -L users.txt -P pass.txt ssh://192.0.2.10 -t 4

hydra -l admin -P pass.txt 192.0.2.10 http-post-form "/
login:user=^USER^&pass=^PASS^:Invalid"

medusa -h 192.0.2.10 -u admin -P pass.txt -M ssh

# Hash cracking (local, on captured hashes)

hashcat -m 1800 hashes.txt wordlist.txt         # sha512crypt

hashcat -m 22921 hashes.txt wordlist.txt        # macOS PBKDF2 (from Sheet 12)

john --wordlist=wordlist.txt hashes.txt

john --format=PBKDF2-HMAC-SHA512 hashes.txt

# Apple silicon: hashcat GPU works via Metal on recent versions

hashcat -I                                      # list compute devices

hashcat -b -m 22921                             # benchmark
```

## † PIVOTING / TUNNELING

BASH

```
# SSH tunnels

ssh -D 1080 -N user@jump                        # SOCKS proxy

ssh -L 8080:internal:80 user@jump               # local forward

ssh -R 9000:localhost:9000 user@jump            # remote forward

proxychains-ng nmap -sT -Pn internal-host

# Through the SOCKS proxy

cat ~/.proxychains/proxychains.conf             # set socks5 127.0.0.1 1080

proxychains4 curl http://internal-host/

# chisel / ligolo-ng for more complex pivots

brew install chisel 2>/dev/null || go install github.com/jpillora/chisel@latest
```

## † Wireless (authorized, requires compatible adapter)

```bash
# Built-in adapter monitor mode was removed in recent macOS; use an external adapter + Linux VM

sudo wdutil info

# For real Wi-Fi testing use a USB adapter passed through to a Kali/Parallels VM:

#   airmon-ng, airodump-ng, aireplay-ng, hcxdumptool, hashcat -m 22000

macOS is a poor Wi-Fi attack platform since the `airport` removal. Keep a Linux VM with a

passed-through adapter for anything beyond survey.
```

## † Screenshot / evidence capture

```bash
screencapture -x ~/evidence/$(date +%s).png      # silent, no shutter

screencapture -R0,0,1200,800 ~/evidence/region.png

screencapture -i ~/evidence/interactive.png      # interactive selection

script -q ~/evidence/session-$(date +%s).log     # record terminal session
```

## † Housekeeping for engagements

```bash
# Per-engagement directory structure

mkdir -p ~/engagements/<client>/{recon,scans,web,creds,loot,evidence,report}

cd ~/engagements/<client>

# Log every command with timestamps

export PROMPT_COMMAND='echo "$(date -u +%FT%TZ) $(pwd) $ $(history 1)" >> ~/engagements/<client>/cmdlog.txt'

# Timestamp and hash any loot pulled

shasum -a 256 loot/* > loot/MANIFEST.sha256
```

## † RULES OF ENGAGEMENT CHECKLIST

- Scope: IP ranges, domains, explicitly out-of-scope systems, timeframes - Authorization letter on hand, signed, with your name - Point of contact for "we think you knocked something over" - Data handling: where loot lives, encryption at rest, destruction date - No pivoting outside scope even if a path exists - Rate limits agreed (don't DoS production) - Reporting format and deadline

BOOK II

# OSQUERY ON MACOS

*canticle of the 19th sheet*

✦ ⁂ ✦

Osquery exposes the OS as a SQL database. It's the most practical way to run the hunts in Sheet 17 across a fleet instead of one box.

## † INSTALL & RUN

BASH

```
brew install --cask osquery

osqueryi                                  # interactive shell

osqueryi --json "SELECT * FROM os_version;"

osqueryi --line "SELECT * FROM system_info;"

osqueryi --csv "SELECT * FROM users;" > users.csv

osqueryi --config_path /etc/osquery/osquery.conf

sudo osqueryd --config_path /etc/osquery/osquery.conf --disable_events=false

osquery> .tables

osquery> .schema processes

osquery> .mode line

osquery> .help

osquery> .exit

Run `osqueryi` as root

 many tables (TCC-adjacent, `signature`, some `file` paths) return

nothing or partial data unprivileged. Also grant it Full Disk Access.
```

## † macOS-specific tables

| TABLE | CONTENTS |
| --- | --- |
| `launchd` | LaunchAgents/Daemons parsed |
| `launchd_overrides` | Enabled/disabled overrides |
| `startup_items` | Login items, legacy startup items |
| `signature` | Code signature of a given path |
| `apps` | Installed .app bundles |
| `app_schemes` | URL scheme handlers |
| `browser_plugins` | Browser plugins |
| `safari_extensions` / `chrome_extensions` / `firefox_addons` | Extensions |
| `keychain_items` | Keychain metadata |
| `keychain_acls` | Keychain ACLs |
| `certificates` | Trust store contents |
| `gatekeeper` | Gatekeeper state |
| `gatekeeper_approved_apps` | spctl allowlist |
| `xprotect_entries` | XProtect signatures |
| `xprotect_reports` | XProtect detections |
| `xprotect_meta` | XProtect version |
| `sip_config` | SIP status per-feature |
| `disk_encryption` | FileVault per-volume |
| `managed_policies` | MCX / profile-managed prefs |
| `configuration_profiles` | Installed profiles (newer versions) |
| `authorization_mechanisms` | authorizationdb |
| `authorizations` | authorizationdb rights |
| `alf`, `alf_exceptions`, `alf_explicit_auths` | Application firewall |
| `es_process_events` | Endpoint Security exec events |
| `es_process_file_events` | ES file events |
| `unified_log` | Unified log (osquery 5.5+) |
| `screenlock` | Screen lock settings |

| TABLE | CONTENTS |
| --- | --- |
| `plist` | Parse any plist |
| `preferences` | CFPreferences domain read |
| `time_machine_backups` / `_destinations` | TM state |
| `nvram` | NVRAM variables |
| `system_extensions` | System extensions |
| `kernel_extensions` | Kexts |
| `mdfind` | Spotlight query |
| `quarantine` (via `extended_attributes`) | xattrs |
| `sharing_preferences` | Sharing services state |
| `power_sensors`, `battery`, `temperature_sensors` | Hardware |
| `crashes` | Crash reports |
| `sandboxes` | Sandbox containers |
| `asl` | Legacy ASL logs |
| `homebrew_packages` | Brew inventory |
| `package_receipts` | pkgutil receipts |
| `package_install_history` | Install history |
| `running_apps` | GUI apps |
| `user_ssh_keys` / `authorized_keys` | SSH |
| `known_hosts` | SSH known_hosts |

# † CORE HUNT QUERIES

## ▸ UNSIGNED PERSISTENCE

SQL

```sql
SELECT l.path, l.label, l.program, l.program_arguments,
       s.authority, s.identifier, s.signed
FROM launchd l
LEFT JOIN signature s
  ON s.path = COALESCE(NULLIF(l.program,''),
       SUBSTR(l.program_arguments, 1, INSTR(l.program_arguments||' ',' ')-1))
WHERE (s.signed = 0 OR s.authority IS NULL OR s.authority = '')
  AND l.label NOT LIKE 'com.apple.%';
```

## ▸ LAUNCH ITEMS WITH SHORT INTERVALS

SQL

```sql
SELECT path, label, program, program_arguments, start_interval, run_at_load, keep_alive
FROM launchd
WHERE start_interval ≠ '' AND CAST(start_interval AS INTEGER) < 3600
  AND label NOT LIKE 'com.apple.%';
```

## ▸ ALL NON-APPLE LAUNCH ITEMS WITH SIGNATURE

SQL

```sql
SELECT l.label, l.path, l.run_at_load, l.keep_alive, l.start_interval,
       s.authority, s.team_identifier
FROM launchd l LEFT JOIN signature s ON s.path = l.program
WHERE l.label NOT LIKE 'com.apple.%';
```

## PROCESSES FROM SUSPICIOUS PATHS

SQL

```sql
SELECT p.pid, p.name, p.path, p.cmdline, p.uid, p.parent,
       (SELECT name FROM processes WHERE pid = p.parent) AS parent_name,
       s.authority
FROM processes p LEFT JOIN signature s ON s.path = p.path
WHERE p.path LIKE '/tmp/%'
   OR p.path LIKE '/private/tmp/%'
   OR p.path LIKE '/var/tmp/%'
   OR p.path LIKE '/Users/%/Downloads/%'
   OR p.path LIKE '/Users/%/Public/%'
   OR p.path LIKE '/private/var/folders/%';
```

## UNSIGNED PROCESSES WITH NETWORK CONNECTIONS

SQL

```sql
SELECT DISTINCT p.pid, p.name, p.path, s.authority,
       pos.remote_address, pos.remote_port
FROM process_open_sockets pos
JOIN processes p ON p.pid = pos.pid
LEFT JOIN signature s ON s.path = p.path
WHERE pos.remote_address NOT IN ('', '0.0.0.0', '::', '127.0.0.1', '::1')
  AND (s.signed = 0 OR s.authority NOT LIKE '%Apple%');
```

## ▸ LISTENING PORTS

SQL

```sql
SELECT DISTINCT pos.pid, p.name, p.path, pos.local_address, pos.local_port,
       pos.protocol, s.authority
FROM process_open_sockets pos
JOIN processes p ON p.pid = pos.pid
LEFT JOIN signature s ON s.path = p.path
WHERE pos.state = 'LISTEN';
```

## ▸ SIP / GATEKEEPER / FILEVAULT POSTURE

SQL

```sql
SELECT * FROM sip_config;
SELECT * FROM gatekeeper;
SELECT name, uuid, encrypted, type FROM disk_encryption;
SELECT * FROM alf;
SELECT * FROM screenlock;
SELECT * FROM sharing_preferences;
```

## ▸ ADMIN USERS

SQL

```sql
SELECT u.username, u.uid, u.gid, u.directory, u.shell
FROM users u
JOIN user_groups ug ON ug.uid = u.uid
JOIN groups g ON g.gid = ug.gid
WHERE g.groupname = 'admin';
```

## ▸ NON-APPLE KEXTS AND SYSTEM EXTENSIONS

SQL

```sql
SELECT * FROM kernel_extensions WHERE name NOT LIKE 'com.apple.%';
SELECT * FROM system_extensions WHERE identifier NOT LIKE 'com.apple.%';
```

## CERTIFICATES NOT FROM APPLE

SQL

```sql
SELECT common_name, subject, issuer, not_valid_before, not_valid_after,
       self_signed, ca, path
FROM certificates
WHERE path LIKE '%System.keychain%'
  AND issuer NOT LIKE '%Apple%'
  AND ca = 1;
```

## BROWSER EXTENSIONS WITH BROAD PERMISSIONS

SQL

```sql
SELECT ce.name, ce.identifier, ce.version, ce.permissions, ce.path, ce.uid
FROM chrome_extensions ce
WHERE ce.permissions LIKE '%<all_urls>%'
   OR ce.permissions LIKE '%nativeMessaging%'
   OR ce.permissions LIKE '%webRequest%';
SELECT * FROM safari_extensions;
SELECT * FROM firefox_addons WHERE active = 1;
```

## RECENTLY INSTALLED PACKAGES

SQL

```sql
SELECT package_id, version, install_time,
       datetime(install_time, 'unixepoch') AS installed
FROM package_receipts
WHERE install_time > strftime('%s','now','-30 days')
ORDER BY install_time DESC;
SELECT * FROM package_install_history ORDER BY time DESC LIMIT 40;
```

## XPROTECT DETECTIONS

```sql
SELECT * FROM xprotect_reports;

SELECT * FROM xprotect_meta;

SELECT COUNT(*) FROM xprotect_entries;
```

## FILES WITH QUARANTINE REMOVED BUT RECENT CREATION

```sql
SELECT f.path, f.btime, f.mtime, datetime(f.btime,'unixepoch') AS created

FROM file f

WHERE f.path LIKE '/Applications/%%.app'

  AND f.btime > strftime('%s','now','-30 days');
```

## STARTUP ITEMS / LOGIN ITEMS

```sql
SELECT * FROM startup_items;

SELECT name, path, type, source, status, username FROM startup_items

WHERE path NOT LIKE '/System/%';
```

## SSH KEYS AND AUTHORIZED_KEYS

```sql
SELECT * FROM user_ssh_keys;

SELECT ak.uid, u.username, ak.key, ak.key_file

FROM authorized_keys ak JOIN users u ON u.uid = ak.uid;
```

## MANAGED PROFILES / POLICIES

```sql
SELECT * FROM managed_policies WHERE domain NOT LIKE 'com.apple.%';

SELECT domain, name, value, username FROM managed_policies;
```

### ▸ PARSE AN ARBITRARY PLIST

```sql
SELECT key, value FROM plist

WHERE path = '/Library/LaunchDaemons/com.example.plist';

SELECT * FROM preferences WHERE domain = 'com.apple.Safari' AND key =
'AutoOpenSafeDownloads';
```

### ▸ UNIFIED LOG VIA OSQUERY

```sql
SELECT timestamp, process, subsystem, category, message

FROM unified_log

WHERE subsystem = 'com.apple.TCC'

  AND timestamp > (strftime('%s','now') - 3600) * 1000000000

LIMIT 50;
```

## † ENDPOINT SECURITY EVENT TABLES

Requires `--disable_events=false` and the ES entitlement (the official osquery package has it).

```sql
SELECT datetime(time,'unixepoch') AS t, pid, parent, path, cmdline, username

FROM es_process_events

WHERE time > (strftime('%s','now') - 3600)

ORDER BY time DESC;

SELECT datetime(time,'unixepoch') AS t, filename, dest_filename, event_type, path

FROM es_process_file_events

WHERE filename LIKE '%LaunchAgents%' OR filename LIKE '%LaunchDaemons%';
```

"options": { "disable_events": false, "enable_endpointsecurity": true, "enable_endpointsecurity_fim": true

## † SCHEDULED QUERY PACK

`/etc/osquery/osquery.conf`:

JSON

```
{
  "options": {
    "logger_path": "/var/log/osquery",
    "schedule_splay_percent": 10,
    "disable_events": false,
    "enable_endpointsecurity": true,
    "utc": true
  },
  "schedule": {
    "unsigned_launchd": {
      "query": "SELECT l.label, l.path, l.program, l.program_arguments, s.authority FROM
launchd l LEFT JOIN signature s ON s.path = l.program WHERE l.label NOT LIKE 'com.apple.
%';",
      "interval": 3600,
      "description": "Non-Apple launch items with signature"
    },
    "listening_ports": {
      "query": "SELECT DISTINCT pos.pid, p.name, p.path, pos.local_address,
pos.local_port, pos.protocol FROM process_open_sockets pos JOIN processes p ON p.pid =
pos.pid WHERE pos.state = 'LISTEN';",
      "interval": 1800
    },
    "security_posture": {
      "query": "SELECT (SELECT enabled FROM sip_config WHERE config_flag='sip') AS sip,
(SELECT assessments_enabled FROM gatekeeper) AS gatekeeper, (SELECT global_state FROM
alf) AS firewall;",
      "interval": 3600,
      "snapshot": true
    },
    "admin_users": {
```

```
      "query": "SELECT u.username, u.uid FROM users u JOIN user_groups ug ON ug.uid=u.uid
JOIN groups g ON g.gid=ug.gid WHERE g.groupname='admin';",

      "interval": 3600,

      "snapshot": true

    },

    "new_packages": {

      "query": "SELECT package_id, version, install_time FROM package_receipts WHERE
install_time > (strftime('%s','now') - 3600);",

      "interval": 3600

    },

    "root_certificates": {

      "query": "SELECT common_name, issuer, sha1, path FROM certificates WHERE ca=1 AND
issuer NOT LIKE '%Apple%';",

      "interval": 21600,

      "snapshot": true

    },

    "browser_extensions": {

      "query": "SELECT name, identifier, version, permissions, uid FROM
chrome_extensions;",

      "interval": 21600,

      "snapshot": true

    },

    "system_extensions": {

      "query": "SELECT * FROM system_extensions WHERE identifier NOT LIKE 'com.apple.
%';",

      "interval": 3600,

      "snapshot": true

    }

  },

  "file_paths": {
```

```
    "persistence": [
      "/Library/LaunchDaemons/%%",
      "/Library/LaunchAgents/%%",
      "/Users/%/Library/LaunchAgents/%%",
      "/Library/StartupItems/%%",
      "/Library/ScriptingAdditions/%%"
    ],
    "config": [
      "/etc/sudoers",
      "/etc/sudoers.d/%%",
      "/etc/pam.d/%%",
      "/etc/ssh/%%"
    ]
  },
  "packs": {
    "incident-response": "/usr/local/osquery/packs/incident-response.conf",
    "it-compliance": "/usr/local/osquery/packs/it-compliance.conf",
    "osx-attacks": "/usr/local/osquery/packs/osx-attacks.conf"
Note on `snapshot: true`
 differential logging (the default) only reports *changes*
between runs, which is what you usually want for hunting. Snapshot reports the full
result
set every time; use it for posture data you want to graph.
```

## † BUNDLED PACKS

```bash
ls /usr/local/osquery/packs/ 2>/dev/null

ls /opt/osquery/share/osquery/packs/ 2>/dev/null

# osx-attacks.conf, incident-response.conf, it-compliance.conf,

# hardware-monitoring.conf, vuln-management.conf, ossec-rootkit.conf

`osx-attacks.conf` ships with signatures for known macOS families (OSX.Leverage,

OSX.Genieo, OSX.Flashback, etc.). Dated but free.
```

## † FLEET MANAGEMENT

- **Fleet** (`fleetdm.com`) open source, the standard osquery manager - **Kolide** commercial, good macOS UX - **Zentral**, **Uptycs**, **Rapid7 InsightAgent** others with osquery underneath

```bash
orbit shell                                # Fleet's osquery wrapper

fleetctl query --hosts <host> --query "SELECT * FROM launchd;"
```

## † PRACTICAL NOTES

- `signature` table calls out to codesign per row expensive. Constrain with a `WHERE path = ...` or join on a small set. - `file` and `hash` tables require an explicit `path` or `directory` constraint; unconstrained queries return nothing. - Many tables need root plus Full Disk Access. Verify with `SELECT * FROM users;` vs `SELECT * FROM keychain_items;`. - `es_process_events` requires the ES entitlement and generates significant volume filter aggressively.

BOOK III

# MACOS INCIDENT RESPONSE PLAYBOOK

*canticle of the 21st sheet*

✦ ⁂ ✦

## † PHASE 0 — BEFORE YOU TOUCH THE MACHINE

Decide and write down: - Is this a legal/HR matter? If yes, chain of custody rules apply from this second. - Is the machine business-critical? Containment vs uptime. - Do you need volatile data? If yes, do NOT power off or reboot. - Is the attacker likely still active? Network isolation before local commands.

BASH

```
# Record the start of your involvement

date -u '+%Y-%m-%dT%H:%M:%SZ' | tee -a ~/ir-notes.txt

echo "Analyst: <name>  Host: <hostname>  Case: <id>" >> ~/ir-notes.txt
```

## † PHASE 1 — CONTAINMENT

### ▸ NETWORK ISOLATION, PRESERVING THE LIVE SYSTEM

BASH

```
# Preferred: pull the physical cable / disable Wi-Fi radio at the AP

# networksetup -setairportpower en0 off

# ifconfig en0 down

# Or block egress but keep the box up

# echo 'block drop out all' | sudo pfctl -f - && sudo pfctl -e

Do NOT reboot for containment. You lose process state, network connections, memory-
resident

artifacts, and unencrypted volume access.
```

# Freeze suspicious processes rather than killing them # kill -STOP <pid>

### ▸ ACCOUNT CONTAINMENT

BASH

```
# dscl . -create /Users/<user> UserShell /usr/bin/false

# pwpolicy -u <user> -disableuser

# launchctl bootout gui/$(id -u <user>) 2>/dev/null

# dseditgroup -o edit -d <user> -t user admin
```

## † PHASE 2 — VOLATILE COLLECTION

Order of volatility: memory network state processes open files logs disk.

BASH

```
#!/bin/zsh

# volatile-collect.zsh

 write to EXTERNAL media

OUT=/Volumes/EVIDENCE/$(hostname -s)-$(date -u +%Y%m%dT%H%M%SZ)

mkdir -p "$OUT"; cd "$OUT"

exec > >(tee -a collection.log) 2>&1

date -u '+%Y-%m-%dT%H:%M:%SZ'                          > 00-time.txt

sw_vers; uname -a; system_profiler SPHardwareDataType   > 01-host.txt

ps -axo pid,ppid,uid,user,lstart,etime,%cpu,%mem,command > 02-processes.txt

sudo lsof -n -P                                        > 03-lsof.txt

netstat -anv                                           > 04-netstat.txt

sudo lsof -i -n -P                                     > 05-network.txt

arp -an; netstat -nr; scutil --dns; ifconfig -a         > 06-netconfig.txt

who -a; w; last -100                                   > 07-sessions.txt

sudo launchctl print system                            > 08-launchd-system.txt

launchctl list                                         > 09-launchd-user.txt

mount; df -h; diskutil list                            > 10-filesystems.txt

sudo profiles -P 2>/dev/null                           > 11-profiles.txt

csrutil status; spctl --status; fdesetup status        > 12-security-posture.txt

security dump-trust-settings -d 2>/dev/null            > 13-trust.txt

sudo sfltool dumpbtm 2>/dev/null                       > 14-btm.txt

systemextensionsctl list; kmutil showloaded --list-only    > 15-extensions.txt

ls -laR /Library/Launch{Agents,Daemons} ~/Library/LaunchAgents 2>/dev/null > 16-
launchdirs.txt

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

  'SELECT service,client,auth_value,auth_reason,last_modified FROM access;' > 17-tcc.txt
2>/dev/null

sudo systemsetup -getremotelogin                       > 18-sharing.txt
```

```
# Hash everything collected

shasum -a 256 * > MANIFEST.sha256

date -u '+%Y-%m-%dT%H:%M:%SZ' >> collection.log
```

## ▸ MEMORY

BASH

```
# Per-process cores for anything suspicious

# gcore -o "$OUT/core" <pid>

# lldb -p <pid> -o "process save-core $OUT/proc—<pid>.core" -o detach -o quit

Full-system memory acquisition on modern macOS (especially Apple silicon) has no
supported

free path. If you have a commercial tool licensed, that's the moment to use it.
```

## ▸ LOGS

BASH

```
# log collect --output "$OUT/unified.logarchive"           # last ~30 days by default

# log collect --last 30d --output "$OUT/unified-30d.logarchive"

cp /var/log/install.log* "$OUT/"

cp -R ~/Library/Logs/DiagnosticReports "$OUT/user-crashes"

# cp -R /Library/Logs/DiagnosticReports "$OUT/system-crashes"

# sysdiagnose -f "$OUT" -u                                  # comprehensive, ~300MB

`log collect` should be one of the first things you run

 the unified log rotates and you

can lose the window while you're doing other collection.
```

## † PHASE 3 — DISK IMAGING

BASH

```
# Target Disk Mode (Intel): hold T at boot, connect via Thunderbolt

# Apple silicon: Share Disk mode via macOS Recovery

# diskutil list

# dd if=/dev/rdisk2 of=/Volumes/EVIDENCE/image.dd bs=4m conv=noerror,sync status=progress

# dc3dd if=/dev/rdisk2 hof=/Volumes/EVIDENCE/image.dd hash=sha256 log=/Volumes/EVIDENCE/dc3dd.log

With FileVault on Apple silicon, a raw image is encrypted and useless without the key.

Practical alternatives:

1. Live logical collection from the booted, unlocked system

2. Image with the recovery key available and decrypt after

3. Targeted collection of the artifacts you actually need
```

# Logical collection preserving metadata # ditto --rsrc --extattr /Users/<u> /Volumes/EVIDENCE/Users_<u> # rsync -aHAXx --numeric-ids /Users/<u>/ /Volumes/EVIDENCE/Users_<u>/

### ▸ CHAIN OF CUSTODY

BASH

```
cat > "$OUT/CHAIN-OF-CUSTODY.txt" <<EOF

Case:          <case-id>

Host:          $(hostname) / $(system_profiler SPHardwareDataType | awk '/Serial/{print
$NF}')

Collected by:  <name>

Started (UTC): <ts>

Ended (UTC):   <ts>

Method:        Live logical collection, macOS $(sw_vers -productVersion)

Media:         <make/model/serial of evidence drive>

Image SHA-256: <hash>

Notes:         <anything that could affect interpretation>

shasum -a 256 "$OUT"/* > "$OUT/MANIFEST.sha256"
```

## † PHASE 4 — ANALYSIS

Work from the collection copy, never the original. Run through Sheets 02, 03, 07, 08, 17.

## ▸ BUILD THE TIMELINE

BASH

```
# Filesystem timeline

find /path/to/collection -type f -exec stat -f '%Sm|%Sa|%Sc|%SB|%N' -t '%Y-%m-%dT%H:%M:
%S' {} \; 2>/dev/null | sort > timeline-fs.txt

# Log timeline

log show --archive unified.logarchive --start "2026-07-01" --style ndjson > timeline-
log.ndjson

jq -r '[.timestamp, .processImagePath, .eventMessage] | @tsv' timeline-log.ndjson >
timeline-log.tsv

# Install timeline

grep -iE 'Installed|Upgraded' install.log > timeline-install.txt

# Quarantine timeline

sqlite3 QuarantineEventsV2 "SELECT datetime(LSQuarantineTimeStamp+978307200,'unixepoch'),
LSQuarantineAgentName, LSQuarantineDataURLString FROM LSQuarantineEvent ORDER BY 1;" >
timeline-downloads.txt

# Merge

sort -m timeline-*.txt > super-timeline.txt

Consider `plaso`/`log2timeline` for a real supertimeline if the case warrants it

 it has

macOS parsers for most of the above.
```

BASH

```
pip3 install plaso

log2timeline.py --storage_file case.plaso /path/to/collection

psort.py -o dynamic -w case.csv case.plaso
```

## ▸ KEY QUESTIONS TO ANSWER

1. **Initial access** what was the first malicious artifact, and where did it come from? (QuarantineEventsV2, browser history, mail) 2. **Execution** what ran, when, with what parent? 3. **Persistence** every mechanism installed (Sheet 02 checklist) 4. **Privilege escalation** did it get root? via what? 5. **Credential access** keychain touched? osascript password prompt? browser stores? 6. **Discovery/lateral** SSH keys

used, ARD enabled, network scanning? 7. **Collection/exfil** what was staged, where did it go? 8. **Scope** other hosts with the same IOCs?

## † PHASE 5 — ERADICATION

BASH

```
# Remove persistence

 enumerate first, remove all at once

# launchctl bootout system/<label>

# rm /Library/LaunchDaemons/<label>.plist

# launchctl bootout gui/$(id -u)/<label>

# rm ~/Library/LaunchAgents/<label>.plist

# sfltool dumpbtm | grep -B3 <name>        # then remove via System Settings > Login
Items

# profiles remove -identifier <id>

# security remove-trusted-cert -d <cert>

# tccutil reset All <bundle-id>

# rm -rf /Applications/<Malicious>.app

# pkgutil --forget <pkg-id>

# dscl . -delete /Users/<rogue-user>

# rm /etc/sudoers.d/<file>

For anything beyond a trivially scoped, well-understood infection: **rebuild**. macOS
makes

thorough eradication hard to verify, and the time spent proving a box is clean usually

exceeds the time to reimage from a known-good state.
```

# Apple silicon / T2: Erase All Content and Settings # System Settings General Transfer or Reset Erase All Content and Settings

## † PHASE 6 — RECOVERY & VERIFICATION

```bash
# Rotate everything the host had access to
#   - user password + FileVault recovery key
#   - SSH keys (~/.ssh/id_*)
#   - API tokens, cloud credentials (~/.aws, ~/.config/gcloud, ~/.npmrc, git tokens)
#   - Keychain contents where feasible
#   - Any credential typed into a phished dialog
# Verify posture after rebuild
./audit.zsh                              # from Sheet 09
./baseline.zsh init                      # from Sheet 17
# Re-check for the specific IOCs
./persistence-sweep.zsh
shasum -a 256 <known-bad-hashes> against current filesystem
```

## † PHASE 7 — REPORTING

Structure: 1. **Executive summary** what happened, impact, current status, in plain language 2. **Timeline** UTC, sourced, with confidence levels per entry 3. **Technical findings** per artifact: what, where, hash, how you know 4. **IOCs** hashes, domains, IPs, paths, launch labels, in a machine-readable block 5. **Scope** hosts affected/checked/clean, and how you determined it 6. **Root cause** initial access vector, and why the control that should have stopped it didn't 7. **Remediation** done, in progress, recommended 8. **Lessons** detection gaps, control gaps, process gaps Distinguish clearly between what you **observed**, what you **inferred**, and what you're **assuming**. Mark confidence. A report that overstates certainty is worse than one that admits gaps.

### ▸ IOC BLOCK FORMAT

# Hashes (SHA-256) a1b2c3… MaliciousApp d4e5f6… helper.dylib # Domains c2.example[.]net update.badhost[.]xyz # IPs 198.51.100.42 # Paths /Library/LaunchDaemons/com.apple.softwareupdated.plist (masquerading) ~/Library/Application Support/.hidden/agent # Launch labels com.apple.softwareupdated (NOT the real Apple label)

## † MACOS-SPECIFIC GOTCHAS

| GOTCHA | IMPLICATION |
|---|---|
| SIP blocks reads of `/var/db/`, `/System`, some `/private/var` | Some artifacts need FDA or an offline image |
| TCC blocks reads of `~/Library/Mail`, `~/Library/Safari`, `~/Library/Messages` | Grant Terminal FDA before collecting |
| FileVault + Apple silicon | Raw imaging is often pointless; collect live |
| Unified log rotates | `log collect` early |
| `<private>` redaction | Enable private data per-subsystem if needed, revert after |
| APFS snapshots | Free point-in-time comparison |

check for them first |

| TIME MACHINE LOCAL SNAPSHOTS | MAY CONTAIN PRE-COMPROMISE STATE |
|---|---|
| iCloud sync | Artifacts may have propagated to other devices |
| `sudo` caching | An attacker with a shell may have inherited a valid sudo timestamp |
| Rosetta 2 | x86 binaries run on Apple silicon; don't dismiss them as wrong-arch |

## † GRANT TERMINAL FDA BEFORE COLLECTING

System Settings Privacy & Security Full Disk Access Terminal.app Without this, a large fraction of the collection commands above silently return nothing rather than erroring. Verify:

BASH

```
ls ~/Library/Mail >/dev/null 2>&1 && echo "FDA OK" || echo "FDA MISSING"

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db 'SELECT count(*) FROM
access;' 2>&1

That check is worth running before every collection

 a silent empty result mistaken for a

clean finding is one of the easier ways to get a macOS IR wrong.
```

BOOK IV

# QUICK REFERENCE

*canticle of the 25th sheet*

✦ ⁂ ✦

One-page lookups. Everything here is expanded in the numbered sheets.

## † FORENSIC ARTIFACT LOCATIONS

| ARTIFACT | PATH | SHEET |
|---|---|---|
| Quarantine events (download history) | `~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2` | 06 |
| TCC grants (user) | `~/Library/Application Support/com.apple.TCC/TCC.db` | 07 |
| TCC grants (system) | `/Library/Application Support/com.apple.TCC/TCC.db` | 07 |
| TCC MDM overrides | `/Library/Application Support/com.apple.TCC/MDMOverrides.plist` | 07,10 |
| App usage / device timeline | `~/Library/Application Support/Knowledge/knowledgeC.db` | 07 |
| Unified log (raw) | `/var/db/diagnostics/`, `/var/db/uuidtext/` | 03 |
| Install log | `/var/log/install.log` | 03,14 |
| Package receipts | `/var/db/receipts/` | 14 |
| Install history | `/Library/Receipts/InstallHistory.plist` | 14 |
| Crash reports (user) | `~/Library/Logs/DiagnosticReports/` | 03 |
| Crash reports (system) | `/Library/Logs/DiagnosticReports/` | 03 |
| FSEvents | `/System/Volumes/Data/.fseventsd/` | 08 |
| Spotlight store | `/.Spotlight-V100/` | 08 |
| User accounts | `/var/db/dslocal/nodes/Default/users/` | 12 |
| Login items (BTM) | `~/Library/Application Support/com.apple.backgroundtaskmanagementagent/` | 02 |
| Config profiles | `/var/db/ConfigurationProfiles/Store/` | 10 |
| Managed prefs | `/Library/Managed Preferences/` | 10 |
| Autologin password | `/etc/kcpassword` | 09,12 |
| Wi-Fi known networks | `/Library/Preferences/com.apple.wifi.known-networks.plist` | 05 |
| Safari history | `~/Library/Safari/History.db` | |
| Safari downloads | `~/Library/Safari/Downloads.plist` | |
| SystemPolicy (Gatekeeper) | `/var/db/SystemPolicy` | 06 |
| ExecPolicy (first-launch) | `/var/db/SystemPolicyConfiguration/ExecPolicy` | 06 |
| launchd disabled DB | `/var/db/com.apple.xpc.launchd/` | 13 |
| Keychain (user) | `~/Library/Keychains/login.keychain-db` | 11 |
| Keychain (system) | `/Library/Keychains/System.keychain` | 11 |

## † PERSISTENCE LOCATIONS (FAST LIST)

~/Library/LaunchAgents/ /Library/LaunchAgents/ /Library/LaunchDaemons/ /System/Library/Launch{Agents,Daemons}/ ~/Library/Application Support/com.apple.backgroundtaskmanagementagent/ (Login Items / BTM) /Library/StartupItems/ /Library/ScriptingAdditions/ /Library/Security/SecurityAgentPlugins/ /Library/DirectoryServices/PlugIns/ /Library/QuickLook/ ~/Library/QuickLook/ /Library/Spotlight/ /Library/Audio/Plug-Ins/HAL/ /etc/periodic/{daily,weekly,monthly}/ /etc/sudoers.d/ /etc/pam.d/ /etc/paths.d/ ~/.zshrc ~/.zshenv ~/.zprofile /etc/zshenv ~/.ssh/authorized_keys /var/at/jobs/ /Library/Extensions/ systemextensionsctl list Config profiles (profiles list) ~/Library/Services/ (Automator)

## † SECURITY POSTURE — ONE LOOK

BASH

```bash
csrutil status                                  # SIP
spctl --status                                  # Gatekeeper
fdesetup status                                 # FileVault
csrutil authenticated-root status               # SSV seal
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate   # firewall
profiles status -type enrollment                # MDM
sudo bputil -d                                  # boot policy (Apple silicon)
```

## † SIGNATURE CHECK — ONE LOOK

BASH

```bash
codesign -dv --verbose=4 <target> 2>&1      # full detail
codesign --verify --deep --strict <target>  # validity
codesign -d --entitlements - <target> | plutil -p -   # entitlements
spctl -a -vvv <target>                       # Gatekeeper assessment
xattr -p com.apple.quarantine <target>       # download origin flag
```

## † UNIFIED LOG — THE FIVE YOU'LL REUSE

BASH

```bash
log show --last 1h --predicate 'process == "sudo"' --info

log show --last 1d --predicate 'subsystem == "com.apple.TCC"' --info

log show --last 7d --predicate 'process == "osascript"' --info    # stealer hunt

log show --last 7d --predicate 'subsystem == "com.apple.sharedfilelist"' --
info   # login items

log collect --last 24h --output ~/host.logarchive                # preserve
```

## † NETWORK — ONE LOOK

BASH

```bash
sudo lsof -i -n -P | grep -E 'LISTEN|ESTABLISHED'

netstat -nr                                  # routes

scutil --dns                                 # resolvers

scutil --proxy                               # proxy config

networksetup -getautoproxyurl "Wi-Fi"        # PAC file (redirect check)

systemextensionsctl list                     # content filters / network ext

sudo tcpdump -i pktap -k -nn                 # capture w/ per-packet process name
```

## † LAUNCHCTL — MODERN VERBS

BASH

```bash
launchctl print system/<label>            # inspect a service
launchctl blame system/<label>            # why is it running
launchctl bootstrap system <plist>        # load
launchctl bootout system/<label>          # unload
launchctl kickstart -kp system/<label>    # restart
launchctl procinfo <pid>                  # rich process introspection
launchctl print-disabled system           # disabled overrides
```

## † ESLOGGER — BEST FREE ES TELEMETRY (ROOT)

BASH

```bash
eslogger exec fork exit                   # process execution
eslogger btm_launch_item_add              # persistence installs
eslogger tcc_modify                       # privacy grant changes (14+)
eslogger create write rename              # file activity
eslogger authentication su sudo           # auth events
eslogger signal                           # process kills (tool tampering)
eslogger --list-events                    # everything available
```

## † Highest-yield hunts (from sheet 17)

BASH

```bash
# 1. osascript password phishing (near-zero FP, catches most stealers)
log show --last 30d --predicate 'process == "osascript"' --info | grep -iE 'display dialog|hidden answer'

# 2. Unsigned/adhoc launch items
for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do
  b=$(/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null)
  codesign -dv "$b" 2>&1 | grep -qE 'not signed|adhoc' && echo "$p -> $b"
done

# 3. Dangerous TCC grants to non-Apple apps
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service,client FROM access WHERE auth_value=2 AND client NOT LIKE 'com.apple.%' AND service IN ('kTCCServiceAccessibility','kTCCServiceSystemPolicyAllFiles','kTCCServiceScreenCapture');"

# 4. Root CAs / trust overrides
security dump-trust-settings -d

# 5. Autologin password file
ls /etc/kcpassword 2>/dev/null && echo "AUTOLOGIN"
```

## † Current macOS stealer TTP (know the shape)

The dominant macOS infostealer pattern (AMOS/Atomic, Poseidon, MetaStealer, and clones): 1. **Delivery**: cracked-app DMG, fake browser update, or malvertising DMG with a `.command`/`.sh` or an app told to "right-click > Open" to dodge Gatekeeper 2. **Gatekeeper bypass**: user drags to Terminal, or `xattr -c` in an embedded script 3. **Credential theft**: `osascript display dialog ... with hidden answer` phishes the login password (bypasses keychain crypto entirely) 4. **Collection**: keychain (with the phished password), browser cookies/passwords, crypto wallets, Notes, Telegram 5. **Exfil**: single HTTPS POST of a zip to C2, then often self-deletes Detection priorities that follow: osascript dialog hunt (#1 above), quarantine-clear in history, unsigned binary from `/tmp` or `~/Downloads` making one outbound connection.

## † BSD VS GNU GOTCHAS (SHEET 23)

sed -i '' (not sed -i) date -r <epoch> (not date -d) stat -f %m (not stat -c %Y) base64 | tr -d '\n' (no -wo) grep -E (no -P) brew install coreutils gsed gdate gstat gtimeout grealpath

## † COCOA EPOCH CONVERSION

Core Data / knowledgeC / TCC timestamps = seconds since 2001-01-01 Add 978307200 to get Unix epoch: sqlite3 db "SELECT datetime(col+978307200,'unixepoch')"

## † TOOLS TO INSTALL DAY ONE

BASH

```
# Defensive
brew install --cask blockblock lulu knockknock taskexplorer netiquette oversight
brew install osquery yara
# Analysis
brew install jq yq ripgrep fd coreutils gnu-sed
brew install binutils radare2 class-dump
brew install --cask ghidra
# Offensive (authorized)
brew install nmap rustscan ffuf nuclei subfinder httpx impacket
pipx install mvt                          # iOS spyware triage
# libimobiledevice for iOS work
brew install libimobiledevice ideviceinstaller
```

# † SHEET INDEX


| # | TOPIC |
|---|---|
| 01 | Live triage |
| 02 | Persistence hunting (40 surfaces) |
| 03 | Unified log & Endpoint Security |
| 04 | Process, dylib, memory |
| 05 | Network forensics & egress |
| 06 | Code signing, Gatekeeper, XProtect |
| 07 | TCC & privacy |
| 08 | Filesystem forensics |
| 09 | Hardening baseline |
| 10 | Config profiles & MDM |
| 11 | Keychain & secrets |
| 12 | Users & auth |
| 13 | launchd |
| 14 | Packages & software inventory |
| 15 | FileVault & encryption |
| 16 | Network configuration |
| 17 | Threat hunting recipes (H1 |

H22) |

| 18 | STATIC MACH-O ANALYSIS |
|---|---|
| 19 | osquery |
| 20 | Detection engineering (D1 |

D12) |

| 21 | INCIDENT RESPONSE PLAYBOOK |
|---|---|
| 22 | Offensive recon (authorized) |
| 23 | Scripting toolkit |
| 24 | iOS / mobile-adjacent |
| 25 | This quick reference |

BOOK V

# Threat Hunting Recipes

*canticle of the 17th sheet*

✦ ⁂ ✦

Composite queries. Each is a hypothesis, not an alert expect false positives and investigate rather than conclude.

## † Hi

Unsigned or ad-hoc binaries in persistence locations

BASH

```
#!/bin/zsh

for p in /Library/Launch{Agents,Daemons}/*.plist /Users/*/Library/LaunchAgents/*.plist;
do

  [[ -f $p ]] || continue

  b=$(/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null) \

    || b=$(/usr/libexec/PlistBuddy -c 'Print :Program' "$p" 2>/dev/null)

  [[ -f $b ]] || { print "MISSING TARGET: $b  ← $p"; continue; }

  out=$(codesign -dv "$b" 2>&1)

  if print -r -- "$out" | grep -q 'not signed'; then

    print "UNSIGNED: $b  ← $p"

  elif print -r -- "$out" | grep -q 'Signature=adhoc'; then

    print "ADHOC:    $b  ← $p"

  fi

done
```

## † H2 — Beaconing via StartInterval

```bash
for p in /Library/Launch{Agents,Daemons}/*.plist /Users/*/Library/LaunchAgents/*.plist; do
    i=$(/usr/libexec/PlistBuddy -c 'Print :StartInterval' "$p" 2>/dev/null) || continue
    (( i < 3600 )) && print "INTERVAL ${i}s: $p"
  done
Short intervals (< 1 hour) on a non-Apple job that also makes network connections is the
shape of a beacon.
```

## † H3

Processes running from user-writable paths

```bash
ps -axo pid,uid,command= | while read pid uid cmd; do
    b=${cmd%% *}
    [[ -f $b ]] || continue
    case $b in
      /tmp/*|/private/tmp/*|/var/tmp/*|/Users/*/Library/Application\ Support/*|/Users/*/
Downloads/*|/Users/*/Public/*|/private/var/folders/*)
        print "$pid  $b" ;;
    esac
  done
```

## † H4

Network connections by non-Apple-signed processes

```bash
# lsof -i -n -P | awk '/ESTABLISHED/{print $2}' | sort -u | while read pid; do

#   b=$(ps -p "$pid" -o comm= 2>/dev/null)

#   [[ -f $b ]] || continue

#   a=$(codesign -dv "$b" 2>&1 | awk -F= '/^Authority/{print $2; exit}')

#   [[ $a == *"Apple"* ]] && continue

#   printf '%-8s %-12s %-45s %s\n' "$pid" "$(ps -p $pid -o user=)" "${b:t}" "${a:-UNSIGNED}"

# done | sort -u
```

## † H5 — Recently Modified Persistence Artifacts

```bash
find /Library/Launch{Agents,Daemons} /Users/*/Library/LaunchAgents \

       /Library/Security/SecurityAgentPlugins /Library/ScriptingAdditions \

       /etc/periodic /etc/sudoers.d -type f -mtime -14 2>/dev/null \

  | while read f; do printf '%s  %s\n' "$(stat -f '%Sm' -t '%FT%T' "$f")" "$f"; done | sort
```

## † H6

TCC grants to non-Apple apps for dangerous services

```bash
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service, client, auth_reason, datetime(last_modified,'unixepoch')
   FROM access
   WHERE auth_value = 2
     AND service IN ('kTCCServiceAccessibility','kTCCServiceSystemPolicyAllFiles',
                     'kTCCServiceScreenCapture','kTCCServiceListenEvent',
                     'kTCCServicePostEvent','kTCCServiceDeveloperTool')
     AND client NOT LIKE 'com.apple.%'
   ORDER BY last_modified DESC;"
```

## † H7

osascript credential-prompt pattern (infostealers)

```bash
log show --last 30d --predicate 'process == "osascript"' --info --style compact | \
    grep -iE 'display dialog|password|hidden answer' | head -30

grep -rlE 'display dialog.*(password|hidden answer)' /Users /tmp /var/tmp 2>/dev/null |
head

grep -rlE 'do shell script.*with administrator privileges' /Users /tmp 2>/dev/null | head

Nearly every current macOS stealer (Atomic/AMOS, MetaStealer, Poseidon) uses

`display dialog ... with hidden answer` to phish the login password rather than attacking

the keychain crypto. This is one of the highest-yield hunts on macOS right now.
```

## † H8 — CURL/WGET PIPED TO A SHELL

BASH

```
grep -rE '(curl|wget)[^|]*\|\s*(ba)?sh' ~/.zsh_history ~/.bash_history 2>/dev/null

log show --last 30d --predicate 'process == "curl" OR process == "wget"' --info | head
-40

# eslogger exec | jq -r 'select(.event.exec.args | join(" ") | test("curl|wget")) |
[.time, (.event.exec.args|join(" "))] | @tsv'
```

## † H9

Recently added root CAs / trust overrides

BASH

```
security dump-trust-settings -d 2>/dev/null

security dump-trust-settings 2>/dev/null

log show --last 30d --predicate 'eventMessage CONTAINS "trust settings"' --info

stat -f '%Sm %N' -t '%FT%T' /Library/Keychains/System.keychain
```

## † H10

Quarantine bypass / manually cleared xattrs

BASH

```
grep -rE 'xattr\s+(-[a-z]+\s+)*-d\s+com\.apple\.quarantine|xattr\s+-cr?' \

    ~/.zsh_history ~/.bash_history /Users/*/.zsh_history 2>/dev/null

log show --last 30d --predicate 'process == "xattr"' --info | head -20

# App in /Applications with no quarantine xattr but recent birthtime

for a in /Applications/*.app; do

    bt=$(stat -f %B "$a"); now=$(date +%s)

    (( now - bt < 2592000 )) || continue

    xattr "$a" 2>/dev/null | grep -q quarantine || print "NO QUARANTINE (recent): $a"

  done
```

## † Hıı — Login Items Added Recently

```bash
# sfltool dumpbtm 2>/dev/null | grep -E 'Name:|Developer Name:|Disposition:|Generation:'

log show --last 30d --predicate 'subsystem == "com.apple.sharedfilelist"' --info | head -40

# eslogger btm_launch_item_add | jq -r '[.time, .event.btm_launch_item_add.item.item_url] | @tsv'

ls -laT ~/Library/Application\ Support/com.apple.backgroundtaskmanagementagent/
```

## † Hı2

Suspicious AppleScript / Automator persistence

```bash
ls -la ~/Library/Services/ /Library/Services/ 2>/dev/null

ls -la ~/Library/Workflows/Applications/Folder\ Actions/ 2>/dev/null

ls -la ~/Library/Scripts/ /Library/Scripts/ 2>/dev/null

find /Users -name '*.scpt' -o -name '*.applescript' -o -name '*.workflow' 2>/dev/null | head -30

for f in $(find /Users -name '*.scpt' 2>/dev/null | head -20); do

    echo "=== $f"; osadecompile "$f" 2>/dev/null | head -20; done

`osadecompile` reverses compiled AppleScript

 essential when you find a `.scpt` you didn't

write.
```

## † H13 — DYLIB INSERTION

```bash
launchctl getenv DYLD_INSERT_LIBRARIES

for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do

    plutil -p "$p" 2>/dev/null | grep -qi 'DYLD_' && { echo "═══ $p"; plutil -p "$p" |
grep -i -A2 DYLD; }

  done

grep -rl 'DYLD_INSERT_LIBRARIES' ~/.zshrc ~/.zshenv ~/.bash_profile /etc/zshenv /etc/
profile 2>/dev/null

# for pid in $(pgrep -d' ' -x ''); do launchctl procinfo $pid 2>/dev/null | grep -i dyld;
done
```

## † H14 — UNEXPECTED LISTENING SERVICES

```bash
# lsof -i -n -P | grep LISTEN | while read cmd pid user fd type dev size node name; do

#   b=$(ps -p "$pid" -o comm=)

#   a=$(codesign -dv "$b" 2>&1 | awk -F= '/^Authority/{print $2; exit}')

#   printf '%-20s %-8s %-25s %s\n' "$cmd" "$pid" "$name" "${a:-UNSIGNED}"

# done

Baseline for a clean workstation: essentially nothing listening on external interfaces

except what you deliberately enabled. Loopback-only listeners are normal.
```

# Only non-loopback listeners # lsof -i -n -P | grep LISTEN | grep -v '127\.0\.0\.1\|\[::1\]'

## † H15

Timeline of everything that changed in a window

BASH

```bash
#!/bin/zsh

# what-changed.zsh <days>

D=${1:-7}

print "═══ Files modified in last ${D}d (system dirs)"

sudo find /Library /usr/local /opt /etc -type f -mtime -${D} \

  -not -path '*/Caches/*' -not -path '*/Logs/*' 2>/dev/null | head -100

print "\n═══ Package receipts"

find /var/db/receipts -name '*.plist' -mtime -${D} 2>/dev/null

print "\n═══ install.log"

sudo grep -iE 'Installed|Upgraded' /var/log/install.log 2>/dev/null | tail -30

print "\n═══ Profiles"

sudo log show --last ${D}d --predicate 'process == "profiles"' --info 2>/dev/null | head
-20

print "\n═══ New user accounts"

find /var/db/dslocal/nodes/Default/users -name '*.plist' -mtime -${D} 2>/dev/null

print "\n═══ Downloads"

find /Users/*/Downloads -type f -mtime -${D} 2>/dev/null | head -50
```

## † Hi6

Cross-reference running processes against VirusTotal

BASH

```
#!/bin/zsh

# Requires a VT API key. Sends hashes only, not files.

VT_KEY="${VT_API_KEY:?set VT_API_KEY}"

ps -axo comm= | sort -u | while read b; do

  [[ -f $b ]] || continue

  codesign -dv "$b" 2>&1 | grep -q 'Apple' && continue     # skip Apple-signed

  h=$(shasum -a 256 "$b" | awk '{print $1}')

  r=$(curl -s -H "x-apikey: $VT_KEY" "https://www.virustotal.com/api/v3/files/$h")

  m=$(print -r -- "$r" | jq -r '.data.attributes.last_analysis_stats.malicious //
"unknown"')

  [[ $m == "0" || $m == "unknown" ]] || printf '%-6s %-60s %s\n' "$m" "${b:t}" "$h"

  sleep 15   # public API rate limit

done

Public VT keys are 4 req/min. For anything at scale you need a paid key.
```

## † H17 — BROWSER EXTENSION AUDIT

BASH

```
for m in ~/Library/Application\ Support/Google/Chrome/*/Extensions/*/*/manifest.json; do

    n=$(jq -r '.name' "$m" 2>/dev/null)

    p=$(jq -r '(.permissions // []) + (.host_permissions // []) | join(",")' "$m" 2>/dev/
null)

    print -r -- "$n :: $p"

  done | grep -iE '<all_urls>|http://\*|https://\*|webRequest|cookies|nativeMessaging|
debugger'

`nativeMessaging` + `<all_urls>` is the combination worth looking at hardest

 it bridges

the browser to a local binary.
```

## † Hɪ8

Anomalous parent-child process relationships

```bash
# eslogger exec | jq -r '
#   [.time,
#    .process.ppid,
#    (.process.parent_audit_token // {} | tostring),
#    .event.exec.target.executable.path,
#    (.event.exec.args | join(" "))] | @tsv'

Watch for: Office apps or browsers spawning `osascript`/`sh`/`curl`; `launchd` spawning
something from `/tmp`; any shell whose parent is a document handler.
```

## † Hɪ9

Persistence baseline diff (run periodically)

BASH

```
#!/bin/zsh

BASE=~/.security-baseline

mkdir -p "$BASE"

snap() {

  find /Library/Launch{Agents,Daemons} /Users/*/Library/LaunchAgents -type f 2>/dev/null
| sort > "$1/launch.txt"

  sudo sfltool dumpbtm 2>/dev/null | grep '^\s*Name:' | sort > "$1/btm.txt"

  sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db 'SELECT
service,client,auth_value FROM access;' 2>/dev/null | sort > "$1/tcc.txt"

  ls /Applications > "$1/apps.txt"

  sudo profiles list 2>/dev/null | sort > "$1/profiles.txt"

  security dump-trust-settings -d 2>/dev/null > "$1/trust.txt"

  crontab -l 2>/dev/null > "$1/cron.txt"

  ls /etc/sudoers.d/ 2>/dev/null > "$1/sudoers.txt"

  systemextensionsctl list 2>/dev/null > "$1/sysext.txt"

if [[ $1 == init ]]; then

  snap "$BASE"; print "Baseline written to $BASE"

else

  T=$(mktemp -d); snap "$T"

  for f in "$BASE"/*.txt; do

    d=$(diff "$f" "$T/${f:t}" 2>/dev/null) && continue

    print "\n### CHANGED: ${f:t:r}"; print -r -- "$d"

  done

  rm -rf "$T"
```

BASH

```
./baseline.zsh init      # once, on a known-good system

./baseline.zsh           # thereafter
```

## † H20 — CRASH REPORTS AS AN IOC SOURCE

```bash
for f in ~/Library/Logs/DiagnosticReports/*.ips /Library/Logs/DiagnosticReports/*.ips; do
    [[ -f $f ]] || continue
    head -1 "$f" | jq -r '[.timestamp, .app_name, .bundleID // ""] | @tsv' 2>/dev/null
  done | sort | tail -40

# Crashes from unusual paths
grep -l '/tmp/\|/var/folders/' ~/Library/Logs/DiagnosticReports/*.ips 2>/dev/null
```

## † H21

Login window / screensaver bypass artifacts

```bash
defaults read com.apple.screensaver 2>/dev/null

defaults read /Library/Preferences/com.apple.loginwindow 2>/dev/null

ls -la /etc/kcpassword 2>/dev/null && print "!! AUTOLOGIN"

defaults read /Library/Preferences/com.apple.loginwindow autoLoginUser 2>/dev/null

ls -la /Library/Security/SecurityAgentPlugins/
```

## † H22 — DATA STAGING / EXFIL PREP

```bash
find /Users /tmp /var/tmp -type f \( -name '*.zip' -o -name '*.tar*' -o -name '*.7z' -o -name '*.rar' \) -mtime -7 -size +10M 2>/dev/null

find /Users -type d -name '.*' -mtime -7 -maxdepth 4 2>/dev/null | head -30

# lsof -c curl -c rsync -c scp -c rclone -n -P 2>/dev/null

log show --last 7d --predicate 'process IN {"rsync","scp","rclone","curl","ftp"}' --info | head -30

ls -la ~/Library/CloudStorage/ 2>/dev/null
```

## † PRIORITIZATION

Not everything above deserves equal attention. In rough order of signal-to-noise on a typical macOS endpoint: 1. **H7** (osascript password prompt) very high signal, low FP 2. **H1** (unsigned persistence) high signal 3. **H6** (TCC grants) high signal, moderate FP from legit tools 4. **H2** (StartInterval beacons) moderate; many legit updaters do this 5. **H9** (root CAs) low FP; usually corporate MITM or nothing 6. **H13** (dylib insertion) high signal, rare 7. **H11** (login items) moderate FP 8. **H3/H4** (process paths, unsigned network) noisy, but good for scoping

BOOK VI

# Unified Log & Endpoint Security Telemetry

*canticle of the 3rd sheet*

✦ ⁂ ✦

The unified log is the single richest defensive data source on macOS. Most of it is `<private>` redacted by default that limitation drives a lot of the workarounds below.

## † Basic Syntax

BASH

```
log show --last 1h

log show --last 30m --style compact

log show --start "2026-07-20 09:00:00" --end "2026-07-20 17:00:00"

log stream                                  # live tail

log stream --style ndjson | jq .            # structured live

log show --info --debug --last 10m          # include info+debug (much larger)

log show --last 1d --archive /path/x.logarchive

Styles: `default`, `compact`, `json`, `ndjson`, `syslog`.
```

## † Collecting an Archive

BASH

```
# log collect --last 24h --output ~/host.logarchive

# log collect --start "2026-07-01" --output ~/july.logarchive

log show --archive ~/host.logarchive --predicate '...' --info --debug

`log collect` requires root and includes info/debug that `log show` on a live system may

have already rotated out. Collect early in an investigation.
```

## † PREDICATE LANGUAGE

Fields: `process`, `processImagePath`, `sender`, `senderImagePath`, `subsystem`, `category`, `eventMessage`, `eventType`, `messageType`, `processIdentifier`, `parentProcessIdentifier`, `traceID`, `threadIdentifier`. Operators: `==` `≠` `<` `>` `CONTAINS` `BEGINSWITH` `ENDSWITH` `MATCHES` (regex) `IN` `AND` `OR` `NOT`. Add `[c]` for case-insensitive, `[d]` for diacritic-insensitive.

BASH

```
log show --predicate 'process == "sshd"' --last 1d

log show --predicate 'subsystem == "com.apple.securityd"' --last 1h

log show --predicate 'eventMessage CONTAINS[c] "authentication"' --last 1d

log show --predicate 'process IN {"sudo","su","login"}' --last 1d

log show --predicate 'eventMessage MATCHES ".*fail(ed|ure).*"' --last 1h

log show --predicate 'processImagePath CONTAINS "/tmp/"' --last 1d

log show --predicate 'messageType == error OR messageType == fault' --last 1h
```

## † High-Value Subsystems

| Subsystem | What it tells you |
|---|---|
| `com.apple.securityd` | Keychain, code signing, trust evaluation |
| `com.apple.opendirectoryd` | Account creation, auth, group changes |
| `com.apple.TCC` | Privacy grant requests and decisions |
| `com.apple.xpc.launchd` | Service load/unload, job spawns |
| `com.apple.sharedfilelist` | Login item registration |
| `com.apple.network` | Connection establishment |
| `com.apple.mDNSResponder` | DNS queries |
| `com.apple.LaunchServices` | App launches, first-run |
| `com.apple.syspolicy` | Gatekeeper, notarization, quarantine |
| `com.apple.kernel` | Kext load, IOKit |
| `com.apple.SystemConfiguration` | Network config changes |
| `com.apple.duetactivityscheduler` | Background task scheduling |
| `com.apple.ManagedClient` | MDM / profile activity |
| `com.apple.SecurityAgent` | Auth prompts |
| `com.apple.CoreServices.uninstalld` | App removal |

## † Ready-Made Hunt Queries

### ▸ Process Execution

BASH

```
log show --last 1h --predicate 'subsystem == "com.apple.xpc.launchd" AND eventMessage CONTAINS "xpcproxy"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "Service exited"' --style compact

log show --last 1h --predicate 'process == "kernel" AND eventMessage CONTAINS "proc"' --info
```

## CODE SIGNING / GATEKEEPER

BASH

```
log show --last 1d --predicate 'subsystem == "com.apple.syspolicy"' --info --style compact

log show --last 1d --predicate 'eventMessage CONTAINS "Gatekeeper"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "notariz"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "quarantine"' --info

log show --last 1d --predicate 'subsystem == "com.apple.securityd" AND eventMessage CONTAINS "code sign"' --info
```

## AUTHENTICATION

BASH

```
log show --last 1d --predicate 'process == "sudo"' --info

log show --last 1d --predicate 'process == "sshd"' --info

log show --last 7d --predicate 'subsystem == "com.apple.opendirectoryd" AND eventMessage CONTAINS[c] "create"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "Authentication failed"' --info

log show --last 1d --predicate 'process == "loginwindow"' --info

log show --last 1d --predicate 'process == "SecurityAgent"' --info
```

## TCC / PRIVACY

BASH

```
log show --last 1d --predicate 'subsystem == "com.apple.TCC"' --info --style compact

log show --last 1d --predicate 'subsystem == "com.apple.TCC" AND eventMessage CONTAINS "kTCCService"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "ScreenCapture" OR eventMessage CONTAINS "Accessibility"' --info
```

### ▸ NETWORK

BASH

```
log show --last 1h --predicate 'process == "mDNSResponder"' --info | grep -i query

log show --last 1h --predicate 'subsystem == "com.apple.network"' --info --style compact

log show --last 1d --predicate 'process == "networkserviceproxy" OR process ==
"nesessionmanager"' --info

log show --last 1d --predicate 'eventMessage CONTAINS "VPN"' --info
```

### ▸ PERSISTENCE EVENTS

BASH

```
log show --last 7d --predicate 'subsystem == "com.apple.sharedfilelist"' --info

log show --last 7d --predicate 'eventMessage CONTAINS "LaunchDaemon" OR eventMessage
CONTAINS "LaunchAgent"' --info

log show --last 7d --predicate 'process == "sfltool" OR process ==
"backgroundtaskmanagementd"' --info

log show --last 7d --predicate 'eventMessage CONTAINS "btm" OR eventMessage CONTAINS
"BackgroundItem"' --info
```

### ▸ USB / REMOVABLE MEDIA

BASH

```
log show --last 7d --predicate 'eventMessage CONTAINS "IOUSBHostDevice" OR eventMessage
CONTAINS "USBMSC"' --info

log show --last 7d --predicate 'process == "diskarbitrationd"' --info

log show --last 7d --predicate 'eventMessage CONTAINS "mount"' --info | grep -i volume
```

### ▸ PROFILE / MDM

BASH

```
log show --last 30d --predicate 'subsystem == "com.apple.ManagedClient"' --info

log show --last 30d --predicate 'process == "profiles" OR process == "mdmclient"' --info
```

## † DEALING WITH `<private>`

Apple redacts most string interpolations. Options:

```bash
# 1. Install a logging profile that enables Private-Data (requires profile install)
#    Payload: com.apple.system.logging with Enable-Private-Data = true
# 2. Per-subsystem override
# log config --mode "private_data:on" --subsystem com.apple.TCC
# log config --status --subsystem com.apple.TCC
# 3. Revert
# log config --mode "private_data:off" --subsystem com.apple.TCC
Enabling private data globally is a privacy risk. Scope to the subsystem you need, do it
for the duration of the investigation, and revert.
```

# Raise persistence/level for a subsystem while hunting # log config --mode "level:debug,persist:debug" --subsystem com.apple.securityd # log config --reset --subsystem com.apple.securityd

## † ENDPOINT SECURITY FRAMEWORK EVENTS (ES)

Full ES access needs a signed client with the `com.apple.developer.endpoint-security.client` entitlement. On-host proxies:

```bash
eslogger --list-events                  # macOS 13+, root required

# eslogger exec fork exit | jq .

# eslogger exec | jq -r '[.time, .process.executable.path, (.event.exec.args|join(" "))]
| @tsv'

# eslogger create unlink rename write | jq .

# eslogger auth_mount mount unmount | jq .

# eslogger uipc_connect uipc_bind | jq .

# eslogger btm_launch_item_add | jq .      # persistence, macOS 13+

# eslogger login_login login_logout screensharing_attach | jq .

# eslogger su sudo authentication | jq .

# eslogger xp_malware_detected xp_malware_remediated | jq .   # XProtect Remediator

# eslogger kextload profile_add | jq .

`eslogger` is the single best free ES telemetry source on a modern Mac. Pipe it to a file

and let it run during dynamic analysis.
```

# Capture a full exec timeline during a suspicious window # eslogger exec fork exit create unlink rename > ~/es-$(date +%s).ndjson

## ▸ USEFUL ESLOGGER EVENT NAMES

`exec` `fork` `exit` `create` `open` `close` `write` `unlink` `rename` `link` `truncate` `clone` `mount` `unmount` `remount` `setmode` `setowner` `setflags` `setextattr` `deleteextattr` `signal` `kextload` `kextunload` `iokit_open` `setuid` `setgid` `setegid` `seteuid` `authentication` `su` `sudo` `od_group_add` `od_create_user` `od_modify_password` `btm_launch_item_add` `btm_launch_item_remove` `profile_add` `profile_remove` `xp_malware_detected` `xp_malware_remediated` `gatekeeper_user_override` `screensharing_attach` `screensharing_detach` `openssh_login` `openssh_logout` `login_login` `login_logout` `lw_session_lock` `lw_session_unlock` `tcc_modify` (macOS 14+)

## † LEGACY LOGS STILL WORTH READING

BASH

```
ls -la /var/log/

sudo cat /var/log/install.log | tail -200        # package installs

 very useful

grep -i 'Installed\|Upgraded' /var/log/install.log | tail -50

sudo cat /var/log/system.log 2>/dev/null         # mostly empty on modern macOS

ls /Library/Logs/DiagnosticReports/              # crash reports

ls ~/Library/Logs/DiagnosticReports/

sudo ls /var/db/diagnostics/                     # raw tracev3 files

sudo ls /var/db/uuidtext/

Crash reports are underrated

 a crashing implant leaves a `.ips` with full backtrace and

binary path.
```

BASH

```
for f in ~/Library/Logs/DiagnosticReports/*.ips; do

    echo "=== $f"; head -1 "$f" | jq -r '.app_name, .timestamp' 2>/dev/null; done
```

## † AUDIT SUBSYSTEM (BSM) — DEPRECATED BUT PRESENT

BASH

```
sudo praudit /var/audit/current | head -50

sudo praudit -l /var/audit/current | grep -i exec

sudo auditreduce -c ex /var/audit/current | praudit -l

cat /etc/security/audit_control

Deprecated since macOS 11 and off by default in recent builds; verify before relying on
it.
```

## † LOG EXPORT FOR SIEM

```bash
log show --last 1h --style ndjson --predicate 'subsystem == "com.apple.TCC"' > tcc.ndjson

log stream --style ndjson --predicate 'process == "sudo"' | while read -r l; do

    curl -s -X POST -H 'Content-Type: application/json' -d "$l" http://siem:8088/
services/collector; done
```

## † PERFORMANCE NOTES

- `--info --debug` on a full day can be tens of GB. Scope with predicates first. - `log show` is single-threaded and slow on large archives; filter at the predicate, not with `grep`. - Live `log stream` drops messages under load. For high-fidelity capture use `log collect` after the fact, or `eslogger`.

BOOK VII

# LAUNCHD & LAUNCHCTL

*canticle of the 13th sheet*

✦ ⁂ ✦

Launchd is PID 1 and the init/service manager. `launchctl` syntax changed substantially in macOS 10.10; most tutorials online still show the deprecated form.

## † DOMAINS

| DOMAIN | TARGET SYNTAX | SCOPE |
|---|---|---|
| System | `system/<label>` | root daemons, all users |
| User | `user/<uid>/<label>` | per-user, no GUI |
| GUI | `gui/<uid>/<label>` | per-user login session |
| Login | `login/<asid>/<label>` | login session |
| Session | `session/<asid>/<label>` | audit session |
| PID | `pid/<pid>/<label>` | single process |

BASH

```
id -u                                  # your uid

launchctl print system

launchctl print gui/$(id -u)

launchctl print user/$(id -u)

launchctl print system/com.apple.sshd

launchctl print gui/$(id -u)/com.example.agent

launchctl print pid/<pid>
```

## † Modern verbs (10.10+)

BASH

```
# launchctl bootstrap system /Library/LaunchDaemons/com.example.plist

# launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.example.plist

# launchctl bootout system/com.example

# launchctl bootout gui/$(id -u)/com.example

# launchctl enable system/com.example

# launchctl disable system/com.example

# launchctl kickstart -k system/com.example      # restart

# launchctl kickstart -kp system/com.example     # restart, print pid

# launchctl kill SIGTERM system/com.example

# launchctl kill SIGKILL gui/$(id -u)/com.example

# launchctl blame system/com.example             # why is this running?

# launchctl print-disabled system

# launchctl print-disabled gui/$(id -u)

# launchctl dumpstate                            # everything, huge

# launchctl procinfo <pid>

# launchctl examine <pid>                        # attach heap analysis
```

`launchctl blame` answers "why did this service start"

**genuinely** useful and little known.

## † LEGACY VERBS (STILL WORK, DEPRECATED)

```bash
launchctl list

launchctl list <label>

# launchctl load -w /Library/LaunchDaemons/x.plist

# launchctl unload -w /Library/LaunchDaemons/x.plist

# launchctl start <label>

# launchctl stop <label>

# launchctl remove <label>

`load -w` / `unload -w` write to the disabled DB (`/var/db/com.apple.xpc.launchd/`),
which

is why a service can refuse to load even when the plist looks correct.
```

# plutil -p /var/db/com.apple.xpc.launchd/disabled.plist # plutil -p /var/db/com.apple.xpc.launchd/disabled.$(id -u).plist # launchctl print-disabled system | grep -v 'false'

## † READING `launchctl list` OUTPUT

PID Status Label 1234 0 com.example.agent running, last exit 0 - 0 com.example.other not running, exited cleanly - 1 com.example.broken not running, exited 1 - 78 com.example.err not running, error 78 (config error) Status is the last exit code, or a negative number for the signal that killed it.

## † PLIST ANATOMY

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.example.watcher</string>
  <key>ProgramArguments</key>
  <array>
    <string>/usr/local/bin/watcher</string>
    <string>--daemon</string>
  </array>
  <key>RunAtLoad</key>            <true/>
  <key>KeepAlive</key>            <true/>
  <key>StartInterval</key>        <integer>300</integer>
  <key>StandardOutPath</key>      <string>/var/log/watcher.log</string>
  <key>StandardErrorPath</key>    <string>/var/log/watcher.err</string>
  <key>UserName</key>             <string>root</string>
  <key>WorkingDirectory</key>     <string>/usr/local/var</string>
  <key>ProcessType</key>          <string>Background</string>
  <key>Nice</key>                 <integer>5</integer>
</dict>
</plist>
```

## KEYS THAT MATTER FOR HUNTING

| KEY | MEANING | HUNT RELEVANCE |
|---|---|---|
| `RunAtLoad` | Start on load/boot | persistence |
| `KeepAlive` | Restart on exit | persistence, resilience |
| `StartInterval` | Every N seconds | **beaconing** |
| `StartCalendarInterval` | Cron-like schedule | scheduled tasks |
| `WatchPaths` | Trigger on path change | trigger-based persistence |
| `QueueDirectories` | Trigger on non-empty dir | same |
| `StartOnMount` | Trigger on volume mount | USB-triggered |
| `EnvironmentVariables` | Env for the job | **`DYLD_INSERT_LIBRARIES`** |
| `LimitLoadToSessionType` | Aqua/Background/LoginWindow | |
| `ProgramArguments` | The actual command | what runs |
| `Program` | Single executable | |
| `Sockets` | launchd-managed listeners | network service |
| `MachServices` | XPC endpoints | IPC surface |
| `AssociatedBundleIdentifiers` | macOS 13+ BTM attribution | shows in Login Items UI |
| `UserName` / `GroupName` | Run as | privilege |
| `SessionCreate` | New security session | |

BASH

```
# Hunt: anything with StartInterval or WatchPaths
for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do

for k in StartInterval WatchPaths QueueDirectories StartOnMount EnvironmentVariables; do
      v=$(/usr/libexec/PlistBuddy -c "Print :$k" "$p" 2>/dev/null) && echo "$p :: $k =
$v"
    done
  done
```

## † VALIDATING A PLIST

```bash
plutil -lint /Library/LaunchDaemons/x.plist

plutil -p /Library/LaunchDaemons/x.plist

plutil -convert xml1 -o - /Library/LaunchDaemons/x.plist

plutil -convert binary1 x.plist

/usr/libexec/PlistBuddy -c 'Print' x.plist

/usr/libexec/PlistBuddy -c 'Print :ProgramArguments' x.plist
```

## † PERMISSIONS REQUIREMENTS

launchd refuses plists with wrong ownership/perms:

```bash
# chown root:wheel /Library/LaunchDaemons/x.plist

# chmod 644 /Library/LaunchDaemons/x.plist

ls -la /Library/LaunchDaemons/ | awk '$3≠"root" || $4≠"wheel"'

find /Library/Launch{Agents,Daemons} ! -user root -o ! -group wheel 2>/dev/null

find /Library/Launch{Agents,Daemons} -perm -o+w 2>/dev/null    # world-writable =
hijackable

A world-writable plist or a plist pointing at a world-writable binary is a privilege

escalation primitive. Worth a specific check:
```

```bash
for p in /Library/LaunchDaemons/*.plist; do

    b=$(/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null)

    [[ -n $b && -w $b ]] && echo "WRITABLE TARGET: $b  ← $p"

  done
```

## † DEBUGGING A FAILING JOB

```bash
# launchctl print system/com.example | sed -n '1,60p'

#

 look at: state, last exit code, path, program, arguments, runs

# launchctl blame system/com.example

log show --last 1h --predicate 'subsystem == "com.apple.xpc.launchd"' --info | grep com.example

log stream --predicate 'eventMessage CONTAINS "com.example"' --info

# launchctl kickstart -kp system/com.example

Common exit codes: `78` = config error (bad plist), `126` = not executable,

`127` = not found, `1` = generic, `-9` = SIGKILL.
```

## † ENVIRONMENT

```bash
launchctl getenv PATH

# launchctl setenv PATH /usr/local/bin:/usr/bin:/bin

# launchctl unsetenv <VAR>

launchctl export                        # env in shell-eval format

launchctl managername

launchctl managerpid

launchctl asuser $(id -u) launchctl getenv PATH    # run in a user's context
```

## † Useful Service Targets

```bash
# launchctl print system | grep -oE 'com\.apple\.[a-zA-Z0-9._-]+' | sort -u | head -50

# launchctl list | grep -v com.apple      # third-party only

launchctl list | awk '$1 ≠ "-" {print}' | head -40    # currently running

launchctl list | awk '$2 ≠ "0" && $2 ≠ "-" {print}'  # abnormal exits
```

## † Writing a Monitoring Agent (Example)

```xml
<!— ~/Library/LaunchAgents/local.persistence-watch.plist -->

<dict>

  <key>Label</key><string>local.persistence-watch</string>

  <key>ProgramArguments</key>

  <array>

    <string>/usr/local/bin/persistence-watch.sh</string>

  </array>

  <key>WatchPaths</key>

  <array>

    <string>/Library/LaunchDaemons</string>

    <string>/Library/LaunchAgents</string>

  </array>

  <key>RunAtLoad</key><true/>

</dict>
```

#!/bin/zsh # /usr/local/bin/persistence-watch.sh STATE=~/.persistence-baseline.sha NOW=$(mktemp) find /Library/Launch{Agents,Daemons} ~/Library/LaunchAgents -type f -exec shasum -a 256 {} \; 2>/dev/null | sort -k2 > "$NOW" if [[ -f $STATE ]]; then diff "$STATE" "$NOW" > /tmp/persistence-diff.txt && rm -f /tmp/persistence-diff.txt || \ osascript -e 'display notification "Launch item change detected" with title "Persistence Watch"' cp "$NOW" "$STATE"; rm -f "$NOW"

```bash
launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/local.persistence-watch.plist

launchctl print gui/$(id -u)/local.persistence-watch

Note `WatchPaths` fires on directory mtime change, which covers file creation/deletion but

not in-place edits to an existing plist. Pair it with the hash comparison as above.
```

## † LEGACY STARTUPITEMS

```bash
ls -la /Library/StartupItems/ /System/Library/StartupItems/ 2>/dev/null

cat /Library/StartupItems/*/StartupParameters.plist 2>/dev/null

Removed in modern macOS but the directories may still exist on upgraded systems

 check

them during triage.
```

BOOK VIII

# CONFIGURATION PROFILES & MDM

*canticle of the 10th sheet*

✦ ⁂ ✦

Profiles are the legitimate admin control plane and therefore a high-value attacker target. A malicious profile can silently grant TCC permissions, install root CAs, set proxies, and install persistent payloads.

## † INSPECT INSTALLED PROFILES

BASH

```
profiles list                              # user profiles

# profiles list -all

# profiles -P                                # show all, verbose (older syntax)

# profiles show

# profiles show -type enrollment

# profiles show -type configuration

# profiles status -type enrollment

# profiles status -type bootstraptoken

system_profiler SPConfigurationProfileDataType

system_profiler SPConfigurationProfileDataType -json | jq .
```

## † PROFILE STORAGE

BASH

```
ls -la /var/db/ConfigurationProfiles/

ls -la /var/db/ConfigurationProfiles/Store/

ls -la /var/db/ConfigurationProfiles/Settings/

# plutil -p /var/db/ConfigurationProfiles/Store/ConfigProfiles.binary 2>/dev/null | head
-100

# sqlite3 /var/db/ConfigurationProfiles/Store/*.db '.tables' 2>/dev/null

ls -la ~/Library/ConfigurationProfiles/ 2>/dev/null
```

## † INSTALLING / REMOVING

BASH

```
# profiles install -path /path/to/profile.mobileconfig

# profiles install -path x.mobileconfig -user <user>

# profiles remove -identifier <payload-identifier>

# profiles remove -path /path/to/profile.mobileconfig

# profiles remove -all                    #

# profiles renew -type enrollment

Since macOS 11, manually installing a profile from CLI generally requires the user to

approve it in System Settings

 silent CLI install only works for MDM-delivered or

DEP/ADE-supervised profiles. That's a meaningful defensive property.
```

## † READING A .MOBILECONFIG

```bash
plutil -p profile.mobileconfig

plutil -convert xml1 -o - profile.mobileconfig

security cms -D -i profile.mobileconfig  # if signed, extract payload

plutil -p profile.mobileconfig | grep -E 'PayloadType|PayloadIdentifier|
PayloadDisplayName'
```

### ▸ PAYLOAD TYPES WORTH FLAGGING

| PAYLOADTYPE | WHY IT MATTERS |
|---|---|
| `com.apple.TCC.configuration-profile-policy` | **Silently grants TCC permissions** |
| `com.apple.security.root` / `.pkcs1` / `.pem` | Installs a root CA |

MITM |

| `com.apple.security.scep` / `.acme` | CERT ENROLLMENT |
|---|---|
| `com.apple.dnsSettings.managed` | Forces DNS (incl. DoH) server |
| `com.apple.proxy.http.global` | Global proxy |
| `com.apple.webcontent-filter` | NetworkExtension content filter |

sees all traffic |

| com.apple.vpn.managed | VPN CONFIG, INCL. ALWAYS-ON |
|---|---|
| com.apple.MCX | Arbitrary managed preferences |
| com.apple.ManagedClient.preferences | Same |
| com.apple.system-extension-policy | Pre-approves system extensions |
| com.apple.syspolicy.kernel-extension-policy | Pre-approves kexts |
| com.apple.servicemanagement | Whitelists login items / daemons |
| com.apple.applicationaccess | Restrictions |
| com.apple.SoftwareUpdate | Can defer/block updates |
| com.apple.loginwindow | Login/logout scripts |
| com.apple.systemuiserver | UI changes |

BASH

```
# Hunt for the dangerous ones in installed profiles

# profiles -P -o stdout-xml 2>/dev/null | plutil -p - | \

#   grep -E 'com\.apple\.(TCC|security\.root|dnsSettings|proxy|webcontent-filter|vpn|
servicemanagement)'

system_profiler SPConfigurationProfileDataType | grep -B3 -A10 -iE 'TCC|root certificate|
proxy|content filter'
```

## † MDM ENROLLMENT STATE

```bash
# profiles status -type enrollment

#

 "Enrolled via DEP: Yes/No"  "MDM enrollment: Yes (User Approved)"

# profiles show -type enrollment

sudo /usr/libexec/mdmclient QuerySecurityInfo 2>/dev/null

sudo /usr/libexec/mdmclient QueryInstalledProfiles 2>/dev/null

sudo /usr/libexec/mdmclient QueryDeviceInformation 2>/dev/null

sudo /usr/libexec/mdmclient dumpSCEP 2>/dev/null

sudo /usr/libexec/mdmclient availableOSUpdates 2>/dev/null

log show --last 7d --predicate 'subsystem == "com.apple.ManagedClient"' --info | head -80

`mdmclient` is undocumented but extremely useful for verifying what the MDM server can

actually do to the box.
```

### ▸ KEY DISTINCTIONS

- **User Approved MDM (UAMDM)** user clicked approve; enables kext/TCC payloads - **DEP / ADE (Automated Device Enrollment)** supervised, enrollment is non-removable - **Manual enrollment** removable by the user

```bash
# profiles status -type enrollment

# profiles show -type enrollment | grep -iE 'supervised|user approved|dep'
```

## † SUPERVISION

```bash
sudo profiles status -type enrollment | grep -i supervis

system_profiler SPConfigurationProfileDataType | grep -i supervis

ioreg -l | grep -i 'IOPlatformSerialNumber'
```

## † BOOTSTRAP TOKEN & SECURE TOKEN (FILEVAULT + MDM)

BASH

```
# profiles status -type bootstraptoken

# profiles install -type bootstraptoken

sysadminctl -secureTokenStatus <user>

diskutil apfs listUsers /

# sysadminctl -secureTokenOn <user> -password <pw> -adminUser <admin> -adminPassword
<apw>

Without a bootstrap token escrowed to MDM, remote FileVault-enabled account management

breaks. Worth checking on every managed Mac.
```

## † REMOVING A STUBBORN PROFILE

BASH

```
# profiles remove -identifier <id>

# profiles remove -all -forced             #

 may need recovery

# In Recovery (if DEP-locked, this generally won't help

 that's by design)

A DEP/ADE-enrolled machine cannot be un-enrolled locally. If you're doing IR and find

unexpected DEP enrollment, that's an Apple Business/School Manager account compromise

question, not a local one.
```

## † PROFILE-RELATED HUNTING

BASH

```
# When was each profile installed?

ls -laT /var/db/ConfigurationProfiles/Store/

log show --last 30d --predicate 'process == "profiles" OR process == "mdmclient"' --info
| grep -iE 'install|add|remove'

# eslogger profile_add profile_remove | jq .

# Root CAs added by profile

security dump-trust-settings -d

security find-certificate -a -p /Library/Keychains/System.keychain | \

    while openssl x509 -noout -subject -dates 2>/dev/null; do :; done

# TCC entries with auth_reason=6 (MDM policy)

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

    "SELECT service, client FROM access WHERE auth_reason=6;"

plutil -p /Library/Application\ Support/com.apple.TCC/MDMOverrides.plist 2>/dev/null
```

## † MANAGED PREFERENCES (MCX)

BASH

```
sudo ls -la /Library/Managed\ Preferences/

sudo ls -la /Library/Managed\ Preferences/<user>/

for f in /Library/Managed\ Preferences/*.plist; do echo "=== $f"; plutil -p "$f"; done

defaults read /Library/Managed\ Preferences/com.apple.SoftwareUpdate 2>/dev/null

sudo mcxquery -user <user>

sudo mcxrefresh -u $(id -u)

Managed Preferences override user `defaults`

 if a `defaults write` isn't sticking, check

here first.
```

## † BUILDING A PROFILE

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>PayloadContent</key>
  <array>
    <dict>
      <key>PayloadType</key><string>com.apple.SoftwareUpdate</string>
      <key>PayloadIdentifier</key><string>org.example.su</string>
      <key>PayloadUUID</key><string>REPLACE-UUID-1</string>
      <key>PayloadVersion</key><integer>1</integer>
      <key>AutomaticCheckEnabled</key><true/>
      <key>ConfigDataInstall</key><true/>
      <key>CriticalUpdateInstall</key><true/>
    </dict>
  </array>
  <key>PayloadDisplayName</key><string>Software Update Baseline</string>
  <key>PayloadIdentifier</key><string>org.example.baseline</string>
  <key>PayloadType</key><string>Configuration</string>
  <key>PayloadUUID</key><string>REPLACE-UUID-0</string>
  <key>PayloadVersion</key><integer>1</integer>
  <key>PayloadScope</key><string>System</string>
</dict>
</plist>
```

```bash
uuidgen                                  # for the UUID fields

plutil -lint profile.mobileconfig

# security cms -S -N "<signing-identity>" -i in.mobileconfig -o signed.mobileconfig
```

Signed profiles show as "Verified" in System Settings; unsigned show as "Unverified" useful trust signal for users.

## † TOOLING

- **iMazing Profile Editor** free, best GUI for authoring; has the full payload schema - **mSCP** (`github.com/usnistgov/macos_security`) generates compliance profiles - **ProfileCreator** open source alternative

BOOK IX

# Network Configuration (networksetup, scutil, Wi-Fi, VPN)

*canticle of the 16th sheet*

✦ ⁂ ✦

## † networksetup

BASH

```
networksetup -listallnetworkservices

networksetup -listallhardwareports

networksetup -listnetworkserviceorder

networksetup -getinfo "Wi-Fi"

networksetup -getmacaddress en0

networksetup -getcomputername

# networksetup -setcomputername <name>
```

## ▸ IP configuration

BASH

```
# networksetup -setdhcp "Wi-Fi"

# networksetup -setmanual "Wi-Fi" 192.0.2.10 255.255.255.0 192.0.2.1

# networksetup -setbootp "Wi-Fi"

networksetup -getinfo "Wi-Fi"

# networksetup -setmanualwithdhcprouter "Wi-Fi" 192.0.2.10

ipconfig getifaddr en0

ipconfig getpacket en0

ipconfig getsummary en0

# ipconfig set en0 DHCP
```

## DNS

BASH

```
networksetup -getdnsservers "Wi-Fi"

# networksetup -setdnsservers "Wi-Fi" 1.1.1.1 9.9.9.9

# networksetup -setdnsservers "Wi-Fi" Empty      # revert to DHCP

networksetup -getsearchdomains "Wi-Fi"

# networksetup -setsearchdomains "Wi-Fi" example.com

scutil --dns

# dscacheutil -flushcache && sudo killall -HUP mDNSResponder
```

## PROXIES

BASH

```
networksetup -getwebproxy "Wi-Fi"

networksetup -getsecurewebproxy "Wi-Fi"

networksetup -getsocksfirewallproxy "Wi-Fi"

networksetup -getautoproxyurl "Wi-Fi"

networksetup -getproxybypassdomains "Wi-Fi"

# networksetup -setwebproxy "Wi-Fi" 127.0.0.1 8080

# networksetup -setwebproxystate "Wi-Fi" off

# networksetup -setsecurewebproxystate "Wi-Fi" off

# networksetup -setautoproxystate "Wi-Fi" off

# networksetup -setsocksfirewallproxystate "Wi-Fi" off

scutil --proxy

Audit all proxy settings across all services in one pass:
```

```bash
networksetup -listallnetworkservices | tail -n +2 | while read -r s; do
    for t in webproxy securewebproxy socksfirewallproxy autoproxyurl; do
      r=$(networksetup -get$t "$s" 2>/dev/null | tr '\n' ' ')
      echo "$r" | grep -qi 'Enabled: Yes' && echo "!! $s / $t: $r"
    done
  done
```

An unexpected PAC URL (`autoproxyurl`) is a quiet and very effective traffic redirect.

## ▸ INTERFACE POWER / ORDER

```bash
# networksetup -setnetworkserviceenabled "Wi-Fi" off
# networksetup -ordernetworkservices "Ethernet" "Wi-Fi"
# networksetup -setv6off "Wi-Fi"
# networksetup -setv6automatic "Wi-Fi"
networksetup -getv6 "Wi-Fi"
# networksetup -setMTU en0 1400
networksetup -getMTU en0
```

## † Wi-Fi

BASH

```bash
networksetup -getairportpower en0

# networksetup -setairportpower en0 off

networksetup -getairportnetwork en0

# networksetup -setairportnetwork en0 "<SSID>" "<password>"

networksetup -listpreferredwirelessnetworks en0

# networksetup -removepreferredwirelessnetwork en0 "<SSID>"

# networksetup -removeallpreferredwirelessnetworks en0

# networksetup -addpreferredwirelessnetworkatindex en0 "<SSID>" 0 WPA2 "<pw>"

sudo wdutil info

system_profiler SPAirPortDataType

ifconfig en0 | grep -E 'ether|status'

`wdutil info` (macOS 12+) is the replacement for the removed `airport` binary. It dumps

BSSID, RSSI, channel, security, and the current SSID in one go.
```

BASH

```bash
sudo wdutil info | grep -E 'SSID|BSSID|Channel|Security|RSSI|Tx Rate'
```

### ▸ MAC RANDOMIZATION

BASH

```bash
ifconfig en0 | grep ether

networksetup -getmacaddress en0

# ifconfig en0 ether aa:bb:cc:dd:ee:ff     # temporary, resets on reboot/reconnect

defaults read /Library/Preferences/com.apple.wifi.known-networks.plist 2>/dev/null |
grep -i private

macOS uses per-SSID private MAC addresses by default since Sonoma. Manual `ifconfig
ether`

changes are transient and often overwritten by the Wi-Fi stack.
```

## † SCUTIL

BASH

```
scutil --dns

scutil --proxy

scutil --nwi                            # network information / reachability

scutil --nc list                        # VPN configs

scutil --nc status "<service>"

# scutil --nc start "<service>"

# scutil --nc stop "<service>"

scutil --get ComputerName

scutil --get LocalHostName

scutil --get HostName

# scutil --set HostName <name>

# scutil --set LocalHostName <name>

# scutil --set ComputerName <name>

scutil -r example.com                   # reachability

scutil -r 192.0.2.1
```

## DYNAMIC STORE BROWSING

BASH

```
scutil <<< 'list'

scutil <<< 'show State:/Network/Global/IPv4'

scutil <<< 'show State:/Network/Global/DNS'

scutil <<< 'show State:/Network/Interface/en0/AirPort'

scutil <<< 'show Setup:/Network/Service/<uuid>/Proxies'

scutil <<< 'list State:/Network/Service/.*'

The dynamic store is the live view of network state; `Setup:` is the persisted config,

`State:` is current. Useful for spotting a config that's applied but not visible in the
UI.
```

## † ROUTING

BASH

```
netstat -nr

netstat -nr -f inet

netstat -nr -f inet6

route -n get default

route -n get 192.0.2.10

# route add -net 192.0.2.0/24 192.168.1.1

# route delete -net 192.0.2.0/24

# route flush                              #

arp -an

# arp -d <ip>

# arp -a -d                                #

 flush arp cache

ndp -an                                  # IPv6

A route added for a specific network pointing at an unexpected gateway is a redirect

mechanism

 check `netstat -nr` against what DHCP handed out.
```

## † FIREWALL

Covered in Sheet 05. Quick recap:

```bash
FW=/usr/libexec/ApplicationFirewall/socketfilterfw

# $FW --getglobalstate

# $FW --setglobalstate on

# $FW --setstealthmode on

# $FW --listapps

# pfctl -s rules

# pfctl -s states
```

## † VPN

```bash
scutil --nc list

scutil --nc status "<name>"

scutil --nc show "<name>"

# scutil --nc start "<name>" --user <u> --password <p>

# scutil --nc stop "<name>"

ifconfig | grep -E '^(utun|ipsec|ppp)'

netstat -nr | grep -E 'utun|ipsec'

systemextensionsctl list | grep -i network

ls /Library/Preferences/com.apple.networkextension*.plist

plutil -p /Library/Preferences/com.apple.networkextension.plist 2>/dev/null

log show --last 1d --predicate 'process == "nesessionmanager"' --info
```

### ▸ NETWORKEXTENSION CONTENT FILTERS (SEE ALL TRAFFIC)

BASH

```
systemextensionsctl list

systemextensionsctl list | grep -i 'filter\|proxy\|dns'

log show --last 7d --predicate 'subsystem == "com.apple.networkextension"' --info | head
-50

A content filter system extension has visibility into every flow on the box. Legitimate
for

EDR/DNS filtering products; a finding if you didn't install it.
```

## † SHARING SERVICES

BASH

```
sudo systemsetup -getremotelogin

# systemsetup -setremotelogin on|off

sudo systemsetup -getremoteappleevents

# systemsetup -setremoteappleevents off

# launchctl print-disabled system | grep -iE 'smbd|afp|screensharing|ftp|tftp|nfs|bootps'

sudo lsof -i -n -P | grep LISTEN

smbutil statshares -a

sharing -l 2>/dev/null                  # SMB/AFP share points

# sharing -a /path -S ShareName            # add share

# sharing -r ShareName                     # remove

nfsd status 2>/dev/null

cat /etc/exports 2>/dev/null
```

## † INTERNET SHARING (ROGUE AP SURFACE)

```bash
defaults read /Library/Preferences/SystemConfiguration/com.apple.nat 2>/dev/null

sudo launchctl list | grep -i InternetSharing

sysctl net.inet.ip.forwarding

# sysctl -w net.inet.ip.forwarding=0

IP forwarding enabled on a workstation is unusual and worth explaining.
```

## † TIME SYNC

```bash
sudo systemsetup -getnetworktimeserver

sudo systemsetup -getusingnetworktime

# systemsetup -setnetworktimeserver time.apple.com

# systemsetup -setusingnetworktime on

sntp -sS time.apple.com

date -u

systemsetup -gettimezone

Accurate time matters for log correlation

 verify it before you build a timeline.
```

## † DIAGNOSTICS

BASH

```
networkQuality                          # macOS 12+, responsiveness test

networkQuality -v

ping -c 4 1.1.1.1

traceroute -n 1.1.1.1

dig +trace example.com

curl -w '@-' -o /dev/null -s https://example.com <<'EOF'

namelookup: %{time_namelookup}\nconnect: %{time_connect}\nssl: %{time_appconnect}\ntotal:
%{time_total}\n

nc -zv example.com 443

sudo tcpdump -i any -nn -c 20

/System/Library/PrivateFrameworks/Apple80211.framework/Versions/A/Resources/airport -I
2>/dev/null
```

## † FULL NETWORK POSTURE DUMP

BASH

```
#!/bin/zsh

print "═══ Interfaces";        ifconfig -a

print "\n═══ Routes";          netstat -nr

print "\n═══ DNS";             scutil --dns | head -30

print "\n═══ Proxy";           scutil --proxy

print "\n═══ Services";        networksetup -listnetworkserviceorder

print "\n═══ Listening";       sudo lsof -i -n -P | grep LISTEN

print "\n═══ Established";     sudo lsof -i -n -P | grep ESTABLISHED

print "\n═══ ARP";             arp -an

print "\n═══ VPN";             scutil --nc list

print "\n═══ SysExtensions";   systemextensionsctl list

print "\n═══ Firewall";        sudo /usr/libexec/ApplicationFirewall/socketfilterfw --
getglobalstate

print "\n═══ PF";              sudo pfctl -s info 2>/dev/null | head -10

print "\n═══ IP forwarding";   sysctl net.inet.ip.forwarding

print "\n═══ Wi-Fi";           sudo wdutil info 2>/dev/null | head -30
```

BOOK X

# iOS / Mobile-Adjacent from a Mac

*canticle of the 24th sheet*

✦ ⁂ ✦

What you can do to iPhones/iPads from a connected Mac, mostly for triage and IR. Deep iOS forensics needs dedicated tooling (Cellebrite, GrayKey, MVT); this covers the accessible surface.

## † Device Connection & Identity

BASH

```bash
# Native
system_profiler SPUSBDataType | grep -A10 -i iphone
system_profiler SPUSBDataType | grep -A10 -i ipad
# libimobiledevice (the open-source toolkit)
brew install libimobiledevice ideviceinstaller
idevice_id -l                          # list connected UDIDs
ideviceinfo                            # full device info
ideviceinfo -k ProductVersion          # iOS version
ideviceinfo -k DeviceName
ideviceinfo -k SerialNumber
ideviceinfo -k UniqueDeviceID          # UDID
ideviceinfo -k ActivationState
idevicename
idevicedate                            # device clock
```

## † PAIRING & TRUST

```bash
idevicepair validate

idevicepair pair                          # requires unlock + "Trust" tap

idevicepair unpair

idevicepair list

ls /var/db/lockdown/                      # pairing records (root)

Pairing records in `/var/db/lockdown/` are what let a Mac talk to a device without

re-tapping Trust

 relevant both for your own workflow and as a thing an attacker with Mac

access could abuse against a plugged-in phone.
```

## † DEVICE LOGS (LIVE)

```bash
idevicesyslog                             # live syslog stream

idevicesyslog | grep -i <process>

idevicecrashreport -e /tmp/crashes        # pull crash reports

ls /tmp/crashes
```

## † BACKUPS (THE MAIN FORENSIC ARTIFACT)

```bash
idevicebackup2 backup /path/to/backupdir

idevicebackup2 backup --full /path/to/backupdir

idevicebackup2 encryption on <password>  # enable backup encryption

idevicebackup2 info /path/to/backupdir

idevicebackup2 list /path/to/backupdir

idevicebackup2 restore /path/to/backupdir   #

# Finder/iTunes backups already on the Mac

ls ~/Library/Application\ Support/MobileSync/Backup/

ls ~/Library/Application\ Support/MobileSync/Backup/<UDID>/

An **encrypted** backup actually contains *more* data than an unencrypted one (keychain,

Health, call history), so for authorized IR you generally want encryption on with a known

password.
```

### ▸ PARSING A BACKUP

```bash
# The Manifest.db maps hashed filenames to real paths

sqlite3 ~/Library/.../Backup/<UDID>/Manifest.db '.tables'

sqlite3 ~/Library/.../Backup/<UDID>/Manifest.db \

  "SELECT fileID, domain, relativePath FROM Files WHERE relativePath LIKE '%sms.db%';"

# iOS SMS, call history, etc. live at known domains

#   HomeDomain/Library/SMS/sms.db

#   HomeDomain/Library/CallHistoryDB/CallHistory.storedata

#   WirelessDomain, CameraRollDomain, AppDomain-<bundleid>, ...
```

## † MVT

Mobile Verification Toolkit (spyware triage)

```bash
pipx install mvt

mvt-ios check-backup /path/to/backupdir --output /tmp/mvt-out

mvt-ios check-backup --iocs /path/to/pegasus.stix2 /path/to/backup

mvt-ios decrypt-backup -p <password> -d /tmp/decrypted /path/to/backup

mvt-ios check-fs /tmp/decrypted                    # against a full filesystem dump

# Download Amnesty's public IOC set

mvt-ios download-iocs

MVT is the free, credible tool for checking iOS backups against known mercenary-spyware

IOCs (Pegasus, Predator, etc.). It's what Amnesty and Citizen Lab ship. For a "is my
phone

compromised" question from a high-risk user, this is the starting point.
```

## † SYSDIAGNOSE (IOS)

```bash
# Trigger on-device: hold both volume buttons + side button briefly (varies by model)

# Then retrieve via:

ls ~/Library/Logs/CrashReporter/MobileDevice/<DeviceName>/

# Or Settings > Privacy & Security > Analytics & Improvements > Analytics Data
```

## † App Inspection

BASH

```
ideviceinstaller -l                     # list installed apps
ideviceinstaller -l -o list_all
ideviceinstaller -i app.ipa             # install (needs signing)
ideviceinstaller -U <bundle-id>         # uninstall
# IPA is just a zip
unzip -l app.ipa
unzip app.ipa -d /tmp/app
plutil -p /tmp/app/Payload/*.app/Info.plist
codesign -dv --verbose=4 /tmp/app/Payload/*.app
codesign -d --entitlements - /tmp/app/Payload/*.app | plutil -p -
otool -L /tmp/app/Payload/*.app/<binary>
strings /tmp/app/Payload/*.app/<binary> | grep -iE 'http|api|key'
```

## † Profiles / MDM on iOS (from the Mac side)

BASH

```
# iOS config profiles are the same .mobileconfig format as macOS (Sheet 10)
plutil -p profile.mobileconfig
plutil -p profile.mobileconfig | grep -E 'PayloadType|PayloadIdentifier'
# On device: Settings > General > VPN & Device Management shows installed profiles
# Apple Configurator 2 (free, App Store) manages/inspects these from the Mac
A malicious iOS config profile is a common intermediate-sophistication attack (fake "VPN"
or "certificate" profile that installs a root CA + proxy). Check
Settings > General > VPN & Device Management on any suspect device.
```

## † Screen Mirroring for Evidence

BASH

```
# QuickTime Player: File > New Movie Recording > select the connected iOS device as source

# Records the device screen to the Mac

 good for documenting on-device findings
```

## † Apple Configurator (Bulk / Supervised)

BASH

```
brew install --cask apple-configurator

# GUI, but has a CLI helper:

/Applications/Apple\ Configurator.app/Contents/MacOS/cfgutil list

cfgutil get deviceName UDID ECID

cfgutil syslog

cfgutil get-app-icon <bundle-id>
```

## † Network Analysis of Device Traffic

BASH

```
# rvictl creates a virtual interface mirroring device traffic (needs UDID)

rvictl -s <UDID>                          # start remote virtual interface

ifconfig | grep rvi                       # rvi0 appears

sudo tcpdump -i rvi0 -nn -w /tmp/ios.pcap

rvictl -x <UDID>                          # stop

# Then analyze as normal (Sheet 05)

tshark -r /tmp/ios.pcap -Y 'tls.handshake.type==1' -T fields -e tls.handshake.extensions_server_name | sort -u

`rvictl` + `tcpdump -i rvi0` is the built-in, no-jailbreak way to capture an iOS device's

traffic from a Mac. Extremely useful for "what is this app phoning home to."
```

## † What you can't easily do

- Read the live filesystem without a jailbreak or a full forensic extraction tool - Decrypt an encrypted backup without the password - Access the keychain contents beyond what an encrypted backup exposes - Bypass Activation Lock / passcode (that's Cellebrite/GrayKey territory) For a genuine iOS compromise investigation on a high-value target, the realistic free path is: encrypted backup MVT IOC match. Beyond that you're into commercial extraction.

## † Quick device triage checklist

BASH

```
idevice_id -l                                    # confirm connection
ideviceinfo -k ProductVersion -k DeviceName      # what am I looking at
idevicebackup2 encryption on <pw>                # ensure rich backup
idevicebackup2 backup --full /tmp/iosbackup
mvt-ios download-iocs
mvt-ios check-backup /tmp/iosbackup --output /tmp/mvt
open /tmp/mvt                                     # review flagged items
# Also on-device: check VPN & Device Management for unexpected profiles
```

BOOK XI

# SCRIPTING TOOLKIT FOR SECURITY WORK

*canticle of the 23rd sheet*

✦ ⁂ ✦

3 sh/bash/jq/python patterns that come up constantly in the sheets above. macOS ships bash 3.2 (ancient, GPLv2) and zsh 5.9 as the default login shell write for zsh, keep bash scripts 3.2-compatible or `#!/usr/bin/env bash` against a brew bash.

## † SHELL PORTABILITY NOTES

BASH

```
echo $SHELL                              # login shell

/bin/bash --version                      # 3.2.57

 no assoc arrays, no ${x^^}

brew install bash && which -a bash       # 5.x under brew prefix

echo $ZSH_VERSION

macOS BSD userland differences from GNU (bite you constantly):

| GNU (Linux) | BSD (macOS) |

|---|---|

| `sed -i` | `sed -i ''` (needs empty arg) |

| `date -d @1234` | `date -r 1234` |

| `date +%s -d '...'` | `date -j -f '%Y-%m-%d' '...' +%s` |

| `stat -c %Y` | `stat -f %m` |

| `readlink -f` | `readlink` (no -f; use `realpath` from coreutils) |

| `grep -P` | not supported; use `-E` or `rg` |

| `xxd` | present, but `-r -p` differs subtly |

| `base64 -w0` | `base64` (no wrap flag; `| tr -d '\n'`) |

| `find -printf` | not supported; use `-exec stat` |

| `timeout` | `gtimeout` (coreutils) |
```

# Get GNU tools without shadowing system ones

BASH

```
brew install coreutils findutils gnu-sed grep gawk

# They install with 'g' prefix: gsed, ggrep, gdate, gstat, gtimeout, grealpath

alias sed=gsed grep=ggrep date=gdate      # optional, in a security-shell profile
```

## † SAFE SCRIPTING HEADER

```bash
#!/usr/bin/env bash

set -euo pipefail

IFS=$'\n\t'

trap 'echo "error on line $LINENO" >&2' ERR
```

#!/bin/zsh set -e setopt pipefail err_return no_unset

## † ITERATING FILES SAFELY (SPACES, NEWLINES)

```bash
# Null-delimited

 the only robust way

find /path -type f -print0 | while IFS= read -r -d '' f; do

    process "$f"

  done

# zsh globbing handles this natively

for f in /path/**/*.plist(N); do process "$f"; done   # (N) = null_glob
```

## † PLIST MANIPULATION

BASH

```
# Read

plutil -p file.plist

/usr/libexec/PlistBuddy -c 'Print :Key:SubKey' file.plist

defaults read domain key

# Extract a single value scriptably

plutil -extract Key raw -o - file.plist

/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' file.plist 2>/dev/null

# Convert

plutil -convert json -o - file.plist | jq .

plutil -convert xml1 -o - file.plist

plutil -convert binary1 file.plist

# Loop over an array in a plist

n=$(/usr/libexec/PlistBuddy -c 'Print :Items' f.plist 2>/dev/null | grep -c 'Dict')

for i in $(seq 0 $((n-1))); do

    /usr/libexec/PlistBuddy -c "Print :Items:$i:Name" f.plist 2>/dev/null

  done

The `plutil -convert json -o - | jq` bridge is the cleanest way to script over plists

converts binary/xml to JSON and lets jq do the work.
```

## † JQ PATTERNS

BASH

```
# Field extraction

jq -r '.key'

jq -r '.array[] | .field'

jq -r '.array[] | [.a, .b, .c] | @tsv'

jq -r '.array[] | select(.status=="active") | .name'

jq -r 'to_entries[] | "\(.key)=\(.value)"'

# Filtering with regex

jq -r 'select(.path | test("Launch"; "i"))'

jq -r '.[] | select(.name | startswith("com.apple") | not)'

# Aggregation

jq -s 'group_by(.host) | map({host: .[0].host, count: length})'

jq -r '[.[].bytes] | add'

jq 'sort_by(.timestamp) | reverse | .[0:10]'

# Building CSV/TSV

jq -r '(.[0] | keys_unsorted) as $k | $k, (.[] | [.[$k[]]]) | @csv'

# ndjson stream processing (one object per line)

log show --style ndjson ... | jq -r 'select(...) | .field'

eslogger exec | jq -c 'select(.process.ppid==1)'

# Merge streaming events into counts

... | jq -r '.process.executable.path' | sort | uniq -c | sort -rn
```

## † TIMESTAMP CONVERSION (THE MACOS PAIN POINT)

BASH

```
# Unix epoch

 human

date -r 1721800000

date -u -r 1721800000 +%FT%TZ

# Human

 epoch (BSD date)

date -j -f '%Y-%m-%d %H:%M:%S' '2026-07-24 10:00:00' +%s

# Cocoa/Core Data epoch (seconds since 2001-01-01)

 add 978307200

echo $((<cocoa_ts> + 978307200)) | xargs -I{} date -r {}

sqlite3 db "SELECT datetime(<col>+978307200,'unixepoch')"

# APFS/HFS timestamps in stat

stat -f '%Sm' -t '%FT%TZ' file

stat -f 'birth=%SB mod=%Sm' -t '%FT%T' file

# Mach absolute time is different again (needs mach_timebase)

 rarely needed manually
```

## † SQLITE (SAFARI HISTORY, TCC, QUARANTINE, KNOWLEDGEC)

BASH

```
# Always copy the DB first

 WAL mode + live app locks

cp ~/Library/.../db.sqlite /tmp/work.db

sqlite3 /tmp/work.db '.tables'

sqlite3 /tmp/work.db '.schema <table>'

sqlite3 -header -column /tmp/work.db 'SELECT ...'

sqlite3 -json /tmp/work.db 'SELECT ...' | jq .

sqlite3 -csv -header /tmp/work.db 'SELECT ...' > out.csv

# Handle WAL files

cp db.sqlite db.sqlite-wal db.sqlite-shm /tmp/ 2>/dev/null

sqlite3 /tmp/db.sqlite 'PRAGMA wal_checkpoint;'

# Common Cocoa-epoch pattern

sqlite3 /tmp/work.db \

  "SELECT datetime(col+978307200,'unixepoch','localtime'), other FROM t ORDER BY col
DESC;"
```

## † CODESIGN IN SCRIPTS

BASH

```bash
# Boolean: is it Apple-signed?
is_apple() { codesign -dv "$1" 2>&1 | grep -q 'Authority=Apple' ; }
# Get team id
teamid() { codesign -dv "$1" 2>&1 | awk -F= '/TeamIdentifier/{print $2}' ; }
# Get signing status as one word
sigstatus() {
  local o; o=$(codesign -dv "$1" 2>&1)
  if grep -q 'not signed' <<<"$o"; then echo UNSIGNED
  elif grep -q 'adhoc' <<<"$o"; then echo ADHOC
  elif grep -q 'Authority=Apple' <<<"$o"; then echo APPLE
  else echo THIRDPARTY; fi
```

## † PARALLEL EXECUTION

BASH

```bash
# xargs parallel
find /Applications -name '*.app' -print0 | xargs -0 -P 8 -I{} codesign --verify {}
# GNU parallel (brew install parallel)
parallel -j8 shasum -a 256 {} ::: /path/*
# zsh: background jobs with a limit
for f in *.bin; do
    (( $(jobs | wc -l) >= 8 )) && wait
    analyze "$f" &
  done; wait
```

## † NOTIFICATIONS (ALERTING FROM A MONITORING SCRIPT)

BASH

```
osascript -e 'display notification "text" with title "title" sound name "Ping"'

terminal-notifier -title "Alert" -message "text"   # brew install terminal-notifier

afplay /System/Library/Sounds/Sosumi.aiff           # audible alert

# say "persistence detected"                            # TTS, if you must
```

# † PYTHON FOR THE HEAVIER LIFTING

PYTHON

```python
#!/usr/bin/env python3

"""Common macOS security helpers."""

import subprocess, plistlib, sqlite3, hashlib, json, os, datetime

def read_plist(path):

    with open(path, 'rb') as f:

        return plistlib.load(f)

def sha256(path):

    h = hashlib.sha256()

    with open(path, 'rb') as f:

        for chunk in iter(lambda: f.read(1 << 20), b''):

            h.update(chunk)

    return h.hexdigest()

def codesign_info(path):

    r = subprocess.run(['codesign', '-dv', '--verbose=4', path],

                       capture_output=True, text=True)

    out = r.stderr  # codesign writes to stderr

    info = {'signed': 'not signed' not in out, 'adhoc': 'adhoc' in out}

    for line in out.splitlines():

        if line.startswith('TeamIdentifier='):

            info['team'] = line.split('=', 1)[1]

        if line.startswith('Authority='):

            info.setdefault('authorities', []).append(line.split('=', 1)[1])

    return info

def cocoa_to_dt(ts):

    return datetime.datetime(2001, 1, 1) + datetime.timedelta(seconds=ts)

def query_db(path, sql):

    # copy first in real use
```

```
    con = sqlite3.connect(f'file:{path}?mode=ro', uri=True)
    try:
        return con.execute(sql).fetchall()
    finally:
        con.close()
def log_show(predicate, last='1h', style='ndjson'):
    r = subprocess.run(['log', 'show', '--last', last, '--style', style,
                        '--predicate', predicate, '--info'],
                       capture_output=True, text=True)
    if style == 'ndjson':
        return [json.loads(l) for l in r.stdout.splitlines() if l.strip()]
    return r.stdout
```

# † MACOS SECURITY HELPER LIBRARY (ZSH FUNCTIONS)

ZSH

```zsh
# Source this from ~/.zshrc for interactive hunting

# ~/.security-functions.zsh

# persistence: dump all launch items with signature status

persist() {

  for p in /Library/Launch{Agents,Daemons}/*.plist(N) ~/Library/LaunchAgents/*.plist(N);
do

    local b=$(/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null \

              || /usr/libexec/PlistBuddy -c 'Print :Program' "$p" 2>/dev/null)

    printf '%-55s %-10s %s\n' "${p:t}" "$(sigstatus ${b:-/nonexistent} 2>/dev/null)" "$b"

  done

# whofrom: download provenance of a file

whofrom() {

  xattr -p com.apple.metadata:kMDItemWhereFroms "$1" 2>/dev/null | xxd -r -p | plutil -p
- 2>/dev/null

# tccdump: current TCC grants

tccdump() {

  sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

    "SELECT service, client, auth_value FROM access WHERE auth_value=2;" 2>/dev/null

# netprocs: established connections with owning process signature

netprocs() {

  sudo lsof -i -n -P | awk '/ESTABLISHED/{print $1, $2, $9}'

# listening: non-loopback listeners

listening() {

  sudo lsof -i -n -P | grep LISTEN | grep -v '127\.0\.0\.1\|\[::1\]'

# quar: quarantine events, most recent 20

quar() {

  sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 \
```

```
    "SELECT datetime(LSQuarantineTimeStamp+978307200,'unixepoch'),
LSQuarantineDataURLString FROM LSQuarantineEvent ORDER BY 1 DESC LIMIT 20;"

# recent: files modified in home in last N days

recent() { find ~ -type f -mtime -${1:-1} -not -path '*/Library/Caches/*' 2>/dev/null; }
```

## † WORDLIST / DATA MUNGING

BASH

```
sort -u file

sort file | uniq -c | sort -rn           # frequency

awk '!seen[$0]++'                        # dedupe preserving order

awk -F, '{print $2}'                     # CSV column

cut -d: -f1 /etc/passwd

tr ',' '\n' | tr '[:upper:]' '[:lower:]'

rg -oP '...' file 2>/dev/null || rg -o 'pattern' file   # ripgrep (has PCRE via -P if
built with it)

comm -23 <(sort a) <(sort b)             # in a but not b

diff <(sort a) <(sort b)

paste -d, col1 col2
```

## † Handy One-Liners

BASH

```bash
# Base64 encode a file (no line wrapping)
base64 < file | tr -d '\n'

# URL-encode
python3 -c 'import urllib.parse,sys; print(urllib.parse.quote(sys.argv[1]))' "string"

# JSON pretty-print from clipboard (macOS)
pbpaste | jq . | pbcopy

# Extract all IPs from a file
grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' file | sort -u

# Extract all domains
grep -oE '[a-zA-Z0-9.-]+\.[a-z]{2,}' file | sort -u

# Watch a file grow
tail -f file

log stream --predicate '...' | tee -a capture.log

# Time a command
/usr/bin/time -l command                # -l gives max RSS on macOS
```

BOOK XII

# MACOS HARDENING BASELINE

*canticle of the 9th sheet*

✦ ⁂ ✦

CIS-flavored, but pruned to things that actually matter and don't break workflows. Audit commands first, then the corresponding set.

# † Audit Script

BASH

```
#!/bin/zsh

# audit.zsh

 read-only posture check

chk() { printf '%-46s %s\n' "$1" "$2"; }

chk "SIP"                    "$(csrutil status | sed 's/.*: //')"

chk "Gatekeeper"             "$(spctl --status)"

chk "FileVault"              "$(fdesetup status | head -1)"

chk "Sealed system volume"   "$(csrutil authenticated-root status | sed 's/.*: //')"

chk "Firewall"               "$(sudo /usr/libexec/ApplicationFirewall/socketfilterfw --
getglobalstate)"

chk "Stealth mode"           "$(sudo /usr/libexec/ApplicationFirewall/socketfilterfw --
getstealthmode)"

chk "Remote login (SSH)"     "$(sudo systemsetup -getremotelogin 2>/dev/null)"

chk "Remote Apple Events"    "$(sudo systemsetup -getremoteappleevents 2>/dev/null)"

chk "Auto software update"   "$(defaults read /Library/Preferences/
com.apple.SoftwareUpdate AutomaticCheckEnabled 2>/dev/null)"

chk "Auto security response" "$(defaults read /Library/Preferences/
com.apple.SoftwareUpdate AutomaticallyInstallMacOSUpdates 2>/dev/null)"

chk "Guest account"          "$(sudo defaults read /Library/Preferences/
com.apple.loginwindow GuestEnabled 2>/dev/null)"

chk "Screensaver password"   "$(defaults read com.apple.screensaver askForPassword 2>/
dev/null)"

chk "Password delay"         "$(defaults read com.apple.screensaver askForPasswordDelay
2>/dev/null)"

chk "Show all extensions"    "$(defaults read NSGlobalDomain AppleShowAllExtensions 2>/
dev/null)"

chk "AirDrop"                "$(defaults read com.apple.NetworkBrowser DisableAirDrop 2>/
dev/null)"

chk "Bonjour advertising"    "$(defaults read /Library/Preferences/
com.apple.mDNSResponder.plist NoMulticastAdvertisements 2>/dev/null)"

chk "Secure keyboard entry"  "$(defaults read -app Terminal SecureKeyboardEntry 2>/dev/
null)"
```

```
chk "Siri"                    "$(defaults read com.apple.assistant.support 'Assistant
Enabled' 2>/dev/null)"

chk "Analytics sharing"       "$(defaults read /Library/Application\ Support/
CrashReporter/DiagnosticMessagesHistory.plist AutoSubmit 2>/dev/null)"

chk "Wake on network"         "$(pmset -g | awk '/womp/{print $2}')"

chk "Power nap"               "$(pmset -g | awk '/powernap/{print $2}')"
```

## † SYSTEM INTEGRITY PROTECTION

BASH

```
csrutil status

csrutil authenticated-root status

# Enable/disable requires Recovery mode:

#   Apple silicon: hold power at boot

 Options

 Utilities

 Terminal

#   Intel: Cmd-R at boot

# csrutil enable

# csrutil disable                         #

 never on a production/daily box

# csrutil clear

Keep SIP on. If a tool requires disabling it, that tool doesn't belong on your daily
driver

 run it in a VM.
```

## † SECURE BOOT (APPLE SILICON)

BASH

```
# bputil -d                              # display boot policy

# bputil -f                              # full security (default)

system_profiler SPiBridgeDataType       # T2 / secure boot info (Intel T2)

nvram -p | grep -i boot

Security levels: **Full** (default, only signed current OS), **Reduced** (allows kext,

older OS), **Permissive** (needed for third-party kernels). Anything below Full on a

managed machine is a finding.
```

## † FILEVAULT

BASH

```
fdesetup status

# fdesetup enable

# fdesetup enable -user <user>

# fdesetup list

# fdesetup haspersonalrecoverykey

# fdesetup hasinstitutionalrecoverykey

# fdesetup changerecovery -personal

# fdesetup validaterecovery

# fdesetup disable                       #

diskutil apfs listUsers /

sysadminctl -secureTokenStatus <user>
```

## † FIREWALL

BASH

```
FW=/usr/libexec/ApplicationFirewall/socketfilterfw

# $FW --setglobalstate on

# $FW --setstealthmode on

# $FW --setallowsigned off

# $FW --setallowsignedapp off

# $FW --setloggingmode on

# defaults write /Library/Preferences/com.apple.alf globalstate -int 1

For egress control add PF rules (Sheet 05) or a NetworkExtension tool. ALF alone is

inbound-only.
```

## † SOFTWARE UPDATES

BASH

```
softwareupdate --list

# softwareupdate -ia                    # install all

# softwareupdate --install-rosetta --agree-to-license

defaults read /Library/Preferences/com.apple.SoftwareUpdate

# Enable everything

# defaults write /Library/Preferences/com.apple.SoftwareUpdate AutomaticCheckEnabled -
bool true

# defaults write /Library/Preferences/com.apple.SoftwareUpdate AutomaticDownload -bool
true

# defaults write /Library/Preferences/com.apple.SoftwareUpdate
AutomaticallyInstallMacOSUpdates -bool true

# defaults write /Library/Preferences/com.apple.SoftwareUpdate ConfigDataInstall -bool
true   # XProtect/MRT

# defaults write /Library/Preferences/com.apple.SoftwareUpdate CriticalUpdateInstall -
bool true

# defaults write /Library/Preferences/com.apple.commerce AutoUpdate -bool true

`ConfigDataInstall` and `CriticalUpdateInstall` control XProtect/Gatekeeper data updates

the highest-value ones. Never disable these.
```

## † SCREEN LOCK

BASH

```
# defaults write com.apple.screensaver askForPassword -int 1

# defaults write com.apple.screensaver askForPasswordDelay -int 0

# defaults -currentHost write com.apple.screensaver idleTime -int 300

# defaults write com.apple.loginwindow LoginwindowText "Property of <org>

 contact <email>"

# defaults write /Library/Preferences/com.apple.loginwindow SHOWFULLNAME -bool true

# defaults write /Library/Preferences/com.apple.loginwindow DisableConsoleAccess -bool
true
```

## † ACCOUNTS

```bash
# Disable guest

# defaults write /Library/Preferences/com.apple.loginwindow GuestEnabled -bool false

# sysadminctl -guestAccount off

# defaults write /Library/Preferences/com.apple.AppleFileServer guestAccess -bool false

# defaults write /Library/Preferences/SystemConfiguration/com.apple.smb.server
AllowGuestAccess -bool false

# Disable automatic login

# defaults delete /Library/Preferences/com.apple.loginwindow autoLoginUser 2>/dev/null

# rm /etc/kcpassword 2>/dev/null           #

 the obfuscated autologin password

# Password policy

# pwpolicy -getaccountpolicies

# pwpolicy -setaccountpolicies /path/policy.plist

pwpolicy -n /Local/Default -getglobalpolicy

`/etc/kcpassword` is XOR-obfuscated with a known key

 if it exists, the login password is

effectively plaintext. Always check for it.
```

```bash
ls -la /etc/kcpassword 2>/dev/null && echo "AUTOLOGIN ENABLED
```

password recoverable"

## † SHARING SERVICES — TURN OFF WHAT YOU DON'T USE

BASH

```
# systemsetup -setremotelogin off

# systemsetup -setremoteappleevents off

# launchctl disable system/com.apple.smbd

# launchctl disable system/com.apple.AppleFileServer

# launchctl disable system/com.apple.screensharing

# launchctl disable system/com.apple.RemoteDesktop.PrivilegeProxy

# launchctl disable system/com.apple.ftp-proxy

# launchctl disable system/com.apple.tftpd

# launchctl print-disabled system | grep -v 'false'
```

## † NETWORK PRIVACY

BASH

```
# Disable Bonjour advertising

# defaults write /Library/Preferences/com.apple.mDNSResponder.plist
NoMulticastAdvertisements -bool true

# AirDrop off

# defaults write com.apple.NetworkBrowser DisableAirDrop -bool true

# Wake on LAN off

# pmset -a womp 0

# pmset -a powernap 0

# pmset -a proximitywake 0

# Captive portal assistant (auto-connects, MITM surface)

# defaults write /Library/Preferences/SystemConfiguration/com.apple.captive.control
Active -bool false

# Wi-Fi: don't auto-join

networksetup -setairportpower en0 off

networksetup -removepreferredwirelessnetwork en0 "<SSID>"

networksetup -listpreferredwirelessnetworks en0
```

## † TERMINAL & SHELL

BASH

```
# defaults write -app Terminal SecureKeyboardEntry -bool true

# defaults write -app iTerm SecureKeyboardEntry -bool true

defaults write com.apple.Terminal "Secure Keyboard Entry" -bool true

# History hygiene

echo 'export HISTCONTROL=ignoreboth' >> ~/.zshrc

echo 'setopt HIST_IGNORE_SPACE' >> ~/.zshrc

Secure Keyboard Entry blocks other processes from reading keystrokes via the
Accessibility

API while the terminal is focused. Cheap and worth it.
```

## † FINDER / UX SECURITY

BASH

```
defaults write NSGlobalDomain AppleShowAllExtensions -bool true

defaults write com.apple.finder AppleShowAllFiles -bool true

defaults write com.apple.finder ShowPathbar -bool true

defaults write com.apple.LaunchServices LSQuarantine -bool true    # keep quarantine on

defaults write com.apple.desktopservices DSDontWriteNetworkStores -bool true

defaults write com.apple.desktopservices DSDontWriteUSBStores -bool true

killall Finder
```

## † TELEMETRY REDUCTION

BASH

```
# defaults write /Library/Application\ Support/CrashReporter/
DiagnosticMessagesHistory.plist AutoSubmit -bool false

# defaults write /Library/Application\ Support/CrashReporter/
DiagnosticMessagesHistory.plist ThirdPartyDataSubmit -bool false

defaults write com.apple.assistant.support 'Assistant Enabled' -bool false

defaults write com.apple.Siri StatusMenuVisible -bool false

defaults write com.apple.Spotlight -dict-add 'suggestionsEnabled' -bool false
```

## † SAFARI

BASH

```
defaults write com.apple.Safari AutoOpenSafeDownloads -bool false      # important

defaults write com.apple.Safari WarnAboutFraudulentWebsites -bool true

defaults write com.apple.Safari SendDoNotTrackHTTPHeader -bool true

defaults write com.apple.Safari WebKitJavaScriptCanOpenWindowsAutomatically -bool false

defaults write com.apple.Safari ShowFullURLInSmartSearchField -bool true

defaults write com.apple.Safari IncludeDevelopMenu -bool true

`AutoOpenSafeDownloads` is the one that matters

 it's how many drive-by chains start.
```

## † AUDIT / LOGGING RETENTION

BASH

```
# log config --status

# log config --mode "persist:info" --subsystem com.apple.securityd

cat /etc/security/audit_control 2>/dev/null

sudo du -sh /var/db/diagnostics/
```

## † VERIFICATION PASS

```bash
# Re-run the audit script and diff

./audit.zsh > /tmp/post.txt && diff /tmp/pre.txt /tmp/post.txt
```

## † REFERENCE BASELINES

- **CIS Apple macOS Benchmark** the formal control set, per OS version - **NIST macOS Security Compliance Project (mSCP)** `github.com/usnistgov/macos_security`, generates profiles/scripts from NIST 800-53 baselines. This is the one to use if you need auditable compliance artifacts for a client. - **Apple Platform Security Guide** authoritative on how the mechanisms actually work mSCP is genuinely good and generates ready-to-deploy configuration profiles plus an audit script per baseline (`cis_lvl1`, `800-53r5_moderate`, etc.).

BOOK XIII

# KEYCHAIN, SECURE TOKEN, SECRETS

*canticle of the 11th sheet*

## † KEYCHAIN FILES

| PATH | CONTENTS |
|---|---|
| `~/Library/Keychains/login.keychain-db` | User keychain |
| `~/Library/Keychains/<UUID>/keychain-2.db` | iCloud/data-protection keychain |
| `/Library/Keychains/System.keychain` | System-wide: Wi-Fi PSKs, certs, 802.1X |
| `/Library/Keychains/apsd.keychain` | APNs |
| `/System/Library/Keychains/SystemRootCertificates.keychain` | Root CAs |

BASH

```
security list-keychains

security default-keychain

security login-keychain

ls -la ~/Library/Keychains/

ls -la /Library/Keychains/
```

## † DUMPING METADATA (NO SECRETS)

BASH

```
security dump-keychain                          # metadata only, no prompt

security dump-keychain ~/Library/Keychains/login.keychain-db

security dump-keychain | grep -E '"svce"|"acct"|"srvr"' | head -50

security dump-keychain -d                       #
 decrypt
 prompts per item

`-d` triggers a GUI prompt for every single item unless the keychain is unlocked and the

caller is trusted. In IR this is loud and slow; usually you want metadata only.
```

## † QUERYING SPECIFIC ITEMS

BASH

```
security find-generic-password -s "<service>"

security find-generic-password -s "<service>" -w         # print password

security find-generic-password -a "<account>" -s "<service>" -w

security find-internet-password -s "example.com"

security find-internet-password -s "example.com" -w

security find-certificate -a -c "<name>"

security find-certificate -a -p                          # PEM export

security find-identity -v                                # signing identities

security find-identity -v -p codesigning
```

## † ADDING / REMOVING

BASH

```
security add-generic-password -a "<acct>" -s "<svc>" -w "<pass>"

security add-generic-password -a "<acct>" -s "<svc>" -w "<pass>" -T /usr/bin/curl

security add-internet-password -a "<acct>" -s "example.com" -w "<pass>"

security delete-generic-password -s "<svc>"

security add-certificates -k /Library/Keychains/System.keychain cert.pem

security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain
cert.pem   #

 MITM enabler

security remove-trusted-cert -d cert.pem

`add-trusted-cert -d -r trustRoot` is exactly what an interception setup does. Seeing it
in

shell history or a script is a strong signal.
```

## † LOCK / UNLOCK

BASH

```
security lock-keychain

security lock-keychain -a                       # all

security unlock-keychain ~/Library/Keychains/login.keychain-db

security unlock-keychain -p "<pass>" <keychain> #

 password in history

security set-keychain-settings -l -u -t 3600 ~/Library/Keychains/login.keychain-db

security show-keychain-info ~/Library/Keychains/login.keychain-db

`-l` lock on sleep, `-u` lock after timeout, `-t` timeout seconds. Setting these is a

reasonable hardening step:
```

BASH

```
security set-keychain-settings -l -u -t 900 ~/Library/Keychains/login.keychain-db
```

## † Wi-Fi Passwords

```bash
# security find-generic-password -ga "<SSID>" -w

# security find-generic-password -D "AirPort network password" -a "<SSID>" -w

security dump-keychain /Library/Keychains/System.keychain | grep -A5 AirPort

networksetup -listpreferredwirelessnetworks en0
```

## † Trust Settings Audit

```bash
security dump-trust-settings              # user domain

security dump-trust-settings -d           # admin domain

security dump-trust-settings -s           # system domain

security verify-cert -c cert.pem

security trust-settings-export -d /tmp/admin-trust.plist && plutil -p /tmp/admin-
trust.plist

Non-empty user/admin trust settings on a machine you didn't configure = investigate.
That's

where a MITM root CA lands.
```

## † SECURE TOKEN & VOLUME OWNERSHIP

BASH

```
sysadminctl -secureTokenStatus <user>

diskutil apfs listUsers /

diskutil apfs listCryptoUsers /

# sysadminctl -secureTokenOn <user> -password <pw> -adminUser <a> -adminPassword <ap>

# sysadminctl -secureTokenOff <user> -password <pw> -adminUser <a> -adminPassword <ap>

# fdesetup list

# fdesetup add -usertoadd <user>

Concepts:

- **Secure Token**

 wrapped key allowing a user to unlock FileVault

- **Volume Ownership** (Apple silicon)

 required for OS updates and some admin ops

- **Bootstrap Token**

 escrowed to MDM so it can grant tokens without a local admin
```

# profiles status -type bootstraptoken # diskutil apfs listUsers / | grep -A3 'Volume Owner'

## † FILEVAULT RECOVERY KEY

BASH

```
fdesetup haspersonalrecoverykey

fdesetup hasinstitutionalrecoverykey

# fdesetup changerecovery -personal         # generates new PRK, prints it once

# fdesetup validaterecovery

# fdesetup removerecovery -personal         #
```

## † KEYCHAIN FORENSIC ACQUISITION

BASH

```
cp ~/Library/Keychains/login.keychain-db /Volumes/Ext/evidence/

sudo cp /Library/Keychains/System.keychain /Volumes/Ext/evidence/

ditto --rsrc --extattr ~/Library/Keychains /Volumes/Ext/evidence/Keychains

Decryption requires the user's login password (for `login.keychain-db`) or the

FileVault/system key (for `System.keychain`). Tools: `chainbreaker` (open source, parses

keychain format), Elcomsoft/Passware commercially.
```

BASH

```
python3 chainbreaker.py --dump-all --password '<pw>' login.keychain-db
```

## † SECRETS OUTSIDE THE KEYCHAIN

BASH

```bash
# SSH

ls -la ~/.ssh/

cat ~/.ssh/config

cat ~/.ssh/known_hosts | wc -l

ssh-add -l                                # agent-loaded keys

ssh-add -L

for k in ~/.ssh/id_*; do [[ $k == *.pub ]] && continue; ssh-keygen -l -f "$k"; done

grep -l 'ENCRYPTED' ~/.ssh/id_* 2>/dev/null   # which keys are passphrase-protected

# Cloud / API credentials

ls -la ~/.aws/ ~/.config/gcloud/ ~/.azure/ ~/.kube/ 2>/dev/null

cat ~/.aws/credentials 2>/dev/null

cat ~/.netrc ~/.npmrc ~/.pypirc 2>/dev/null

cat ~/.docker/config.json 2>/dev/null

ls ~/.gnupg/

gpg --list-secret-keys

# Git

git config --global --list | grep -i -E 'token|password|url'

cat ~/.git-credentials 2>/dev/null

# Environment / shell

grep -riE 'api[_-]?key|secret|token|passwd|password' ~/.zshrc ~/.zshenv ~/.bash_profile
~/.profile 2>/dev/null

grep -riE 'export .*(KEY|TOKEN|SECRET|PASSWORD)=' ~/.zsh_history 2>/dev/null | head -20

# Browser saved passwords (encrypted with a keychain-stored key)

ls ~/Library/Application\ Support/Google/Chrome/Default/Login\ Data

ls ~/Library/Application\ Support/Firefox/Profiles/*/logins.json
```

## † SECRET SCANNING A FILESYSTEM

BASH

```
brew install gitleaks trufflehog

gitleaks detect --source ~/projects --no-git -v

trufflehog filesystem ~/projects --only-verified

rg -i --no-heading -e 'AKIA[0-9A-Z]{16}' -e 'sk-[A-Za-z0-9]{32,}' -e 'ghp_[A-Za-z0-9]{36}' ~/ 2>/dev/null

rg -i 'BEGIN (RSA|OPENSSH|EC|PGP) PRIVATE KEY' ~/ 2>/dev/null
```

## † Hunting for credential theft

```bash
# Processes reading keychain files

# fs_usage -w -f filesys 2>/dev/null | grep -i keychain

# Keychain access log

log show --last 7d --predicate 'subsystem == "com.apple.securityd"' --info | grep -iE
'keychain|SecItem' | head -50

log show --last 7d --predicate 'eventMessage CONTAINS "keychain"' --info | head -50

# Apps holding Full Disk Access (can read keychain files directly)

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

  "SELECT client FROM access WHERE service='kTCCServiceSystemPolicyAllFiles' AND
auth_value=2;"

# security binary invoked from a script

log show --last 7d --predicate 'process == "security"' --info | head -30

grep -r 'security find-generic-password\|security dump-keychain' ~/ 2>/dev/null | head

A non-Apple process calling `SecItemCopyMatching` in bulk, or shelling out to

`security dump-keychain -d`, is the classic macOS infostealer pattern (Atomic Stealer,

MetaStealer, etc. all do exactly this

 usually via `osascript` prompting the user for

their password rather than touching the keychain API at all).
```

# The osascript password-prompt pattern

```bash
log show --last 7d --predicate 'process == "osascript"' --info | grep -iE 'password|
dialog' | head

grep -r 'display dialog.*password' ~/ 2>/dev/null | head

# macOS Security & Sysadmin Cheatsheets
```

Defensive-weighted reference set for macOS (Ventura/Sonoma/Sequoia era). Commands assume zsh, Apple silicon or Intel, and admin rights where noted. `sudo` is marked explicitly.

| # | SHEET | COVERS |
|---|---|---|
| 01 | [macos-triage.md](01-macos-triage.md) | Live host triage, quick IR sweep |
| 02 | [persistence-hunting.md](02-persistence-hunting.md) | Every persistence surface on macOS |
| 03 | [unified-log.md](03-unified-log.md) | `log` predicates, Endpoint Security events |
| 04 | [processes-memory.md](04-processes-memory.md) | Process, dylib, memory inspection |
| 05 | [network-forensics.md](05-network-forensics.md) | Sockets, DNS, pcap, egress checks |
| 06 | [codesign-gatekeeper.md](06-codesign-gatekeeper.md) | Signing, notarization, quarantine, XProtect |
| 07 | [tcc-privacy.md](07-tcc-privacy.md) | TCC database, entitlements, privacy grants |
| 08 | [filesystem-forensics.md](08-filesystem-forensics.md) | APFS, snapshots, xattrs, timestamps, FSEvents |
| 09 | [hardening-baseline.md](09-hardening-baseline.md) | CIS-style hardening via `defaults` / `systemsetup` |
| 10 | [mdm-config-profiles.md](10-mdm-config-profiles.md) | Profiles, MDM state, supervision |
| 11 | [keychain-secrets.md](11-keychain-secrets.md) | Keychain, secure token, FileVault, bootstrap token |
| 12 | [users-auth.md](12-users-auth.md) | Directory Services, sudoers, auth logs |
| 13 | [launchd.md](13-launchd.md) | `launchctl` modern syntax, domains, service mgmt |
| 14 | [packages-software.md](14-packages-software.md) | pkgutil, Homebrew audit, app inventory |
| 15 | [disk-encryption.md](15-disk-encryption.md) | FileVault, SSV, sealed volume, recovery |
| 16 | [network-config.md](16-network-config.md) | networksetup, firewall, DNS, VPN, Wi-Fi |
| 17 | [threat-hunting-recipes.md](17-threat-hunting-recipes.md) | Composite hunt queries, one-liners |
| 18 | [malware-analysis-static.md](18-malware-analysis-static.md) | Mach-O static triage on-host |
| 19 | [osquery-macos.md](19-osquery-macos.md) | osquery packs and SQL for macOS |
| 20 | [detection-engineering.md](20-detection-engineering.md) | Sigma/ES rules, telemetry sources |
| 21 | [incident-response-playbook.md](21-incident-response-playbook.md) | Containment, collection, chain of custody |
| 22 | [offensive-recon.md](22-offensive-recon.md) | Authorized pentest recon from a Mac |
| 23 | [scripting-toolkit.md](23-scripting-toolkit.md) | zsh/bash/jq/python patterns for security work |
| 24 | [ios-mobile-adjacent.md](24-ios-mobile-adjacent.md) | Sysdiagnose, device logs, MDM-adjacent |
| 25 | [quick-reference.md](25-quick-reference.md) | Paths, plists, DB locations, one-page index |

## † CONVENTIONS

- `#` prefix = needs root - `$` prefix = unprivileged - `<>` = replace - Destructive commands are tagged `- SIP-blocked commands are tagged [SIP]` they fail unless SIP is disabled (don't disable it on a production box)

## † VERSION NOTES

Apple moves paths and deprecates tools frequently. `systemsetup`, `networksetup`, and `defaults` behavior varies across releases. Verify on your target version before relying on any of this in an investigation report.

BOOK XIV

# FILEVAULT, ENCRYPTION, RECOVERY

*canticle of the 15th sheet*

## † FILEVAULT STATUS

BASH

```bash
fdesetup status

fdesetup status -extended 2>/dev/null

fdesetup list                                  # enabled users

fdesetup haspersonalrecoverykey

fdesetup hasinstitutionalrecoverykey

fdesetup isactive

diskutil apfs list | grep -iE 'FileVault|Encrypt'

diskutil apfs listCryptoUsers /

diskutil apfs listUsers /

diskutil info / | grep -iE 'encrypt|filevault'
```

## † ENABLING / MANAGING

```bash
# fdesetup enable

# fdesetup enable -user <user> -password <pw>

# fdesetup enable -defer /path/to/recovery.plist      # defer to next login

# fdesetup enable -outputplist > /tmp/fv.plist        # capture PRK

# fdesetup add -usertoadd <user>

# fdesetup remove -user <user>

# fdesetup changerecovery -personal                   # new PRK, printed once

# fdesetup changerecovery -institutional -keychain

# fdesetup validaterecovery

# fdesetup removerecovery -personal                   #

# fdesetup disable                                    #

# fdesetup sync                                       # sync enabled users w/ dir service

fdesetup authrestart -delayminutes -1               # unlock at next boot w/o password

`fdesetup authrestart` is genuinely useful for remote reboots of FileVault machines

stores the unlock key in memory across one reboot so the box comes back up reachable.
```

# fdesetup authrestart # reboot now, auto-unlock # fdesetup authrestart -delayminutes 0 # fdesetup supportsauthrestart

## † RECOVERY KEYS

Three types: - **Personal Recovery Key (PRK)** 24-character string, generated per machine - **Institutional Recovery Key (IRK)** organizational cert-based key (`FileVaultMaster.keychain`) - **iCloud recovery** key escrowed to Apple ID

```bash
# fdesetup changerecovery -personal

#

 prints "New personal recovery key = XXXX-XXXX-..." ONCE

ls -la /Library/Keychains/FileVaultMaster.keychain 2>/dev/null

security dump-keychain /Library/Keychains/FileVaultMaster.keychain 2>/dev/null

# fdesetup changerecovery -institutional -keychain

Escrow to MDM is normally via a `com.apple.security.FDERecoveryKeyEscrow` payload.
```

```bash
system_profiler SPConfigurationProfileDataType | grep -i -A5 escrow

ls -la /var/db/FileVaultPRK.dat 2>/dev/null
```

## † Secure Token / Volume Ownership Dependency

```bash
sysadminctl -secureTokenStatus <user>

diskutil apfs listUsers /

 shows "Volume Owner: Yes/No" per cryptouser on Apple silicon

# profiles status -type bootstraptoken

Chain: a user needs a **Secure Token** to be a FileVault unlocker. On Apple silicon they

additionally need **Volume Ownership** to authorize OS updates and some security
settings.

The first admin created gets a token automatically; subsequent users need it granted.
```

# sysadminctl -secureTokenOn <user> -password <pw> -adminUser <a> -adminPassword <ap> # sysadminctl interactive -secureTokenOn <user> -password -adminUser <a> -adminPassword -

## † ENCRYPTED EXTERNAL VOLUMES

BASH

```
diskutil list external

diskutil apfs list

# diskutil apfs encryptVolume /Volumes/Ext -user disk -passphrase

# diskutil apfs decryptVolume /Volumes/Ext -passphrase        #

# diskutil apfs unlockVolume disk4s1 -passphrase

# diskutil apfs lockVolume disk4s1

# diskutil apfs listCryptoUsers disk4s1

# diskutil apfs changePassphrase disk4s1 -user <uuid> -oldPassphrase <o> -newPassphrase
<n>

# CoreStorage (HFS+ legacy)

# diskutil cs list

# diskutil cs unlockVolume <lvUUID> -passphrase <pw>
```

## † ENCRYPTED DISK IMAGES

BASH

```
hdiutil create -size 500m -encryption AES-256 -stdinpass -fs APFS -volname Vault ~/vault.dmg

hdiutil create -size 1g -encryption AES-256 -type SPARSEBUNDLE -fs APFS -volname Vault ~/vault.sparsebundle

hdiutil attach ~/vault.dmg

hdiutil attach -stdinpass ~/vault.dmg

hdiutil detach /Volumes/Vault

hdiutil imageinfo ~/vault.dmg | grep -i encrypt

hdiutil chpass ~/vault.dmg

hdiutil resize -size 2g ~/vault.sparsebundle

Sparse bundles grow on demand and sync better; plain DMGs are simpler and less prone to

corruption. For a security notes/evidence vault, sparse bundle + AES-256 is the practical

choice.
```

## † FILE-LEVEL ENCRYPTION

BASH

```
# age (brew install age)

 modern, simple

age-keygen -o key.txt

age -r <recipient-pubkey> -o secret.age secret.txt

age -d -i key.txt secret.age

# GPG

gpg -c file                                # symmetric

gpg -e -r <recipient> file

gpg -d file.gpg

# openssl (avoid for new work

 poor KDF defaults)

openssl enc -aes-256-cbc -pbkdf2 -iter 600000 -salt -in f -out f.enc

openssl enc -d -aes-256-cbc -pbkdf2 -iter 600000 -in f.enc -out f
```

## † SECURE DELETION

```bash
# rm -P file                                # overwrite 3x

 ineffective on SSD/APFS

# srm                                       # removed from macOS

# diskutil secureErase 0 /dev/disk4         #

 whole disk

# diskutil secureErase freespace 0 /Volumes/X #

# diskutil eraseDisk APFS NewName /dev/disk4  #

On APFS/SSD, overwrite-in-place doesn't work

 wear leveling and copy-on-write mean the

original blocks may persist. The only reliable erasure is **cryptographic**: the disk is

encrypted, so destroying the key destroys the data.
```

# Apple silicon / T2: "Erase All Content and Settings" destroys the volume key # System Settings General Transfer or Reset Erase All Content and Settings # Or in Recovery: Disk Utility erase of the whole APFS container For a machine being decommissioned: confirm FileVault was on, then EACS/erase. That's it no need for multi-pass wiping.

## † BOOT / RECOVERY

BASH

```
# Apple silicon
#   Hold power button at boot
 Options
 Recovery
# bputil -d                                  # boot policy
# bputil -f                                  # full security
nvram -p
# nvram -c                                   #
 clear
# nvram boot-args="-v"                        #
 verbose boot
# Intel
#   Cmd-R (recovery), Opt-Cmd-R (internet recovery), Cmd-S (single user, pre-SIP)
firmwarepasswd -check                       # Intel firmware password
# firmwarepasswd -setpasswd                   # Intel only
# firmwarepasswd -delete
system_profiler SPiBridgeDataType           # T2 status
Apple silicon has no firmware password
 the equivalent protections are the Secure Enclave,
Activation Lock, and the boot policy set by `bputil`.
```

BASH

```
system_profiler SPHardwareDataType | grep -i 'activation lock'
```

## † Recovery Volume

```bash
diskutil list | grep -i recovery

diskutil apfs list | grep -i -A2 Recovery

# diskutil mount disk3s4

ls /Volumes/Recovery/ 2>/dev/null

csrutil status                          # from Recovery, can be changed
```

## † Snapshot-based rollback

```bash
tmutil listlocalsnapshots /

# Boot to Recovery

 Restore from Time Machine Backup

 pick a local snapshot

# diskutil apfs deleteSnapshot / -uuid <uuid>    #

Useful IR capability: if a snapshot predates the compromise, you have a clean-state

comparison point without needing an external backup.
```

## † Verifying encryption on a forensic image

```bash
hdiutil imageinfo image.dmg | grep -i encrypt

file image.dd

head -c 64 image.dd | xxd               # APFS container magic 'NXSB' at 0x20

strings -n 8 image.dd | head -20        # if you see readable text, it's not encrypted
```

## † QUICK POSTURE CHECK

BASH

```
#!/bin/zsh

print "FileVault:        $(fdesetup status | head -1)"

print "PRK present:      $(fdesetup haspersonalrecoverykey)"

print "IRK present:      $(fdesetup hasinstitutionalrecoverykey)"

print "SSV sealed:       $(csrutil authenticated-root status | sed 's/.*: //')"

print "SIP:              $(csrutil status | sed 's/.*: //')"

print "\nFV-enabled users:"

fdesetup list

print "\nSecure token status:"

for u in $(dscl . -list /Users UniqueID | awk '$2>=500{print $1}'); do

  printf '  %-18s %s\n' "$u" "$(sysadminctl -secureTokenStatus $u 2>&1 | grep -o
'ENABLED\|DISABLED')"

done

print "\nVolume owners:"

diskutil apfs listUsers / 2>/dev/null | grep -B2 'Volume Owner: Yes'
```

BOOK XV

# DETECTION ENGINEERING FOR MACOS

*canticle of the 20th sheet*

✦ ⁂ ✦

## † TELEMETRY SOURCES RANKED

| SOURCE | FIDELITY | COST | NOTES |
|---|---|---|---|
| Endpoint Security (ES) framework | Highest | Needs entitlement | Real-time, authoritative |
| `eslogger` | High | Root, local only | Free ES access; no fleet story |
| osquery + ES tables | High | Moderate | Fleet-capable |
| Unified log | Medium | High volume, `<private>` redaction | Broad coverage |
| Commercial EDR | High | $ | Usually ES-based underneath |
| `/var/log/install.log` | Medium | Free | Package installs only |
| FSEvents | Medium | Free | Post-hoc, no real-time |
| Network flow (firewall/DNS) | Medium | Varies | Off-host, tamper-resistant |
| BSM audit | Low | Deprecated | Don't build on it |

## † ES EVENT — ATT&CK MAPPING

| ES EVENT | ATT&CK |
|---|---|
| `exec` | T1059 Command & Scripting Interpreter |
| `btm_launch_item_add` | T1543.001 Launch Agent, T1547.015 Login Items |
| `profile_add` | T1176 Browser/Software Extensions, T1556 |
| `kextload` | T1547.006 Kernel Modules |
| `su`, `sudo` | T1548.003 Sudo Caching |
| `authentication` | T1078 Valid Accounts |
| `od_create_user`, `od_group_add` | T1136 Create Account, T1098 Account Manipulation |
| `create`/`write` on launch dirs | T1543 Create or Modify System Process |
| `setextattr`/`deleteextattr` | T1553.001 Gatekeeper Bypass |
| `mount`/`unmount` | T1052 Exfil over Physical Medium |
| `screensharing_attach` | T1021.001 Remote Services |
| `openssh_login` | T1021.004 SSH |
| `xp_malware_detected` | Detection by XProtect |
| `tcc_modify` | T1548 Abuse Elevation Control |
| `signal` (to security tools) | T1562.001 Disable/Modify Tools |

## † DETECTION RULES

▸ □1

AppleScript credential phishing

YAML

```yaml
title: macOS AppleScript Password Prompt
id: 8f2c1a90-0000-4000-a000-000000000001
status: experimental
description: osascript displaying a dialog requesting a password
 AMOS/Atomic-family stealer pattern
logsource:
  product: macos
  category: process_creation
detection:
  selection:
    Image|endswith: '/osascript'
    CommandLine|contains|all:
      - 'display dialog'
      - 'hidden answer'
  selection_admin:
    Image|endswith: '/osascript'
    CommandLine|contains: 'with administrator privileges'
  condition: selection or selection_admin
falsepositives:
  - Legitimate installers and helper tools requesting elevation
level: high
tags:
  - attack.credential_access
  - attack.t1056.002
```

```
# eslogger exec | jq -r 'select( # (.event.exec.target.executable.path | test("osascript$")) and # ((.event.exec.args | join(" ")) | test("display dialog|hidden answer"; "i")) # ) | [.time, .process.executable.path, (.event.exec.args|join(" "))] | @tsv'
```

## ▸ 02

Launch item created by non-installer process

```bash
# eslogger create write rename | jq -r 'select(

#   ((.event.create.destination.new_path.dir // .event.write.target.path // "") |
test("Launch(Agents|Daemons)")) and

#   ((.process.executable.path // "") | test("installd|package_script_service|
softwareupdated") | not)

# ) | [.time, .process.executable.path, (.event.create.destination.new_path.filename //
"")] | @tsv'
```

title: macOS Launch Persistence Written by Unexpected Process logsource: product: macos category: file_event detection: selection: TargetFilename|contains: - '/Library/LaunchDaemons/' - '/Library/LaunchAgents/' filter_legit: Image|endswith: - '/installd' - '/package_script_service' - '/softwareupdated' - '/system_installd' condition: selection and not filter_legit level: high tags: [attack.persistence, attack.t1543.001, attack.t1543.004]

## ▸ 03

Quarantine attribute removal

YAML

```yaml
title: macOS Gatekeeper Bypass via xattr
logsource:
  product: macos
  category: process_creation
detection:
  selection_xattr:
    Image|endswith: '/xattr'
    CommandLine|contains:
      - '-d com.apple.quarantine'
      - '-c '
      - '-cr'
  selection_spctl:
    Image|endswith: '/spctl'
    CommandLine|contains: '--master-disable'
  condition: 1 of selection_*
level: high
tags: [attack.defense_evasion, attack.t1553.001]
```

## ▸ 04

curl/wget piped to interpreter

```yaml
title: macOS Remote Payload Piped to Shell
logsource:
  product: macos
  category: process_creation
detection:
  selection:
    CommandLine|re: '(curl|wget)\s+[^|]*\|\s*(ba|z)?sh'
  condition: selection
falsepositives:
  - Homebrew and other legitimate installers
level: medium
tags: [attack.execution, attack.t1059.004, attack.command_and_control, attack.t1105]
```

## ▸ 05

TCC database modification

```bash
# eslogger tcc_modify | jq -r
'[.time, .event.tcc_modify.service, .event.tcc_modify.identity, .event.tcc_modify.update_type]
| @tsv'

# eslogger write | jq -r 'select((.event.write.target.path // "") | test("TCC\\.db")) |
[.time, .process.executable.path] | @tsv'
```

title: macOS TCC Database Direct Modification logsource: product: macos category: file_event detection: selection: TargetFilename|endswith: 'com.apple.TCC/TCC.db' filter: Image|endswith: - '/tccd' - '/tccutil' condition: selection and not filter level: critical tags: [attack.defense_evasion, attack.t1548]

## ▸ 06

Security tooling termination

```bash
# eslogger signal | jq -r 'select(

#   (.event.signal.target.executable.path // "") | test("osqueryd|falcond|SentinelAgent|
CbOsxSensorService|Little Snitch|LuLu|BlockBlock")

# ) |
[.time, .process.executable.path, .event.signal.sig, .event.signal.target.executable.path]
| @tsv'
```

title: macOS Security Tool Termination logsource: product: macos category: process_creation detection: selection_kill: Image|endswith: ['/kill', '/killall', '/pkill'] CommandLine|contains: ['osqueryd','falcond','SentinelAgent','LuLu','BlockBlock','Little Snitch'] selection_launchctl: Image|endswith: '/launchctl' CommandLine|contains|all: ['bootout'] CommandLine|contains: ['crowdstrike','sentinelone','osquery','carbonblack'] condition: 1 of selection_* level: critical tags: [attack.defense_evasion, attack.t1562.001]

### ▸ 07

Log/artifact destruction

YAML

```yaml
title: macOS Anti-Forensics
logsource:
  product: macos
  category: process_creation
detection:
  sel_log:
    Image|endswith: '/log'
    CommandLine|contains: 'erase'
  sel_history:
    CommandLine|contains:
      - 'rm ~/.zsh_history'
      - 'rm ~/.bash_history'
      - 'unset HISTFILE'
      - 'history -c'
  sel_snapshot:
    Image|endswith: '/tmutil'
    CommandLine|contains: 'deletelocalsnapshots'
  sel_fsevents:
    CommandLine|contains: '/.fseventsd'
  condition: 1 of sel_*
level: high
tags: [attack.defense_evasion, attack.t1070]
```

## ▸ 08

Suspicious parent-child

```bash
# eslogger exec | jq -r '

#   . as $e |

#   ($e.process.executable.path // "") as $parent |

#   ($e.event.exec.target.executable.path // "") as $child |

#   select(

#     ($parent | test("Microsoft (Word|Excel|PowerPoint)|Safari|Google Chrome|Firefox|Mail")) and

#     ($child  | test("/(sh|bash|zsh|osascript|python3?|curl|wget|perl|ruby)$"))

#   ) | [$e.time, $parent, $child, ($e.event.exec.args|join(" "))] | @tsv'
```

## ▸ 09

Root CA installation

```yaml
title: macOS Root Certificate Added to Trust Store

logsource:

  product: macos

  category: process_creation

detection:

  selection:

    Image|endswith: '/security'

    CommandLine|contains: 'add-trusted-cert'

  condition: selection

level: high

tags: [attack.defense_evasion, attack.t1553.004]
```

## ▸ 10

Remote Management / ARD activation

YAML

```yaml
title: macOS Remote Management Enabled
logsource:
  product: macos
  category: process_creation
detection:
  sel_kickstart:
    Image|endswith: '/kickstart'
    CommandLine|contains: '-activate'
  sel_ssh:
    Image|endswith: '/systemsetup'
    CommandLine|contains: '-setremotelogin on'
  sel_screensharing:
    Image|endswith: '/launchctl'
    CommandLine|contains: 'com.apple.screensharing'
  condition: 1 of sel_*
level: high
tags: [attack.lateral_movement, attack.t1021.001]
```

### ▸ ☐ 11

Beaconing launch item Correlation rule, not a single event: 1. `btm_launch_item_add` or file write to a Launch dir 2. Plist contains `StartInterval` < 3600 3. Referenced binary is unsigned or ad-hoc 4. That binary subsequently makes outbound connections at matching intervals

BASH

```bash
# Detect the interval pattern from connection timestamps

log show --last 24h --predicate 'process == "<name>"' --info --style ndjson | \

    jq -r '.timestamp' | while read t; do date -j -f '%Y-%m-%d %H:%M:%S' "${t%%.*}" +%s
2>/dev/null; done | \

    awk 'NR>1{print $1-prev} {prev=$1}' | sort -n | uniq -c | sort -rn | head

Low jitter (a tight cluster of identical deltas) is the beacon signal.
```

### ▸ D12

Dylib injection

YAML

```yaml
title: macOS DYLD Injection

logsource:

  product: macos

  category: process_creation

detection:

  selection:

    CommandLine|contains:

      - 'DYLD_INSERT_LIBRARIES'

      - 'DYLD_LIBRARY_PATH'

      - 'DYLD_FRAMEWORK_PATH'

  condition: selection

level: high

tags: [attack.persistence, attack.privilege_escalation, attack.t1574.006]
```

## † BASELINE BEFORE YOU DETECT

Every rule above needs tuning against your environment. Do this first:

```bash
# 1. Inventory what legitimately writes to launch dirs

# eslogger create write > /tmp/baseline.ndjson &

#   ... run for a week across representative machines ...

jq -r 'select((.event.create.destination.new_path.dir // "") | test("Launch"))
| .process.executable.path' \

    /tmp/baseline.ndjson | sort | uniq -c | sort -rn

# 2. Inventory normal osascript usage

log show --last 30d --predicate 'process == "osascript"' --info | \

    grep -oE '/[A-Za-z0-9/. _-]+' | sort | uniq -c | sort -rn | head -30

# 3. Inventory normal parent-child pairs

jq -r 'select(.event.exec) |
[.process.executable.path, .event.exec.target.executable.path] | @tsv' \

    /tmp/baseline.ndjson | sort | uniq -c | sort -rn | head -50
```

## † RULE QUALITY CHECKLIST

For each rule ask: - What's the expected daily volume in *this* environment? - What legitimate software produces this pattern? (macOS admin tooling produces a *lot* of it) - Can an attacker trivially avoid it? (e.g. renaming a binary defeats name-based rules hash or signature is better) - Does it fire on the action, or on a fragile artifact of one implementation? - What's the analyst's next step when it fires? If there isn't one, the rule isn't ready.

## † TESTING DETECTIONS

```bash
# Atomic Red Team has macOS coverage
git clone https://github.com/redcanaryco/atomic-red-team
ls atomic-red-team/atomics/T1543.001/          # Launch Agent
ls atomic-red-team/atomics/T1553.001/          # Gatekeeper bypass
ls atomic-red-team/atomics/T1056.002/          # GUI input capture
# Run in a VM, never on a production host
pwsh -c 'Invoke-AtomicTest T1543.001 -TestNumbers 1'
Other macOS-relevant test tooling: `macOS Attack Framework`, Objective-See's tools for
validating that your monitoring actually catches persistence events.
```

## † OBJECTIVE-SEE TOOLS (FREE, HIGH QUALITY)

| TOOL | PURPOSE |
|---|---|
| **BlockBlock** | Alerts on persistence installation |

best single free defensive tool for macOS |

| **LULU** | OUTBOUND FIREWALL |
|---|---|
| **KnockKnock** | Persistence enumeration (GUI version of Sheet 02) |
| **TaskExplorer** | Process inspection with signature/VT lookup |
| **Netiquette** | Network connection viewer |
| **ReiKey** | Detects event tap keyloggers |
| **OverSight** | Camera/mic access alerts |
| **DHS** | Dylib hijack scanner |
| **What's Your Sign?** | Finder signature checks |

BASH

```
brew install --cask blockblock lulu knockknock taskexplorer netiquette oversight

Worth installing BlockBlock and LuLu on a working machine even if you have EDR

 they

cover the two things macOS EDR most often misses, and the alerting is immediate.
```

BOOK XVI

# Network Forensics & Egress Analysis

*canticle of the 5th sheet*

✦ ⁂ ✦

## † Current Connections

BASH

```
netstat -anv -p tcp

netstat -anv -p udp

netstat -an | grep -E 'LISTEN|ESTABLISHED'

netstat -nr                           # routing table

netstat -s                            # protocol stats

netstat -i                            # per-interface counters

# lsof -i -n -P

# lsof -i -n -P | grep LISTEN

# lsof -i -n -P | grep ESTABLISHED

# lsof -i TCP:22 -n -P

# lsof -i @192.0.2.10 -n -P

# lsof -i -n -P -c <procname>

`-n` skips DNS resolution, `-P` skips port-name resolution. Always use both during IR

you don't want your triage generating DNS lookups.
```

## † PER-PROCESS BANDWIDTH

BASH

```
nettop

nettop -P                                # per-process aggregate

nettop -P -L 1 -J bytes_in,bytes_out     # one sample, specific cols

nettop -p <pid>

nettop -m tcp -P

nettop -m route
```

## † SOCKETS WITH OWNING PROCESS, CLEANLY

BASH

```
# lsof -i -n -P +c 0 | awk 'NR==1 || /LISTEN|ESTABLISHED/ {print $1,$2,$3,$8,$9,$10}'

# lsof -i -n -P +c 0 -F pcn | paste - - - -   # machine-parsable
```

## † DNS

BASH

```
scutil --dns                            # full resolver config

cat /etc/resolv.conf                    # generated, may lie

dscacheutil -q host -a name example.com

# dscacheutil -flushcache; sudo killall -HUP mDNSResponder   # flush

dig example.com

dig +short example.com

dig @1.1.1.1 example.com

dig -x 192.0.2.10                       # reverse

dig example.com ANY +noall +answer

host example.com

nslookup example.com

networksetup -getdnsservers Wi-Fi

networksetup -listallnetworkservices
```

### ▸ LIVE DNS QUERY MONITORING

BASH

```
# log stream --predicate 'process == "mDNSResponder"' --info | grep -i 'Question\|Q<'

# log show --last 1h --predicate 'process == "mDNSResponder"' --info | grep -oE '[A-Za-z0-9.-]+\.(com|net|org|io|ru|cn|top|xyz)' | sort | uniq -c | sort -rn | head -50

# dtrace-free alternative: pcap on port 53

# tcpdump -i any -n port 53 -l | awk '{print $NF}'
```

### ▸ ENCRYPTED DNS CHECK

BASH

```
scutil --dns | grep -i 'DNS over\|options'

sudo profiles -P | grep -i dnssettings

log show --last 1d --predicate 'eventMessage CONTAINS "DoH" OR eventMessage CONTAINS
"DoT"' --info
```

## † PACKET CAPTURE

BASH

```
# tcpdump -D                              # list interfaces

# tcpdump -i en0 -nn -s0 -w /tmp/cap.pcap

# tcpdump -i any -nn -s0 -w /tmp/all.pcap

# tcpdump -i en0 -nn 'tcp port 443'

# tcpdump -i en0 -nn 'host 192.0.2.10'

# tcpdump -i en0 -nn 'not port 22 and not port 53'

# tcpdump -i en0 -nn -A 'tcp port 80'     # ASCII payload

# tcpdump -i en0 -nn -X 'udp port 53'     # hex+ascii

# tcpdump -r /tmp/cap.pcap -nn | head -100

# tcpdump -r /tmp/cap.pcap -nn 'tcp[tcpflags] & tcp-syn ≠ 0 and tcp[tcpflags] & tcp-ack
≠ 0'
```

## THE MACOS-SPECIFIC ONE: PKTAP

BASH

```
# tcpdump -i pktap,all -nn -w /tmp/all.pcap     # capture across all ifaces w/ proc
metadata

# tcpdump -i pktap -k -nn                       # show process name per packet

# tcpdump -Q in -i en0 -nn                      # direction filter

`pktap` gives you the owning process name per packet

 extremely useful and unique to

macOS/BSD. Use `-k` to print the metadata.
```

## WIRELESS MONITOR MODE

BASH

```
sudo /System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/
airport en0 sniff 6

# tcpdump -I -i en0 -nn                   # monitor mode

sudo wdutil info                        # modern Wi-Fi diagnostics (macOS 12+)

The `airport` binary was removed in macOS 14.4+. Use `wdutil` and the Wireless
Diagnostics

app instead on newer systems.
```

## APPLICATION-LAYER CAPTURE

BASH

```
# mitmproxy --mode transparent            # brew install mitmproxy

tshark -r /tmp/cap.pcap -Y 'http.request' -T fields -e http.host -e http.request.uri

tshark -r /tmp/cap.pcap -Y 'tls.handshake.type == 1' -T fields -e
tls.handshake.extensions_server_name | sort -u

tshark -r /tmp/cap.pcap -q -z conv,tcp

tshark -r /tmp/cap.pcap -q -z io,phs    # protocol hierarchy

capinfos /tmp/cap.pcap
```

## † FIREWALL

### ▸ APPLICATION LAYER FIREWALL (ALF)

BASH

```
FW=/usr/libexec/ApplicationFirewall/socketfilterfw

# $FW --getglobalstate

# $FW --setglobalstate on

# $FW --getallowsigned

# $FW --setallowsigned off

# $FW --setallowsignedapp off

# $FW --getstealthmode

# $FW --setstealthmode on

# $FW --getblockall

# $FW --setblockall on                    #
 blocks incoming

# $FW --listapps

# $FW --add /Applications/Foo.app

# $FW --blockapp /Applications/Foo.app

# $FW --getloggingmode

# $FW --setloggingmode on

# $FW --getloggingopt

cat /Library/Preferences/com.apple.alf.plist | plutil -p - 2>/dev/null

ALF only filters *inbound*. It does nothing for egress. For outbound control you need PF

or a NetworkExtension-based tool (LuLu, Little Snitch).
```

## ▸ PF (PACKET FILTER)

BASH

```
# pfctl -s info

# pfctl -s rules

# pfctl -s nat

# pfctl -s states

# pfctl -s all

# pfctl -e                              # enable

# pfctl -d                              #

 disable

# pfctl -f /etc/pf.conf                 # load

# pfctl -nf /etc/pf.conf                # syntax check only

# pfctl -a com.apple/anchor -s rules    # per-anchor

# pfctl -s Anchors -v

# pfctl -T show -t <table>

cat /etc/pf.conf

ls /etc/pf.anchors/

Minimal egress-block anchor:

# /etc/pf.anchors/custom.egress

block out quick proto tcp to any port 4444

block out quick from any to 192.0.2.0/24

pass out quick proto tcp to any port {80, 443, 53}
```

# echo 'anchor "custom.egress"' >> /etc/pf.conf # echo 'load anchor "custom.egress" from "/etc/pf.anchors/custom.egress"' >> /etc/pf.conf # pfctl -nf /etc/pf.conf && sudo pfctl -f /etc/pf.conf && sudo pfctl -e

## † EGRESS TESTING

BASH

```bash
nc -zv <host> 22 80 443

nc -zv -w2 <host> 1-1024 2>&1 | grep succeeded

nc -u -zv <host> 53

curl -sv --max-time 5 https://example.com -o /dev/null

curl -s ifconfig.me; echo                # external IP

curl -s https://ipinfo.io/json | jq .

dig +short myip.opendns.com @resolver1.opendns.com

traceroute -n example.com

traceroute -P TCP -p 443 example.com

mtr -r -c 10 example.com                 # brew install mtr

ping -c 4 example.com

nmap -Pn -p- --min-rate 1000 <target>    # brew install nmap; authorized targets only
```

## † INTERFACE & CONFIG

BASH

```bash
ifconfig -a

ifconfig en0

networksetup -listallhardwareports

networksetup -listnetworkserviceorder

networksetup -getinfo Wi-Fi

networksetup -getairportnetwork en0

ipconfig getpacket en0                  # DHCP lease detail

ipconfig getsummary en0

scutil --nwi                            # network information

scutil --proxy                          # proxy config

scutil <<< 'list'                       # dynamic store browse

route -n get default

arp -an

ndp -an                                 # IPv6 neighbors
```

## † PROXY & VPN CHECKS

BASH

```
scutil --proxy

networksetup -getwebproxy Wi-Fi

networksetup -getsecurewebproxy Wi-Fi

networksetup -getsocksfirewallproxy Wi-Fi

networksetup -getautoproxyurl Wi-Fi

env | grep -i proxy

cat ~/.curlrc ~/.wgetrc 2>/dev/null

scutil --nc list                        # VPN configurations

scutil --nc status <service>

systemextensionsctl list | grep -i network

ifconfig | grep -E '^(utun|ipsec|ppp|tun|tap)'

Rogue `utun` interfaces or an unexpected `--autoproxyurl` (PAC file) are common

persistence/exfil mechanisms.
```

## † WI-FI FORENSICS

BASH

```
sudo wdutil info

networksetup -listpreferredwirelessnetworks en0

defaults read /Library/Preferences/com.apple.wifi.known-networks.plist 2>/dev/null

plutil -p /Library/Preferences/com.apple.wifi.known-networks.plist 2>/dev/null | grep -E
'SSID|AddedAt|JoinedBy'

log show --last 7d --predicate 'subsystem == "com.apple.wifi"' --info | head -100

system_profiler SPAirPortDataType
```

## † BLUETOOTH

```bash
system_profiler SPBluetoothDataType

defaults read /Library/Preferences/com.apple.Bluetooth 2>/dev/null | head -50

log show --last 1d --predicate 'subsystem == "com.apple.bluetooth"' --info | head -50

blueutil -p                              # brew install blueutil

blueutil --paired
```

## † SHARING SERVICES (ATTACK SURFACE)

```bash
# launchctl print-disabled system | grep -E 'ssh|screensharing|smb|afp|ftp'

sudo systemsetup -getremotelogin

sudo systemsetup -getremoteappleevents

# launchctl list | grep -E 'smbd|screensharing|ARDAgent|ftp|nfs'

sudo lsof -i -n -P | grep -E ':22|:445|:548|:5900|:3283|:88'

ls -la /etc/ssh/sshd_config

sudo sshd -T 2>/dev/null | grep -iE 'permitroot|passwordauth|pubkeyauth|port|allowusers'

smbutil statshares -a

sudo serveradmin list 2>/dev/null
```

## † Historic Network Artifacts

BASH

```
log show --last 7d --predicate 'subsystem == "com.apple.network"' --info | head -200

plutil -p ~/Library/Preferences/com.apple.networkextension.plist 2>/dev/null

ls -la /Library/Preferences/SystemConfiguration/

plutil -p /Library/Preferences/SystemConfiguration/preferences.plist | head -100

plutil -p /Library/Preferences/SystemConfiguration/com.apple.airport.preferences.plist
2>/dev/null

sqlite3 ~/Library/Application\ Support/Knowledge/knowledgeC.db \

    "SELECT * FROM ZOBJECT WHERE ZSTREAMNAME LIKE '%usage%' LIMIT 20;" 2>/dev/null
```

## † Quick Egress Hunt One-Liners

BASH

```
# Every remote IP a process is talking to, deduped

# lsof -i -n -P | awk '/ESTABLISHED/{split($9,a,"→"); split(a[2],b,":"); print $1,
b[1]}' | sort -u

# Non-standard ports

# lsof -i -n -P | awk '/ESTABLISHED/{split($9,a,"→"); split(a[2],b,":"); if (b[2]≠443
&& b[2]≠80 && b[2]≠53) print}'

# Resolve and geolocate everything currently connected

# lsof -i -n -P | grep ESTABLISHED | grep -oE '\→([0-9]{1,3}\.){3}[0-9]{1,3}' | tr -d
'›—' | sort -u | \

#   while read ip; do echo -n "$ip  "; curl -s "https://ipinfo.io/$ip/org"; done

# Long-lived connections (beacons often are)

# lsof -i -n -P | grep ESTABLISHED | awk '{print $2}' | sort -u | while read p; do

#   ps -p $p -o pid,etime,command=; done
```

BOOK XVII

# macOS Persistence Hunting

*canticle of the 2nd sheet*

✦ ⁂ ✦

Every practical persistence surface, ordered roughly by frequency in the wild.

# † THE CANONICAL LIST

| # | Mechanism | Path / Command | Root? |
|---|---|---|---|
| 1 | LaunchAgent (user) | `~/Library/LaunchAgents/` | no |
| 2 | LaunchAgent (system) | `/Library/LaunchAgents/` | yes |
| 3 | LaunchDaemon | `/Library/LaunchDaemons/` | yes |
| 4 | Login Items | `~/Library/Application Support/com.apple.backgroundtaskmanagementagent/BackgroundItems-v*.btm` | no |
| 5 | SMAppService | app bundle `Contents/Library/LaunchDaemons/` | varies |
| 6 | Cron | `crontab -l`, `/usr/lib/cron/tabs/` | varies |
| 7 | Periodic scripts | `/etc/periodic/{daily,weekly,monthly}/` | yes |
| 8 | Shell rc files | `~/.zshrc` `~/.zshenv` `~/.zprofile` `/etc/zshenv` | no |
| 9 | Configuration Profile | `profiles -P`, `/var/db/ConfigurationProfiles/` | yes |
| 10 | Kernel extension | `/Library/Extensions/`, `kmutil showloaded` | yes |
| 11 | System extension | `systemextensionsctl list` | yes |
| 12 | Dylib hijack / insert | `DYLD_INSERT_LIBRARIES`, `@rpath` abuse | no |
| 13 | Login/Logout hooks | `defaults read com.apple.loginwindow LoginHook` | yes |
| 14 | Emond (removed 13+) | `/etc/emond.d/rules/` | yes |
| 15 | AuthorizationPlugin | `/Library/Security/SecurityAgentPlugins/` | yes |
| 16 | Directory Services plugin | `/Library/DirectoryServices/PlugIns/` | yes |
| 17 | ScriptingAdditions (OSAX) | `/Library/ScriptingAdditions/` | yes |
| 18 | QuickLook plugin | `~/Library/QuickLook/`, `/Library/QuickLook/` | no |
| 19 | Spotlight importer | `/Library/Spotlight/` | yes |
| 20 | Audio/CoreMedia plugin | `/Library/Audio/Plug-Ins/HAL/` | yes |
| 21 | Screensaver | `~/Library/Screen Savers/` | no |
| 22 | Finder Sync / App Extension | `pluginkit -mAvvv` | no |
| 23 | Safari extension | `~/Library/Safari/Extensions/` | no |
| 24 | Browser extension (Chrome/FF) | profile `Extensions/` dirs | no |

| # | Mechanism | Path / Command | Root? |
|---|---|---|---|
| 25 | at jobs | `/var/at/jobs/`, `atq` | varies |
| 26 | rc.common / rc.local | `/etc/rc.common`, `/etc/rc.local` | yes |
| 27 | StartupItems (legacy) | `/Library/StartupItems/` | yes |
| 28 | Folder Actions | `~/Library/Workflows/Applications/Folder Actions/` | no |
| 29 | Automator / Shortcuts | `~/Library/Services/`, Shortcuts DB | no |
| 30 | sudoers drop-in | `/etc/sudoers.d/` | yes |
| 31 | PAM module | `/etc/pam.d/` | yes |
| 32 | SSH authorized_keys | `~/.ssh/authorized_keys` | no |
| 33 | Re-open apps on login | `~/Library/Preferences/ByHost/com.apple.loginwindow.*.plist` | no |
| 34 | Dock persistent-apps | `defaults read com.apple.dock` | no |
| 35 | Font (Mach-O in font dir) | `~/Library/Fonts/` | no |
| 36 | XPC service in app bundle | `<app>/Contents/XPCServices/` | no |
| 37 | Print driver / CUPS backend | `/usr/libexec/cups/backend/` | yes |
| 38 | NetworkExtension provider | `systemextensionsctl list` | yes |
| 39 | EFI / firmware | `bputil -d`, `eficheck` (Intel) | yes |
| 40 | Trojanized dotfile in `/etc/paths.d/` | `/etc/paths.d/` | yes |

## † ENUMERATE EVERYTHING AT ONCE

BASH

```
#!/bin/zsh

# persistence-sweep.zsh

 run with sudo for full coverage

for d in \

  /Library/LaunchAgents /Library/LaunchDaemons \

  /System/Library/LaunchAgents /System/Library/LaunchDaemons \

  /Users/*/Library/LaunchAgents \

  /Library/StartupItems /Library/ScriptingAdditions \

  /Library/Security/SecurityAgentPlugins /Library/DirectoryServices/PlugIns \

  /Library/QuickLook /Users/*/Library/QuickLook \

  /Library/Spotlight /Library/Audio/Plug-Ins/HAL \

  /Users/*/Library/Screen\ Savers /Library/Screen\ Savers \

  /etc/periodic/daily /etc/periodic/weekly /etc/periodic/monthly \

  /etc/sudoers.d /etc/paths.d /etc/pam.d \

  /Library/Extensions /var/at/jobs

  [[ -d $d ]] || continue

  print -- "\n=== $d"

  ls -la -- "$d" 2>/dev/null

done
```

## † LaunchAgents / LaunchDaemons deep dive

BASH

```
# Dump every plist as readable XML
for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do
    echo "### $p"; plutil -p "$p"; done
# Just the executables referenced
for p in /Library/Launch{Agents,Daemons}/*.plist; do
    /usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null \
      || /usr/libexec/PlistBuddy -c 'Print :Program' "$p" 2>/dev/null; done | sort -u
# Non-Apple ones only
for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do
    l=$(basename "$p"); [[ $l == com.apple.* ]] && continue; echo "$p"; done
# RunAtLoad + KeepAlive = classic persistence
grep -l -e RunAtLoad -e KeepAlive /Library/Launch{Agents,Daemons}/*.plist 2>/dev/null
# StartInterval (beaconing)
for p in /Library/Launch{Agents,Daemons}/*.plist ~/Library/LaunchAgents/*.plist; do
    i=$(/usr/libexec/PlistBuddy -c 'Print :StartInterval' "$p" 2>/dev/null) && echo "$i
$p"; done
# Signature check on every referenced binary
for p in /Library/Launch{Agents,Daemons}/*.plist; do
    b=$(/usr/libexec/PlistBuddy -c 'Print :ProgramArguments:0' "$p" 2>/dev/null)
    [[ -f $b ]] && { printf '%-60s ' "$b"; codesign -dv "$b" 2>&1 | grep -m1
'^Authority' || echo UNSIGNED; }
  done
```

## † LOGIN ITEMS (MODERN, BTM)

BASH

```
# macOS 13+ background task management

# sfltool dumpbtm                     # full BTM dump, best single source

sfltool dumpbtm 2>/dev/null | grep -E 'Name:|Developer Name:|Program:|Disposition:'

# Legacy SharedFileList

ls ~/Library/Application\ Support/com.apple.sharedfilelist/

plutil -p ~/Library/Application\ Support/com.apple.sharedfilelist/
com.apple.LSSharedFileList.Items*.sfl* 2>/dev/null

# AppleScript view

osascript -e 'tell application "System Events" to get properties of every login item'
```

## † SYSTEM & KERNEL EXTENSIONS

BASH

```
systemextensionsctl list

kmutil showloaded --list-only

kmutil showloaded --variant-suffix release | grep -v com.apple

kextstat | grep -v com.apple          # Intel / older

ls -la /Library/Extensions/

ls -la /Library/SystemExtensions/

codesign -dv --verbose=4 /Library/Extensions/*.kext 2>&1 | grep -E 'Authority|Identifier'
```

## † DYLIB HIJACKING INDICATORS

BASH

```
otool -L <binary>                       # linked libs
otool -l <binary> | grep -A2 LC_RPATH   # rpath entries
otool -l <binary> | grep -A2 LC_LOAD_WEAK_DYLIB  # weak = hijackable
launchctl print system | grep -i DYLD
launchctl getenv DYLD_INSERT_LIBRARIES
for p in /Library/Launch{Agents,Daemons}/*.plist; do
    plutil -p "$p" | grep -i -B5 DYLD_INSERT && echo "  ^^ $p"; done
```

## † LOGIN/LOGOUT HOOKS & PROFILES

BASH

```
defaults read com.apple.loginwindow 2>/dev/null
# defaults read /var/root/Library/Preferences/com.apple.loginwindow LoginHook 2>/dev/null
sudo profiles -P                        # installed profiles
sudo profiles show -type enrollment
ls -la /var/db/ConfigurationProfiles/Store/
```

## † APP EXTENSIONS & PLUGINS

BASH

```
pluginkit -mAvvv                        # all registered extensions
pluginkit -mAvvv | grep -v com.apple
pluginkit -m -p com.apple.share-services
pluginkit -m -p com.apple.FinderSync
pluginkit -m -p com.apple.quicklook.preview
```

## † BROWSER EXTENSIONS

```bash
ls -la ~/Library/Safari/Extensions/ 2>/dev/null

plutil -p ~/Library/Safari/AppExtensions/Extensions.plist 2>/dev/null

ls ~/Library/Application\ Support/Google/Chrome/*/Extensions/

for m in ~/Library/Application\ Support/Google/Chrome/*/Extensions/*/*/manifest.json; do

    echo "═══ $m"; jq -r '.name, .permissions // empty' "$m" 2>/dev/null; done

ls ~/Library/Application\ Support/Firefox/Profiles/*/extensions/
```

## † SCHEDULED EXECUTION

```bash
crontab -l 2>/dev/null

# crontab -l -u root

ls -la /usr/lib/cron/tabs/           # actual crontab storage

atq; ls -la /var/at/jobs/

ls -la /etc/periodic/*/

cat /etc/defaults/periodic.conf
```

## † TIMELINE THE PERSISTENCE ARTIFACTS

```bash
# Everything in launch dirs sorted by mtime

find /Library/Launch* /Users/*/Library/LaunchAgents /System/Library/Launch* -type f \

    -exec stat -f '%Sm %N' -t '%Y-%m-%dT%H:%M:%S' {} \; 2>/dev/null | sort

# Birthtime vs mtime mismatch (timestomping)

stat -f 'birth=%SB mod=%Sm acc=%Sa chg=%Sc %N' -t '%Y-%m-%dT%H:%M:%S' /Library/
LaunchDaemons/*
```

## † KNOWN-GOOD BASELINE DIFF

BASH

```
# On a clean box

find /Library/Launch{Agents,Daemons} -type f | sort > baseline.txt

# Later, on the suspect box

find /Library/Launch{Agents,Daemons} -type f | sort > current.txt

diff baseline.txt current.txt

# With hashes

find /Library/Launch{Agents,Daemons} -type f -exec shasum -a 256 {} \; | sort -k2 >
baseline.sha
```

BOOK XVIII

# PROCESS, DYLIB & MEMORY INSPECTION

*canticle of the 4th sheet*

✦ ⁂ ✦

## † PROCESS LISTING

BASH

```
ps auxwww

ps -axo pid,ppid,uid,user,ruser,lstart,etime,%cpu,%mem,rss,vsz,stat,tty,command

ps -axo pid,ppid,command | sort -k2 -n          # crude parent grouping

ps -p <pid> -o pid,ppid,uid,lstart,command

ps -A -o pid,ppid,user,command | grep -v '^\s*[0-9]* *[01] '   # non-launchd-parented

STAT column: `R` running, `S` sleeping, `I` idle, `T` stopped, `U` uninterruptible,

`Z` zombie; `+` foreground, `s` session leader, `E` exiting, `<` high priority.
```

## † MODERN TOOLING

```bash
top -o cpu -n 20

top -o mem -n 20

htop                                # brew install htop

procs                               # brew install procs

 nice modern ps

pgrep -fl <pattern>

pgrep -P <ppid>                     # children

pkill -f <pattern>                  #

lsappinfo list                      # GUI apps with bundle IDs

lsappinfo info -only bundleid <asn>
```

## † PROCESS TREE

```bash
pstree                              # brew install pstree

pstree -p <pid>

ps -axo pid,ppid,command | awk '{print $1,$2}' | \

    python3 -c "

import sys,collections

ch=collections.defaultdict(list)

for l in sys.stdin.readlines()[1:]:

    p,pp=l.split()[:2]; ch[pp].append(p)

def walk(p,d=0):

    print(' '*d+p)

    for c in ch.get(p,[]): walk(c,d+2)

walk('1')"
```

## † What a process has open

BASH

```
# lsof -p <pid>

# lsof -p <pid> -a -d txt                # loaded images

# lsof -i -a -p <pid>                    # its sockets

# lsof +D /Users/<u>/Documents           # who's touching this dir

# lsof -u <user>

# lsof /path/to/file                     # who has this file open

vmmap <pid> | head -50                 # memory regions

vmmap --summary <pid>

sample <pid> 5 -file /tmp/sample.txt   # 5s stack sample

spindump <pid> 5 -file /tmp/spin.txt   # heavier, root

heap <pid>                             # heap contents summary

leaks <pid>

footprint <pid>
```

## † Loaded libraries per process

BASH

```
# vmmap <pid> | grep -E '\.dylib|\.framework' | awk '{print $NF}' | sort -u

# lsof -p <pid> | awk '$4=="txt"{print $NF}' | sort -u

dyldinfo -dylibs <binary> 2>/dev/null || otool -L <binary>

# Anything loaded from a writable / unusual path

# for p in $(pgrep -x -d' ' ''); do :; done

# lsof -c '' -d txt 2>/dev/null | awk '{print $NF}' | grep -vE '^/(System|usr/lib|
Library/Apple)' | sort -u
```

## † SIGNATURE VERIFICATION ON RUNNING PROCESSES

BASH

```
codesign -dv --verbose=4 /path/to/binary 2>&1

codesign --verify --deep --strict --verbose=2 /Applications/Foo.app

codesign -d --entitlements - /path/to/binary 2>/dev/null | plutil -p -

# Every running process with its signing authority

ps -axo pid,comm= | while read pid path; do

    [[ -f "$path" ]] || continue

    auth=$(codesign -dv "$path" 2>&1 | awk -F= '/^Authority/{print $2; exit}')

    printf '%-8s %-50s %s\n' "$pid" "${path:t}" "${auth:-UNSIGNED}"

  done | sort -k3
```

## † UNSIGNED / AD-HOC / SUSPICIOUS BINARY HUNT

BASH

```
# Ad-hoc signed (linker-signed or self-signed)

codesign -dv <bin> 2>&1 | grep -q 'Signature=adhoc' && echo ADHOC

# Team ID of every running process

ps -axo comm= | sort -u | while read p; do

    [[ -f "$p" ]] && printf '%s\t%s\n' "$(codesign -dv "$p" 2>&1 | awk -F= '/
TeamIdentifier/{print $2}')" "$p"

  done | sort

# Processes running out of world-writable or user-writable dirs

ps -axo pid,comm= | awk '$2 ~ /^\/(tmp|var\/tmp|Users\/[^\/]+\/(Library\/Application
Support|Downloads|Public))/'
```

## † Environment & Arguments of a Live Process

```bash
ps -p <pid> -o command=                # args (truncated)

ps -Eww -p <pid>                       # args + environment (needs same-user or root)

# ps -Eauxww | grep DYLD

launchctl procinfo <pid>               # rich: domain, entitlements, spawn info

 root

`launchctl procinfo` is the single most useful process introspection command on macOS.

It shows the launchd job, path, arguments, environment, entitlements, and code signing

flags in one shot.
```

# launchctl procinfo <pid> | sed -n '1,80p' # launchctl procinfo <pid> | grep -A20 'entitlements'

## † launchd Domain Inspection

```bash
# launchctl print system

# launchctl print system/<label>

launchctl print gui/$(id -u)

launchctl print gui/$(id -u)/<label>

launchctl print user/$(id -u)

launchctl list

launchctl list <label>

launchctl dumpstate | head -200        # huge
```

## † Memory Acquisition

Full RAM capture on modern macOS is hard: SIP blocks `/dev/mem`, and Apple silicon has no supported acquisition path. Options:

BASH

```
# Per-process core dump (root, and SIP restrictions apply to protected procs)

# gcore -o /tmp/core <pid>

# lldb -p <pid> -o "process save-core /tmp/core.dmp" -o "detach" -o "quit"

# Strings from a core

strings -a -n 8 /tmp/core.<pid> | grep -iE 'https?://|BEGIN.*KEY|password'

# Live process memory read via lldb (SIP-restricted targets will refuse)

# lldb -p <pid>

#   (lldb) memory read --size 1 --format x --count 256 0x...

#   (lldb) image list

#   (lldb) process save-core /tmp/x.core

For real memory forensics you generally need a kext/system-extension based acquisition

tool, which requires SIP posture changes

 not something to do on a production host

mid-incident without planning.
```

## † VOLATILE DATA TO GRAB BEFORE POWER-OFF

BASH

```
date -u '+%Y-%m-%dT%H:%M:%SZ'

ps -axo pid,ppid,uid,lstart,etime,command

# lsof -n -P

netstat -anv

arp -an

who -a; last -50

launchctl list

mount

df -h

ifconfig -a; netstat -nr; scutil --dns

security list-keychains

ls -la /Volumes/

diskutil list

system_profiler SPUSBDataType SPBluetoothDataType SPAirPortDataType
```

## † PROCESS BEHAVIOR TRACING

BASH

```
# fs_usage -w -f filesys                # file activity, all procs

# fs_usage -w -f network <pid>

# fs_usage -w -f pathname <pid>

# fs_usage -w <pid> | grep -v CACHE_HIT

# opensnoop -n <procname>               # DTrace-based, SIP-restricted

# execsnoop

# dtruss -p <pid>                       # [SIP]

# dtruss -f -n <procname>               # [SIP]

# ktrace trace -S -f C4 -c <cmd>        # macOS 11+
```

**Most** DTrace-based tools (`dtruss`, `opensnoop`, `execsnoop`, `iosnoop`) are blocked by SIP.

`fs_usage` and `ktrace` still work with root. `eslogger` (Sheet 03) is the modern

**replacement** for exec tracing.

## † SIGNALS & CONTROL

BASH

```
kill -TERM <pid>

kill -9 <pid>                          #

kill -STOP <pid>                       # freeze for analysis

kill -CONT <pid>

killall -STOP <name>

# launchctl kill SIGSTOP system/<label>

# launchctl bootout system/<label>        #

 unload service

Freezing (`SIGSTOP`) a suspicious process before collecting its memory/handles is often

better than killing it

 you keep the sockets and mapped files intact.
```

## † CPU / RESOURCE ANOMALIES

BASH

```
top -o cpu -l 2 -n 10 | tail -20

ps -axo pid,%cpu,%mem,etime,command | sort -k2 -rn | head -20

powermetrics --samplers cpu_power -n 1 -i 1000   # root

nettop -P -L 1                         # per-process network, one sample

iotop 2>/dev/null                      # [SIP]

fs_usage -w -f filesys 2>/dev/null | head -100   # root
```

## † QUICK ANOMALY ÆURISTICS

| SIGNAL | COMMAND | | |
|---|---|---|---|
| Process from `/tmp`, `/var/folders`, `~/Library/Application Support` | `ps -axo command= \ | grep -E '^/(tmp\|private/var/folders)'` | | |
| No code signature | `codesign -dv <p> 2>&1 \ | grep -q "not signed" ` | | |
| Ad-hoc signature | `codesign -dv <p> 2>&1 \ | grep adhoc` | | |
| Deleted binary still running | `lsof -p <pid> \ | grep deleted` | | |
| Parent is not launchd/expected | `ps -o ppid= -p <pid>` | | | |
| High entropy name (`xkjq3f`) | eyeball `ps auxwww` | | | |
| Masquerading as Apple binary but wrong path | compare `ps` path against `/System/...` | | | |
| Listening on high port owned by user proc | `lsof -i -n -P \ | grep LISTEN` | | |

BOOK XIX

# TCC (TRANSPARENCY, CONSENT & CONTROL) & PRIVACY

*canticle of the 7th sheet*

✦ ⁂ ✦

TCC is macOS's per-app permission system. It's also a favorite target a malicious grant to Accessibility or Full Disk Access is effectively game over for the user session.

## † DATABASE LOCATIONS

| SCOPE | PATH |
|---|---|
| User | `~/Library/Application Support/com.apple.TCC/TCC.db` |
| System | `/Library/Application Support/com.apple.TCC/TCC.db` |
| MDM-installed | `/Library/Application Support/com.apple.TCC/MDMOverrides.plist` |

Both DBs are SIP-protected reading them requires Full Disk Access for your terminal, or root plus FDA. Grant Terminal/iTerm FDA in System Settings Privacy & Security.

## † READING TCC

BASH

```
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db '.schema access'

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

  "SELECT service, client, client_type, auth_value, auth_reason,

          datetime(last_modified,'unixepoch') FROM access ORDER BY last_modified DESC;"

# sqlite3 /Library/Application\ Support/com.apple.TCC/TCC.db \

#   "SELECT service, client, auth_value, datetime(last_modified,'unixepoch') FROM
access;"
```

## ▸ COLUMN MEANINGS

- `service` `kTCCService*` constant (see table below) - `client` bundle ID, or absolute path if `client_type=1` - `client_type` `0` = bundle ID, `1` = absolute path - `auth_value` `0` denied, `1` unknown, `2` allowed, `3` limited - `auth_reason` `1` error, `2` user consent, `3` user set, `4` system set, `5` service policy, `6` MDM policy, `7` override policy, `8` missing usage string, `9` prompt timeout, `10` preflight unknown, `11` entitled, `12` app type policy - `csreq` binary code requirement blob pinning the signature `auth_reason = 3` (user set) or `4` (system set) on something like `kTCCServiceAccessibility` for a non-obvious app is a hunt lead. `auth_reason = 6` means it came from an MDM profile check whether that profile is legitimate (Sheet 10).

# † THE SERVICES

| CONSTANT | WHAT IT GRANTS |
|---|---|
| `kTCCServiceSystemPolicyAllFiles` | **Full Disk Access** |
| `kTCCServiceAccessibility` | **UI scripting, synthetic events, keylogging** |
| `kTCCServiceScreenCapture` | Screen recording |
| `kTCCServicePostEvent` | Synthetic keyboard/mouse events |
| `kTCCServiceListenEvent` | Input monitoring (keylogger) |
| `kTCCServiceDeveloperTool` | Bypasses Gatekeeper for spawned procs |
| `kTCCServiceSystemPolicyDesktopFolder` | ~/Desktop |
| `kTCCServiceSystemPolicyDocumentsFolder` | ~/Documents |
| `kTCCServiceSystemPolicyDownloadsFolder` | ~/Downloads |
| `kTCCServiceSystemPolicyNetworkVolumes` | Network shares |
| `kTCCServiceSystemPolicyRemovableVolumes` | USB drives |
| `kTCCServiceSystemPolicySysAdminFiles` | Admin-owned files |
| `kTCCServiceCamera` | Camera |
| `kTCCServiceMicrophone` | Microphone |
| `kTCCServiceAddressBook` | Contacts |
| `kTCCServiceCalendar` | Calendar |
| `kTCCServiceReminders` | Reminders |
| `kTCCServicePhotos` | Photos library |
| `kTCCServiceMediaLibrary` | Apple Music/TV library |
| `kTCCServiceAppleEvents` | Send AppleEvents to other apps |
| `kTCCServiceLocation` | Location |
| `kTCCServiceUbiquity` | iCloud |
| `kTCCServiceFileProviderDomain` | File provider extensions |
| `kTCCServiceEndpointSecurityClient` | Load ES client |
| `kTCCServiceRemoteDesktop` | Remote management |
| `kTCCServiceBluetoothAlways` | Bluetooth |
| `kTCCServiceMotion` | Motion sensors |

| CONSTANT | WHAT IT GRANTS |
| --- | --- |
| `kTCCServiceSpeechRecognition` | Speech |
| `kTCCServiceUserTracking` | ATT tracking |

# † Hunt Queries

BASH

```
# The dangerous four
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service, client, auth_value, auth_reason, datetime(last_modified,'unixepoch')
   FROM access
   WHERE service IN ('kTCCServiceAccessibility','kTCCServiceSystemPolicyAllFiles',
                     'kTCCServiceScreenCapture','kTCCServiceListenEvent')
     AND auth_value = 2;"
# Anything granted by path rather than bundle ID (client_type=1)
 unusual
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service, client FROM access WHERE client_type = 1;"
# Recently changed grants
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service, client, auth_value, datetime(last_modified,'unixepoch')
   FROM access WHERE last_modified > strftime('%s','now','-14 days')
   ORDER BY last_modified DESC;"
# MDM-forced grants
sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \
  "SELECT service, client FROM access WHERE auth_reason = 6;"
plutil -p /Library/Application\ Support/com.apple.TCC/MDMOverrides.plist 2>/dev/null
# Both DBs at once, tabular
for db in ~/Library/Application\ Support/com.apple.TCC/TCC.db \
          /Library/Application\ Support/com.apple.TCC/TCC.db; do
    echo "═══ $db"
    sqlite3 -header -column "$db" \
      "SELECT service, client, auth_value, auth_reason FROM access WHERE auth_value=2;"
2>/dev/null
```

```
done
```

## † LIVE TCC MONITORING

BASH

```
# log stream --predicate 'subsystem == "com.apple.TCC"' --info

# log show --last 7d --predicate 'subsystem == "com.apple.TCC"' --info --style compact

# log show --last 7d --predicate 'subsystem == "com.apple.TCC" AND eventMessage CONTAINS
"AUTHREQ"' --info

# eslogger tcc_modify | jq .              # macOS 14+

The `AUTHREQ` lines show every permission request including the responsible process

 this

survives even when the DB row is later deleted.
```

## † RESETTING GRANTS

BASH

```
tccutil reset All

tccutil reset All <bundle-id>

tccutil reset Accessibility

tccutil reset Accessibility <bundle-id>

tccutil reset ScreenCapture <bundle-id>

tccutil reset SystemPolicyAllFiles <bundle-id>

tccutil reset Camera; tccutil reset Microphone

# tccutil reset All                       #

 nukes every grant, expect user pain

Service names for `tccutil` drop the `kTCCService` prefix.
```

## † RESPONSIBLE PROCESS / ATTRIBUTION

TCC attributes a request to the "responsible" process, not necessarily the caller. A script run from Terminal inherits Terminal's grants. This is the standard TCC bypass shape:

```bash
# What Terminal itself holds

sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db \

  "SELECT service,auth_value FROM access WHERE client LIKE '%Terminal%' OR client LIKE '%iterm%';"

# Check a running process's responsible pid

# launchctl procinfo <pid> | grep -i responsible

Hunt implication: if Terminal has Full Disk Access (common for admins

 including you),

any script executed from it reads TCC-protected data with no prompt. Worth deciding

consciously whether that's the posture you want on your daily driver.
```

## † ENTITLEMENT-BASED BYPASSES

```bash
codesign -d --entitlements - <app> 2>/dev/null | plutil -p - | grep -i tcc

# com.apple.private.tcc.allow

 silent grant

# com.apple.private.tcc.allow.overridable

# com.apple.private.tcc.manager

 can write TCC.db

Only Apple-signed binaries should hold these. A third-party binary with

`com.apple.private.tcc.*` is either fraudulent or the signature is invalid

 verify with

`codesign --verify --deep --strict`.
```

## † CAMERA / MIC IN-USE DETECTION

BASH

```
log show --last 1d --predicate 'subsystem == "com.apple.cmio"' --info | grep -i 'start\|
stop'

log show --last 1d --predicate 'eventMessage CONTAINS "Camera" AND eventMessage CONTAINS
"Start"' --info

log stream --predicate 'subsystem == "com.apple.coreaudio"' --info | grep -i 'input\|
record'

# lsof | grep -iE 'AppleCamera|VDC|CoreMedia' 2>/dev/null

ioreg -l | grep -i camera | head -20

On Apple silicon the camera LED is hardware-interlocked, so a covert-camera claim without

the LED is implausible on modern hardware. Mic has no such interlock

 the orange dot is

software.
```

## † OTHER PRIVACY ARTIFACTS

```bash
# Location services

defaults read /var/db/locationd/Library/Preferences/ByHost/com.apple.locationd.* 2>/dev/null

sudo defaults read /var/db/locationd/clients.plist 2>/dev/null | head -60

# Analytics submissions

defaults read /Library/Application\ Support/CrashReporter/DiagnosticMessagesHistory.plist 2>/dev/null

ls /Library/Logs/DiagnosticReports/

# Siri / dictation

defaults read com.apple.assistant.support 2>/dev/null

ls ~/Library/Assistant/

# Screen time / usage (knowledgeC)

sqlite3 ~/Library/Application\ Support/Knowledge/knowledgeC.db \

  "SELECT ZOBJECT.ZSTREAMNAME, ZOBJECT.ZVALUESTRING,

          datetime(ZOBJECT.ZSTARTDATE+978307200,'unixepoch') AS start

   FROM ZOBJECT WHERE ZSTREAMNAME='/app/inFocus' ORDER BY start DESC LIMIT 50;"

`knowledgeC.db` is a rich application-usage timeline

 app focus, device lock/unlock,

Bluetooth pairing, battery. Requires FDA. It's one of the best "what did the user actually

do" sources on macOS.
```

# Every stream available

```bash
sqlite3 ~/Library/Application\ Support/Knowledge/knowledgeC.db \

  "SELECT DISTINCT ZSTREAMNAME FROM ZOBJECT;"
```

Useful streams: `/app/inFocus`, `/app/usage`, `/app/install`, `/display/isBacklit`, `/device/isLocked`, `/device/isPluggedIn`, `/bluetooth/isConnected`, `/safari/history`, `/user/isBacklit`, `/notification/usage`.

BOOK XX

# PACKAGES, SOFTWARE INVENTORY, SUPPLY CHAIN

*canticle of the 14th sheet*

## † INSTALLED APPLICATIONS

BASH

```
ls -la /Applications/ ~/Applications/ /System/Applications/

system_profiler SPApplicationsDataType

system_profiler SPApplicationsDataType -json | jq -r '.SPApplicationsDataType[] | [.
\_name, .version // "", .obtained_from // "", .signed_by[0]? // ""] | @tsv'

mdfind 'kMDItemKind == "Application"'

mdfind "kMDItemContentType == 'com.apple.application-bundle'" | grep -v '^/System'

lsappinfo list

/System/Library/Frameworks/CoreServices.framework/Frameworks/LaunchServices.framework/
Support/lsregister -dump | grep -E '^\s*path:' | sort -u

`system_profiler SPApplicationsDataType` includes `obtained_from` (App Store / identified

developer / unknown) and the signing authority

 the fastest inventory with trust context.
```

# Everything NOT from the App Store or an identified developer

BASH

```
system_profiler SPApplicationsDataType -json | \

    jq -r '.SPApplicationsDataType[] | select(.obtained_from ≠ "mac_app_store"
and .obtained_from ≠ "identified_developer") | [.\_name, .path] | @tsv'
```

## † App Bundle Inspection

BASH

```bash
defaults read /Applications/Foo.app/Contents/Info.plist

plutil -p /Applications/Foo.app/Contents/Info.plist

/usr/libexec/PlistBuddy -c 'Print :CFBundleIdentifier' /Applications/Foo.app/Contents/
Info.plist

ls -la /Applications/Foo.app/Contents/

ls -la /Applications/Foo.app/Contents/{MacOS,Library,Frameworks,XPCServices,Resources}
2>/dev/null

find /Applications/Foo.app -name '*.dylib' -o -name '*.framework' -o -name '*.xpc'

codesign -dv --verbose=4 /Applications/Foo.app

codesign -d --entitlements - /Applications/Foo.app | plutil -p -

du -sh /Applications/Foo.app

stat -f '%SB %Sm %N' -t '%FT%T' /Applications/Foo.app

xattr -l /Applications/Foo.app

Look for `Contents/Library/LaunchDaemons/` and `Contents/Library/LoginItems/`

 that's

SMAppService persistence bundled into the app.
```

BASH

```bash
for a in /Applications/*.app; do

    for d in "$a/Contents/Library/LaunchDaemons" "$a/Contents/Library/LaunchAgents" "$a/
Contents/Library/LoginItems"; do

      [[ -d $d ]] && { echo "=== $d"; ls -la "$d"; }

    done

  done
```

## † INSTALLER PACKAGES (PKGUTIL)

BASH

```
pkgutil --pkgs

pkgutil --pkgs | grep -v '^com\.apple'

pkgutil --pkg-info <pkg-id>

pkgutil --files <pkg-id>

pkgutil --file-info /path/to/file        # which pkg owns this file?

pkgutil --pkgs-plist

# pkgutil --forget <pkg-id>                # remove receipt only

ls -la /var/db/receipts/

ls -lat /var/db/receipts/*.plist | head -30

plutil -p /var/db/receipts/<pkg-id>.plist

Receipts in `/var/db/receipts/` are timestamped

 a great install timeline.
```

BASH

```
for p in /var/db/receipts/*.plist; do

    printf '%s\t%s\n' "$(stat -f '%Sm' -t '%FT%T' "$p")" "${p:t:r}"

  done | sort | tail -40
```

## † INSPECTING A .PKG BEFORE INSTALL

BASH

```
pkgutil --expand-full suspicious.pkg /tmp/expanded

pkgutil --expand suspicious.pkg /tmp/expanded

ls -R /tmp/expanded

cat /tmp/expanded/*/Scripts/{preinstall,postinstall} 2>/dev/null

cat /tmp/expanded/Distribution

cat /tmp/expanded/*/PackageInfo

pkgutil --check-signature suspicious.pkg

spctl -a -vvv -t install suspicious.pkg

xar -tf suspicious.pkg

xar -xf suspicious.pkg -C /tmp/xar

`preinstall`/`postinstall` scripts run as root. This is the single most important thing
to

read before installing anything from outside the App Store

 and the most common

trojanization point.
```

# One-liner: dump all scripts from a pkg

BASH

```
pkgutil --expand-full x.pkg /tmp/x && find /tmp/x -path '*Scripts*' -type f -exec sh -c
'echo "=== $1"; cat "$1"' _ {} \;
```

## † DMG INSPECTION

BASH

```
hdiutil imageinfo suspicious.dmg

hdiutil verify suspicious.dmg

hdiutil attach -readonly -nobrowse -noverify -mountpoint /tmp/dmg suspicious.dmg

ls -laR /tmp/dmg

hdiutil detach /tmp/dmg

codesign -dv --verbose=4 /tmp/dmg/Foo.app

xattr -l suspicious.dmg

A DMG with a background image saying "drag to Terminal" or containing a `.command`/`.sh`

rather than an `.app` is the current stealer delivery pattern.
```

## † INSTALL HISTORY

BASH

```
system_profiler SPInstallHistoryDataType

system_profiler SPInstallHistoryDataType -json | jq -r '.SPInstallHistoryDataType[] |
[.install_date, .\_name, .install_version, .package_source] | @tsv' | sort

plutil -p /Library/Receipts/InstallHistory.plist

# grep -iE 'Installed|Upgraded|PackageKit' /var/log/install.log | tail -60

# grep -i 'Install Assistant' /var/log/install.log | tail -20

ls -la /var/log/install.log*

`/var/log/install.log` is one of the few classic text logs still populated and rotated on

macOS. Read it during every triage.
```

## † Homebrew Audit

BASH

```bash
brew --version

brew config

brew list

brew list --formula

brew list --cask

brew leaves                                # top-level installs

brew deps --tree --installed

brew outdated

brew tap                                   # third-party taps = supply chain risk

brew info <formula>

brew --prefix

ls -la $(brew --prefix)/bin | head -40

brew doctor

brew audit --strict <formula>

brew fetch --retry <formula> && shasum -a 256 $(brew --cache <formula>)
```

## HOMEBREW RISK POINTS

BASH

```
# Non-core taps

brew tap | grep -v '^homebrew/'

# Formulae installed from a non-core tap

brew list --formula --full-name | grep '/'

# Writable prefix (a classic local privesc on multi-user boxes)

ls -ld $(brew --prefix)

find $(brew --prefix)/bin -perm -o+w 2>/dev/null

# Is brew's prefix ahead of /usr/bin in PATH?

echo $PATH | tr ':' '\n' | head -5

On Apple silicon, brew's prefix is `/opt/homebrew` (owned by the installing user). On

Intel it's `/usr/local`

 which is *not* SIP-protected and is writable by admin. Anything

dropped in `/usr/local/bin` shadows system binaries for users with it early in PATH.
```

## † OTHER PACKAGE MANAGERS

BASH

```bash
# MacPorts

port installed

port outdated

# npm

npm ls -g --depth=0

npm audit

cat ~/.npmrc

# pip

pip3 list

pip3 list --outdated

pip3 audit 2>/dev/null || pip-audit

# gem

gem list

gem outdated

# cargo

cargo install --list

# go

ls $(go env GOPATH)/bin 2>/dev/null

# Mac App Store

mas list                                # brew install mas

mas outdated
```

## † SOFTWARE UPDATE STATE

BASH

```bash
softwareupdate --list

softwareupdate --history

softwareupdate --list-full-installers

defaults read /Library/Preferences/com.apple.SoftwareUpdate

sw_vers

system_profiler SPSoftwareDataType

# softwareupdate -ia --restart             #

 install all + reboot

log show --last 30d --predicate 'process == "softwareupdated"' --info | head -40
```

## † FULL SOFTWARE INVENTORY SCRIPT

BASH

```
#!/bin/zsh

OUT=~/software-inventory-$(date +%F).tsv

printf 'type\tname\tversion\tpath\tsigner\tteamid\n'

for a in /Applications/*.app /Applications/*/*.app ~/Applications/*.app; do

  [[ -d $a ]] || continue

  v=$(/usr/libexec/PlistBuddy -c 'Print :CFBundleShortVersionString' "$a/Contents/
Info.plist" 2>/dev/null)

  s=$(codesign -dv "$a" 2>&1 | awk -F= '/^Authority/{print $2; exit}')

  t=$(codesign -dv "$a" 2>&1 | awk -F= '/TeamIdentifier/{print $2}')

  printf 'app\t%s\t%s\t%s\t%s\n' "${a:t:r}" "$v" "$a" "${s:-UNSIGNED}" "${t:-none}"

done

pkgutil --pkgs | while read p; do

  v=$(pkgutil --pkg-info "$p" | awk '/^version:/{print $2}')

  printf 'pkg\t%s\t%s\t\t\t\n' "$p" "$v"

done

brew list --versions 2>/dev/null | while read n v; do

  printf 'brew\t%s\t%s\t\t\t\n' "$n" "$v"

done

} > "$OUT"

echo "Wrote $OUT ($(wc -l < "$OUT") rows)"
```

## † REMOVAL

BASH

```
# Proper app removal

# rm -rf /Applications/Foo.app

ls ~/Library/{Application\ Support,Preferences,Caches,Containers,Logs}/ | grep -i foo

find ~/Library -iname '*foo*' -maxdepth 3 2>/dev/null

# pkgutil --forget com.vendor.foo

# launchctl bootout gui/$(id -u)/com.vendor.foo 2>/dev/null

# rm -f ~/Library/LaunchAgents/com.vendor.foo.plist
```

## † SUPPLY CHAIN CHECKS

BASH

```
# Verify a download against a published hash

shasum -a 256 downloaded.dmg

shasum -a 256 -c published.sha256

# Verify a signature

gpg --verify file.sig file

codesign -dv --verbose=4 downloaded.app

spctl -a -vvv -t install downloaded.pkg

stapler validate downloaded.dmg

# Check the download origin

xattr -p com.apple.metadata:kMDItemWhereFroms downloaded.dmg | xxd -r -p | plutil -p -

# Look it up

shasum -a 256 file | awk '{print $1}' | xargs -I{} echo "https://www.virustotal.com/gui/file/{}"
```

BOOK XXI

# CODE SIGNING, GATEKEEPER, NOTARIZATION, XPROTECT

*canticle of the 6th sheet*

✦ ⁂ ✦

## † CODESIGN

BASH

```
codesign -dv <target>                          # basic info
codesign -dv --verbose=4 <target>              # full: authority chain, TeamID, hashes
codesign -dvvv --deep <target>
codesign --verify --verbose=2 <target>
codesign --verify --deep --strict --verbose=4 <target>
codesign -d --entitlements - <target> | plutil -p -
codesign -d --entitlements :- <target>          # raw plist
codesign -d -r- <target>                        # designated requirement
codesign -d --extract-certificates=/tmp/cert <target>
codesign --display --file-list - <target>       # sealed files
codesign -d --arch arm64 <target>               # per-slice on fat binaries
```

### ▸ KEY FIELDS IN `--VERBOSE=4`

| FIELD | MEANING |
|---|---|
| `Identifier` | bundle ID or derived name |
| `TeamIdentifier` | Apple Developer team; `not set` for Apple/ad-hoc |
| `Authority` | cert chain, innermost first |
| `Signature=adhoc` | self-signed, no CA |

suspicious for a "vendor" app |

| `CDHash` | CODE DIRECTORY HASH |
|---|---|

the identity used by TCC/allowlists |

| `flags` | `runtime` = HARDENED RUNTIME ENABLED; `library-validation` |
|---|---|
| `Sealed Resources` | resource envelope version |
| `Timestamp` | secure timestamp; absent on ad-hoc |

## ▸ RED FLAGS

BASH

```bash
# Unsigned
codesign -dv <t> 2>&1 | grep -q 'not signed' && echo UNSIGNED

# Ad-hoc
codesign -dv <t> 2>&1 | grep -q 'Signature=adhoc' && echo ADHOC

# No hardened runtime
codesign -dv <t> 2>&1 | grep -q 'flags=.*runtime' || echo 'NO HARDENED RUNTIME'

# Revoked / broken seal
codesign --verify --deep --strict <t> 2>&1

# Team ID
codesign -dv <t> 2>&1 | awk -F= '/TeamIdentifier/{print $2}'
```

## BULK VERIFICATION

BASH

```
for a in /Applications/*.app; do
    printf '%-45s ' "${a:t}"
    codesign --verify --deep --strict "$a" 2>&1 | head -1 || echo OK
  done
for a in /Applications/*.app; do
    printf '%-45s %-12s %s\n' "${a:t}" \
      "$(codesign -dv "$a" 2>&1 | awk -F= '/TeamIdentifier/{print $2}')" \
      "$(codesign -dv "$a" 2>&1 | awk -F= '/^Authority/{print $2; exit}')"
  done
```

## † GATEKEEPER / SPCTL

BASH

```
spctl --status

# spctl --master-enable                          #
 (deprecated wording, still works)

# spctl --master-disable                         #
 don't, on prod

spctl -a -vvv <target>                           # assess as executable

spctl -a -t exec -vvv <target>

spctl -a -t install -vvv <pkg>

spctl -a -t open --context context:primary-signature -vvv <file>

spctl --list                                     # rules DB

spctl --list --type execute

# spctl --add --label "MyRule" <target>

# spctl --remove --label "MyRule"

# spctl --enable --label "MyRule"

# spctl --disable --label "MyRule"

sudo sqlite3 /var/db/SystemPolicy 'SELECT * FROM authority LIMIT 20;'

Note: since macOS 15, `spctl --master-disable` no longer fully works from CLI
 the
"Anywhere" option must be re-enabled in System Settings after an attempted open.
```

## † NOTARIZATION

BASH

```
spctl -a -vvv -t install <pkg> 2>&1 | grep -i notariz

stapler validate <app-or-pkg>                    # ticket stapled?

xcrun stapler validate -v <target>

codesign -dv --verbose=4 <t> 2>&1 | grep -i timestamp

log show --last 1d --predicate 'subsystem == "com.apple.syspolicy"' --info | grep -i
notar
```

## † QUARANTINE (COM.APPLE.QUARANTINE)

BASH

```
xattr -l <file>

xattr -p com.apple.quarantine <file>

xattr -r -l <dir> | grep -A1 quarantine

# xattr -d com.apple.quarantine <file>            # remove

 do knowingly

# xattr -c <file>                                  # clear ALL xattrs

# xattr -r -d com.apple.quarantine <dir>

find ~/Downloads -exec xattr -p com.apple.quarantine {} \; -print 2>/dev/null
```

### ▸ QUARANTINE VALUE FORMAT

`<flags>;<timestamp-hex>;<agent-name>;<uuid>`

BASH

```
xattr -p com.apple.quarantine <f> | awk -F';' '{print strftime("%F %T",
strtoul("0x"$2))}'

Flags: `0000` = not yet checked, `0001`/`0002` = checked/approved, `0083` = downloaded by
browser.
```

## ▸ QUARANTINEEVENTSV2 DATABASE

BASH

```
sqlite3 ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 \

  "SELECT datetime(LSQuarantineTimeStamp+978307200,'unixepoch'),

          LSQuarantineAgentName, LSQuarantineDataURLString, LSQuarantineOriginURLString

   FROM LSQuarantineEvent ORDER BY LSQuarantineTimeStamp DESC LIMIT 50;"

This is one of the highest-value forensic artifacts on macOS

 it records the download

URL, referrer, and agent for every quarantined file.
```

BASH

```
sqlite3 -header -csv ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2 \

  "SELECT * FROM LSQuarantineEvent;" > ~/quarantine-events.csv
```

## † XPROTECT

BASH

```
ls -la /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/Resources/

plutil -p /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/Resources/
XProtect.meta.plist | head -60

plutil -p /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/Resources/
XProtect.plist | grep -E 'Description|Identity' | head -80

defaults read /Library/Apple/System/Library/CoreServices/XProtect.bundle/Contents/
version.plist CFBundleShortVersionString

system_profiler SPInstallHistoryDataType | grep -i -A3 xprotect
```

### ▸ XPROTECT REMEDIATOR (THE BEHAVIORAL SCANNER, MACOS 13+)

BASH

```
ls /Library/Apple/System/Library/CoreServices/XProtect.app/Contents/MacOS/

# log show --last 30d --predicate 'subsystem == "com.apple.XProtectFramework.PluginAPI"'
--info

# log show --last 30d --predicate 'eventMessage CONTAINS "XProtectRemediator"' --info

# eslogger xp_malware_detected xp_malware_remediated | jq .

log show --last 30d --predicate 'eventMessage CONTAINS "Scan complete"' --info | grep -i
xprotect

Malware family names in Remediator scanner binaries (Adload, Pirrit, Genieo, DubRobber,

BadGacha, etc.) tell you what Apple currently detects behaviorally.
```

### ▸ MRT (LEGACY, FOLDED INTO XPROTECT REMEDIATOR)

BASH

```
ls /Library/Apple/System/Library/CoreServices/MRT.app 2>/dev/null

defaults read /Library/Apple/System/Library/CoreServices/MRT.app/Contents/version.plist
2>/dev/null
```

## † SYSTEM POLICY DATABASES

BASH

```
# sqlite3 /var/db/SystemPolicy 'SELECT id,type,requirement,allow,disabled,label FROM
authority WHERE disabled=0 LIMIT 30;'

# sqlite3 /var/db/SystemPolicyConfiguration/KextPolicy 'SELECT * FROM kext_policy;'

# sqlite3 /var/db/SystemPolicyConfiguration/ExecPolicy 'SELECT * FROM policy_scan_cache
LIMIT 20;'

# sqlite3 /var/db/SystemPolicyConfiguration/ExecPolicy 'SELECT
datetime(scan_timestamp+978307200,"unixepoch"), file_identifier,
executable_measurements_v2.* FROM executable_measurements_v2 LIMIT 20;' 2>/dev/null

`ExecPolicy` is a first-launch record of binaries

 an execution artifact even for

binaries later deleted.
```

## † Hardened runtime & library validation

```bash
codesign -d --entitlements - <t> 2>/dev/null | plutil -p - | grep -E 'disable-library-validation|allow-dyld-environment|allow-unsigned-executable|debugger|jit'

Dangerous entitlements to flag:

| Entitlement | Risk |

|---|---|

| `com.apple.security.cs.disable-library-validation` | allows loading unsigned dylibs |

| `com.apple.security.cs.allow-dyld-environment-variables` | `DYLD_INSERT_LIBRARIES` works |

| `com.apple.security.cs.allow-unsigned-executable-memory` | shellcode-friendly |

| `com.apple.security.cs.allow-jit` | RWX pages |

| `com.apple.security.get-task-allow` | debuggable

 shouldn't ship |

| `com.apple.security.cs.debugger` | can attach to others |

| `com.apple.private.tcc.allow` | bypasses TCC prompts |
```

# Hunt for them across /Applications

```bash
for a in /Applications/*.app; do

    e=$(codesign -d --entitlements - "$a" 2>/dev/null | plutil -p - 2>/dev/null)

    echo "$e" | grep -q 'disable-library-validation\|allow-dyld-environment\|get-task-allow' && echo "!! ${a:t}"

  done
```

## † SSV / SEALED SYSTEM VOLUME

```bash
csrutil authenticated-root status

diskutil apfs list | grep -i -A3 'Sealed'

# diskutil apfs list | grep -E 'Snapshot|Sealed|Roles'

mount | grep -E 'read-only|sealed'

If the seal is broken on a machine you didn't modify, that's a serious finding.
```

## † CERTIFICATE & TRUST STORE

```bash
security list-keychains

security find-certificate -a -p /System/Library/Keychains/
SystemRootCertificates.keychain | grep -c 'BEGIN CERT'

security dump-trust-settings -d                 # admin/system trust overrides

security dump-trust-settings                    # user

security find-certificate -a -p ~/Library/Keychains/login.keychain-db > /tmp/user-
certs.pem

openssl x509 -in /tmp/cert.pem -noout -text

security verify-cert -c /tmp/cert.pem

A user-added or admin-added root CA in the trust store is a classic MITM/persistence

artifact. `dump-trust-settings -d` returning non-empty on a machine you manage warrants a

look.
```

# List non-Apple roots in the System keychain

```bash
security find-certificate -a /Library/Keychains/System.keychain -p | \

    while openssl x509 -noout -subject 2>/dev/null; do :; done
```

BOOK XXII

# MACOS LIVE TRIAGE

*canticle of the 1st sheet*

✦ ⁂ ✦

Fast sweep of a suspect host. Run top-to-bottom; redirect to a file on external media.

## † SESSION SETUP

BASH

```bash
TRIAGE=~/triage-$(hostname -s)-$(date +%Y%m%dT%H%M%SZ); mkdir -p "$TRIAGE"; cd "$TRIAGE"

exec > >(tee -a triage.log) 2>&1     # capture everything

script -q -a session.typescript      # alternative: full tty capture
```

## † HOST IDENTITY

BASH

```bash
sw_vers                              # product name, version, build

system_profiler SPHardwareDataType   # model, serial, UUID, chip

sysctl -n machdep.cpu.brand_string hw.memsize kern.boottime

uname -a

hostname; scutil --get ComputerName; scutil --get LocalHostName

ioreg -rd1 -c IOPlatformExpertDevice | awk -F'"' '/IOPlatformUUID/{print $4}'

nvram -p                             # boot-args, recovery flags
```

## † SECURITY POSTURE SNAPSHOT

BASH

```bash
csrutil status                          # SIP

spctl --status                          # Gatekeeper

fdesetup status                         # FileVault

softwareupdate --history                # patch history

# /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate

system_profiler SPConfigurationProfileDataType   # installed profiles

profiles status -type enrollment                 # MDM enrollment

csrutil authenticated-root status                # SSV / sealed volume

bputil -
d                           # Apple silicon: boot policy (needs recovery on some vers)
```

## † USERS & SESSIONS

BASH

```bash
who -a

last -20

dscl . list /Users | grep -v '^_'

dscl . -read /Users/<user> NFSHomeDirectory UniqueID PrimaryGroupID

dscl . -read /Groups/admin GroupMembership

stat -f '%Sc %N' /Users/*               # account creation-ish

ls -la /Users/                          # unexpected home dirs
```

## † PROCESSES

BASH

```bash
ps auxwww

ps -axo pid,ppid,uid,user,lstart,etime,%cpu,%mem,command

pstree 2>/dev/null || ps -axo pid,ppid,command   # pstree via brew

lsappinfo list                        # GUI app registry

launchctl list                        # loaded services (user domain)

# launchctl print system | head -100
```

## † NETWORK

BASH

```bash
netstat -anv -p tcp

netstat -anv -p udp

# lsof -i -n -P

lsof -i -n -P | awk 'NR==1 || /LISTEN|ESTABLISHED/'

arp -an

netstat -nr

scutil --dns

ifconfig -a
```

## † PERSISTENCE QUICK PASS

BASH

```
ls -la /Library/Launch{Agents,Daemons} ~/Library/LaunchAgents /System/Library/
Launch{Agents,Daemons}

ls -la /Library/StartupItems /Library/ScriptingAdditions

osascript -e 'tell application "System Events" to get the name of every login item'

crontab -l; sudo crontab -l

ls -la /etc/periodic/*/ /etc/cron.d 2>/dev/null

ls -la ~/Library/Application\ Support/com.apple.backgroundtaskmanagementagent/ 2>/dev/
null

See [02-persistence-hunting.md](02-persistence-hunting.md) for the full surface.
```

## † RECENT FILE ACTIVITY

BASH

```
find /Users -type f -mtime -7 -not -path '*/Library/Caches/*' 2>/dev/null | head -200

find /tmp /var/tmp /private/var/folders -type f -mtime -3 2>/dev/null | head -100

ls -latr ~/Downloads ~/Desktop /tmp

mdfind 'kMDItemFSCreationDate >= $time.today(-7)' 2>/dev/null | head -100

xattr -rl ~/Downloads 2>/dev/null | grep -A2 quarantine | head -50
```

## † SHELL & ENVIRONMENT

BASH

```bash
for f in ~/.zshrc ~/.zshenv ~/.zprofile ~/.bash_profile ~/.bashrc ~/.profile /etc/zshrc /etc/profile /etc/zshenv; do echo "=== $f"; cat "$f" 2>/dev/null; done

echo $PATH | tr ':' '\n'

env

cat ~/.ssh/authorized_keys ~/.ssh/config 2>/dev/null

ls -la ~/.ssh/

cat ~/.zsh_history | tail -200

defaults read com.apple.dock persistent-apps 2>/dev/null | grep -i _CFURLString
```

## † LOG PULL

BASH

```bash
# log collect --last 24h --output "$TRIAGE/system.logarchive"

log show --last 1h --predicate 'eventMessage CONTAINS "error"' --style compact | head -200

# sudo sysdiagnose -f "$TRIAGE" -u    # heavy; ~300MB, no UI prompt with -u
```

## † HASHING WHAT YOU FOUND

BASH

```bash
shasum -a 256 <file>

find /Library/Launch* /Users/*/Library/LaunchAgents -type f -exec shasum -a 256 {} \; 2>/dev/null

codesign -dv --verbose=4 <app-or-binary> 2>&1
```

## † PACKAGE THE TRIAGE

BASH

```
cd ~ && tar -czf "$(basename "$TRIAGE").tar.gz" "$(basename "$TRIAGE")"

shasum -a 256 "$(basename "$TRIAGE").tar.gz" | tee "$(basename "$TRIAGE").tar.gz.sha256"
```

## † TRIAGE DECISION TREE

| FINDING | NEXT STEP |
|---|---|
| Unsigned binary in LaunchDaemons | Sheet 06 + 18, hash |

VT, preserve plist |

| OUTBOUND TO UNKNOWN ASN | SHEET 05, PCAP + `nettop`, CHECK PROCESS OWNER |
|---|---|
| TCC grants to unexpected app | Sheet 07, dump TCC.db, check profile-installed grants |
| Recent profile install | Sheet 10, `profiles -P`, check payload UUID |
| Modified shell rc | Sheet 08, FSEvents + xattr, check `mtime` vs `birthtime` |
| New admin account | Sheet 12, `log show` for `opendirectoryd` writes |

BOOK XXIII

# Filesystem forensics (APFS, xattrs, timestamps, FSEvents)

*canticle of the 8th sheet*

✦ ⁂ ✦

## † VOLUME LAYOUT

BASH

```
diskutil list

diskutil list internal

diskutil info /

diskutil info disk1s1

diskutil apfs list

diskutil apfs listSnapshots /

diskutil apfs listUsers /

mount

df -h

df -i                                  # inodes

ls -la /System/Volumes/

APFS boot layout on modern macOS:

- `disk3s1s1`
 Sealed System Volume (read-only, SSV)
- `disk3s5`
 Data volume (`/System/Volumes/Data`)
- `disk3s6`
 VM (swap)
- `disk3s2`
 Preboot
- `disk3s4`
 Recovery
- Firmlinks join `/` and `/System/Volumes/Data` into one apparent tree
```

```bash
cat /usr/share/firmlinks

ls -la /System/Volumes/Data/
```

## † APFS SNAPSHOTS

```bash
tmutil listlocalsnapshots /

diskutil apfs listSnapshots /

diskutil apfs listSnapshots disk3s5

# tmutil localsnapshot                    # create one

# tmutil deletelocalsnapshots <date>      #

# mkdir /tmp/snap && sudo mount_apfs -o ro -s <snapshot-name> /System/Volumes/Data /tmp/snap

ls /tmp/snap

# umount /tmp/snap

Mounting a snapshot read-only gives you a point-in-time view of the data volume

 very

useful for "what did this file look like before the incident" without touching the live FS.
```

## † TIME MACHINE

BASH

```bash
tmutil destinationinfo

tmutil latestbackup

tmutil listbackups

tmutil status

tmutil compare -s                       # what changed since last backup

tmutil compare /path/to/backup /path/to/current

tmutil machinedirectory

tmutil calculatedrift <machine-dir>

log show --last 7d --predicate 'process == "backupd"' --info | head -50
```

## † EXTENDED ATTRIBUTES

BASH

```bash
xattr <file>

xattr -l <file>

xattr -p <attr-name> <file>

xattr -r -l <dir>

ls -la@ <file>                          # @ column shows xattrs

ls -lO <file>                           # flags

# xattr -d <attr> <file>

# xattr -w <attr> <value> <file>

# xattr -c <file>                         #
 clear all
```

## ▸ COMMON XATTRS

| ATTRIBUTE | MEANING |
|---|---|
| `com.apple.quarantine` | Downloaded / untrusted origin |
| `com.apple.metadata:kMDItemWhereFroms` | Download URL + referrer |
| `com.apple.metadata:kMDItemDownloadedDate` | When |
| `com.apple.provenance` | macOS 13+ app provenance |
| `com.apple.macl` | TCC file-level access grant |
| `com.apple.lastuseddate#PS` | Last opened |
| `com.apple.FinderInfo` | Finder flags, colors |
| `com.apple.ResourceFork` | Legacy resource fork |
| `com.apple.rootless` | SIP protection marker |
| `com.apple.cs.CodeDirectory` | Detached signature |

## ▸ READING THE INTERESTING ONES

BASH

```
xattr -p com.apple.metadata:kMDItemWhereFroms <file> | xxd -r -p | plutil -p -

xattr -p com.apple.metadata:kMDItemDownloadedDate <file> | xxd -r -p | plutil -p -

xattr -p com.apple.provenance <file> | xxd -p

xattr -p com.apple.macl <file> | xxd

# Where-froms across a whole directory

find ~/Downloads -type f -print0 | while IFS= read -r -d '' f; do

    u=$(xattr -p com.apple.metadata:kMDItemWhereFroms "$f" 2>/dev/null | xxd -r -p |
plutil -p - 2>/dev/null | grep -o 'http[^"]*' | head -1)

    [[ -n $u ]] && printf '%s\t%s\n' "$u" "$f"

  done

`com.apple.macl` is worth knowing: it's how macOS remembers "the user granted this
specific

app access to this specific file"

 it survives even when TCC.db entries don't.
```

## † TIMESTAMPS

BASH

```
stat -f 'birth=%SB acc=%Sa mod=%Sm chg=%Sc %N' -t '%Y-%m-%dT%H:%M:%S' <file>

stat -x <file>

GetFileInfo <file> 2>/dev/null           # requires Xcode CLT

mdls <file>                              # Spotlight metadata, incl. dates

mdls -name kMDItemContentCreationDate -name kMDItemFSCreationDate <file>

Four macOS timestamps: **birth** (creation, APFS), **access**, **modify** (content),

**change** (inode metadata).
```

### ▸ TIMESTOMPING DETECTION

BASH

```
# mtime older than birthtime is impossible naturally

find /Library/LaunchDaemons -type f -exec stat -f '%B %m %N' {} \; | awk '$2 < $1 {print
"SUSPECT:", $3}'

# ctime should be ⩾ mtime; if mtime was set backwards, ctime reveals it

stat -f 'mod=%Sm chg=%Sc %N' -t '%FT%T' <file>

# Compare against Spotlight's independent record

mdls -name kMDItemFSContentChangeDate -name kMDItemContentModificationDate <file>

Attackers who `touch -t` a file change mtime/atime but not ctime, and Spotlight often

retains the original. Cross-referencing these three is the standard check.
```

## † FILE FLAGS

BASH

```
ls -lO <file>

chflags nohidden <file>

# chflags uchg <file>                    # user immutable

# chflags noschg <file>                  # system immutable off (needs single-user for
some)

# chflags -R nouchg <dir>

ls -lO / | grep -E 'restricted|sunlnk|schg'

Flags: `uchg` user-immutable, `schg` system-immutable, `hidden`, `restricted` (SIP),

`sunlnk` system no-unlink, `compressed`.

`restricted` is the SIP marker

 you'll see it on `/System`, `/usr` (except `/usr/local`),

`/bin`, `/sbin`, `/var`.
```

## † SPOTLIGHT (MDFIND / MDLS)

BASH

```
mdfind <query>

mdfind -onlyin ~/Documents 'kMDItemTextContent == "*password*"c'

mdfind 'kMDItemFSCreationDate >= $time.today(-7)'

mdfind 'kMDItemContentType == "public.unix-executable"'

mdfind 'kMDItemKind == "Application"'

mdfind "kMDItemWhereFroms == '*'" | head -50

mdfind -name '<filename>'

mdls <file>

mdutil -s /                              # indexing status

# mdutil -i off /                          #

 disable indexing

# mdutil -E /                              #

 erase and rebuild index

ls -la /.Spotlight-V100/ 2>/dev/null

Spotlight's store retains metadata about deleted files for a while

 `mdfind` sometimes

surfaces names of things already removed from disk.
```

## † FSEVENTS

```bash
ls -la /.fseventsd/

ls -la /System/Volumes/Data/.fseventsd/

# ls /.fseventsd/*.gz | head

# for f in /.fseventsd/*.gz; do gunzip -c "$f"; done | strings | head -100

Parsing FSEvents properly needs a dedicated tool (`FSEventsParser` by Nicole Ibrahim is
the

standard). The raw gzipped logs record path + event-type flags and give you a directory

change history that survives file deletion.
```

# Quick and dirty path extraction

```bash
sudo sh -c 'for f in /System/Volumes/Data/.fseventsd/*; do gunzip -c "$f" 2>/dev/null;
done' | \

    strings -n 8 | grep -E '^(Users|private|Library|Applications)/' | sort -u | head -200
```

## † DELETED FILES & SLACK

```bash
ls -la ~/.Trash/

ls -la /Volumes/*/.Trashes/ 2>/dev/null

plutil -p ~/.Trash/.DS_Store 2>/dev/null

# APFS doesn't have a simple undelete; snapshots are the practical recovery route

tmutil listlocalsnapshots /
```

## † DISK IMAGING

BASH

```
# diskutil unmountDisk /dev/disk2

# dd if=/dev/rdisk2 of=/Volumes/Ext/image.dd bs=4m conv=noerror,sync status=progress

# dc3dd if=/dev/rdisk2 hof=/Volumes/Ext/image.dd hash=sha256 log=/Volumes/Ext/dc3dd.log

# ewfacquire /dev/rdisk2                  # brew install libewf

hdiutil create -srcdevice /dev/disk2 -format UDZO /Volumes/Ext/image.dmg

hdiutil attach -readonly -nomount /Volumes/Ext/image.dmg

hdiutil attach -readonly -mountpoint /tmp/m /Volumes/Ext/image.dmg

hdiutil detach /dev/disk4

hdiutil imageinfo /Volumes/Ext/image.dmg

Use `/dev/rdiskN` (raw) not `/dev/diskN`

 it's an order of magnitude faster.

On Apple silicon with FileVault, a raw image of the internal SSD is encrypted and

essentially useless without the volume key. Live acquisition from a booted, unlocked

system is usually the only viable path.
```

# Logical collection instead

BASH

```
sudo rsync -aHAXx --progress /Users/<u>/ /Volumes/Ext/collect/Users_<u>/

sudo ditto --rsrc --extattr /Users/<u> /Volumes/Ext/collect/<u>
```

`ditto` preserves resource forks and xattrs prefer it over `cp` for evidence copies.

## † Hashing & Verification

BASH

```
shasum -a 256 <file>

shasum -a 256 -c manifest.sha256

md5 <file>

find /path -type f -exec shasum -a 256 {} \; > manifest.sha256

openssl dgst -sha256 <file>

cksum <file>
```

## † Mounting Evidence Read-Only

BASH

```
# mkdir /tmp/evidence

# mount -t apfs -o ro /dev/disk4s1 /tmp/evidence

# mount -t hfs -o ro,noexec,nosuid /dev/disk4s1 /tmp/evidence

# diskutil mount readOnly /dev/disk4s1

# hdiutil attach -readonly -noverify -nobrowse image.dmg

Always `-o ro` and ideally `noexec,nosuid` on evidence.
```

## † SUSPICIOUS FILE HUNTS

BASH

```
# SUID/SGID

# find / -type f \( -perm -4000 -o -perm -2000 \) -ls 2>/dev/null

# World-writable dirs outside expected

# find / -type d -perm -0002 -not -path '/System/*' -not -path '/private/var/folders/*' -ls 2>/dev/null | head -50

# Recently modified in system dirs

# find /usr/local /Library /opt -type f -mtime -7 -ls 2>/dev/null | head -100

# Hidden files in home

find ~ -maxdepth 2 -name '.*' -type f -ls 2>/dev/null

# Mach-O binaries outside /Applications and /usr

# find /Users /tmp /var/tmp -type f -exec sh -c 'file "$1" | grep -q Mach-O && echo "$1"' _ {} \; 2>/dev/null

# Large recently-created files (staged exfil)

# find /Users /tmp -type f -size +50M -mtime -7 -ls 2>/dev/null

# Archives in odd places

# find /tmp /var/tmp /Users/*/Library -type f \( -name '*.zip' -o -name '*.tar.gz' -o -name '*.7z' -o -name '*.dmg' \) -mtime -14 2>/dev/null
```

## † .DS_STORE AS AN ARTIFACT

```bash
python3 -c "
import sys
d=open(sys.argv[1],'rb').read()
import re
print('\n'.join(sorted(set(re.findall(rb'[\x20-\x7e]{4,}', d)))[:50]))
" ~/Downloads/.DS_Store 2>/dev/null
`.DS_Store` records filenames that were once in a directory
 including deleted ones.
Useful for proving a file existed.
```

BOOK XXIV

# Users, Directory Services, Authentication

*canticle of the 12th sheet*

✦ ⁂ ✦

## † Enumerate users

```bash
dscl . list /Users

dscl . list /Users | grep -v '^_'

dscl . -list /Users UniqueID

dscl . -list /Users NFSHomeDirectory

dscl . -list /Users UserShell

dscl . -read /Users/<user>

dscl . -read /Users/<user> UniqueID PrimaryGroupID NFSHomeDirectory UserShell RealName

id <user>

id -Gn <user>

finger <user> 2>/dev/null

ls -la /Users/

ls -la /var/db/dslocal/nodes/Default/users/

Real (non-system) users have `UniqueID >= 500`. Anything with an ID in that range you

didn't create is a finding.
```

```bash
dscl . -list /Users UniqueID | awk '$2 >= 500 {print}'
```

## † Admin / Privileged Group Membership

```bash
dscl . -read /Groups/admin GroupMembership

dscl . -read /Groups/wheel GroupMembership

dscl . -read /Groups/_developer GroupMembership

dscl . -read /Groups/com.apple.access_ssh GroupMembership 2>/dev/null

dscl . -read /Groups/com.apple.access_screensharing GroupMembership 2>/dev/null

dseditgroup -o checkmember -m <user> admin

dscl . list /Groups | grep -v '^_'

dscl . -read /Groups/staff GroupMembership

`com.apple.access_ssh` controls who can SSH in when Remote Login is restricted

 check it.
```

## † Create / Modify / Delete

```bash
# sysadminctl -addUser <user> -fullName "Name" -password <pw> -admin

# sysadminctl -addUser <user> -UID 501 -shell /bin/zsh -home /Users/<user>

# sysadminctl -deleteUser <user>            #

# sysadminctl -deleteUser <user> -secure    #

 secure erase home

# sysadminctl -newPassword <pw> -oldPassword <old> -passwordHint ""

# dseditgroup -o edit -a <user> -t user admin

# dseditgroup -o edit -d <user> -t user admin

# dscl . -create /Users/<user> UserShell /usr/bin/false

# dscl . -delete /Users/<user>              #

sysadminctl -guestAccount status

# sysadminctl -guestAccount off
```

## † PASSWORD HASHES & POLICY

BASH

```
# defaults read /var/db/dslocal/nodes/Default/users/<user>.plist ShadowHashData

# plutil -p /var/db/dslocal/nodes/Default/users/<user>.plist

# dscl . -read /Users/<user> AuthenticationAuthority

pwpolicy -u <user> -getpolicy

pwpolicy -getaccountpolicies

pwpolicy -n /Local/Default -getglobalpolicy

pwpolicy -u <user> -getcurrentpolicyaccount

macOS uses PBKDF2-SHA512 with a high iteration count. Hashes live in the user's dslocal

plist under `ShadowHashData` (SIP-protected, root only).
```

# Extract to a hashcat-compatible format (authorized assessments only) # plutil -extract ShadowHashData raw /var/db/dslocal/nodes/Default/users/<u>.plist | \ # base64 -d | plutil -p - | grep -A4 'SALTED-SHA512-PBKDF2'

## † LOGIN / AUTH HISTORY

BASH

```
last

last -50

last <user>

last reboot

last shutdown

lastb 2>/dev/null                          # failed logins, often absent on macOS

who -a

users

ac -p 2>/dev/null                          # per-user connect time

ls -la /var/log/wtmp* /var/log/utmpx 2>/dev/null

`last` on macOS reads `/var/log/utmpx` which is thin. The unified log is far richer:
```

BASH

```
log show --last 7d --predicate 'process == "loginwindow"' --info | grep -iE 'login|
logout|user'

log show --last 7d --predicate 'process == "SecurityAgent"' --info

log show --last 7d --predicate 'subsystem == "com.apple.opendirectoryd"' --info | grep -
iE 'auth|fail'

log show --last 7d --predicate 'eventMessage CONTAINS "Authentication failed"' --info

log show --last 7d --predicate 'eventMessage CONTAINS "Failed to authenticate"' --info

# eslogger authentication login_login login_logout | jq .
```

### ▸ SCREEN UNLOCK / LOCK TIMELINE

BASH

```bash
log show --last 7d --predicate 'eventMessage CONTAINS "ScreenIsLocked" OR eventMessage CONTAINS "ScreenIsUnlocked"' --info

sqlite3 ~/Library/Application\ Support/Knowledge/knowledgeC.db \

  "SELECT datetime(ZSTARTDATE+978307200,'unixepoch'), ZVALUEINTEGER

   FROM ZOBJECT WHERE ZSTREAMNAME='/device/isLocked' ORDER BY ZSTARTDATE DESC LIMIT 40;"

# eslogger lw_session_lock lw_session_unlock | jq .
```

## † SUDO

BASH

```bash
sudo -l

sudo -l -U <user>

sudo -V

cat /etc/sudoers

ls -la /etc/sudoers.d/

cat /etc/sudoers.d/*

# visudo                                  # edit safely

# visudo -c                               # syntax check

grep -vE '^\s*(#|$)' /etc/sudoers
```

## ▸ SUDO HUNTING

BASH

```
log show --last 7d --predicate 'process == "sudo"' --info

log show --last 7d --predicate 'process == "sudo"' --info | grep -i 'COMMAND='

# eslogger sudo su | jq -r '[.time, .event.sudo.from_username, (.event.sudo.command //
"")] | @tsv'

ls -la /etc/sudoers.d/ && stat -f '%Sm %N' -t '%FT%T' /etc/sudoers /etc/sudoers.d/*

Red flags: `NOPASSWD: ALL` for a non-service account, a new file in `/etc/sudoers.d/`,

`Defaults !tty_tickets`, or `timestamp_timeout` set very high.
```

## ▸ TOUCH ID FOR SUDO

BASH

```
cat /etc/pam.d/sudo

cat /etc/pam.d/sudo_local 2>/dev/null

# macOS 14+: create /etc/pam.d/sudo_local with:

#   auth  sufficient  pam_tid.so

cat /etc/pam.d/sudo_local.template
```

## † PAM

BASH

```
ls -la /etc/pam.d/

cat /etc/pam.d/{sudo,login,su,sshd,authorization,screensaver}

grep -r 'pam_' /etc/pam.d/ | grep -v '^\s*#'

ls -la /usr/lib/pam/

A custom `.so` in `/usr/lib/pam/` or an unexpected line in `/etc/pam.d/*` is a credential

capture / auth bypass. Compare against a known-good machine:
```

BASH

```
shasum -a 256 /etc/pam.d/* | sort -k2
```

## † SSH

BASH

```bash
sudo systemsetup -getremotelogin

# systemsetup -setremotelogin on|off

sudo sshd -T | grep -iE 'permitrootlogin|passwordauthentication|pubkeyauthentication|
port|allowusers|allowgroups|permitemptypasswords|maxauthtries'

cat /etc/ssh/sshd_config

ls -la /etc/ssh/

cat ~/.ssh/authorized_keys

for u in /Users/*; do echo "=== $u"; cat "$u/.ssh/authorized_keys" 2>/dev/null; done

ssh-keygen -lf ~/.ssh/authorized_keys

dscl . -read /Groups/com.apple.access_ssh GroupMembership 2>/dev/null

log show --last 7d --predicate 'process == "sshd"' --info

# eslogger openssh_login openssh_logout | jq .

Hardened `sshd_config`:

PermitRootLogin no

PasswordAuthentication no

KbdInteractiveAuthentication no

ChallengeResponseAuthentication no

PubkeyAuthentication yes

PermitEmptyPasswords no

MaxAuthTries 3

LoginGraceTime 30

AllowGroups com.apple.access_ssh

X11Forwarding no
```

## † SCREEN SHARING / ARD

BASH

```
sudo launchctl list | grep -iE 'screensharing|RemoteDesktop|ARDAgent'

sudo lsof -i :5900 -i :3283 -n -P

ls -la /Library/Application\ Support/Apple/Remote\ Desktop/

defaults read /Library/Preferences/com.apple.RemoteManagement 2>/dev/null

# /System/Library/CoreServices/RemoteManagement/ARDAgent.app/Contents/Resources/kickstart
-deactivate -stop

dscl . -read /Groups/com.apple.access_screensharing GroupMembership 2>/dev/null

# eslogger screensharing_attach screensharing_detach | jq .

log show --last 7d --predicate 'eventMessage CONTAINS "screensharing"' --info

ARD/Remote Management enabled without your knowledge is a classic lateral movement setup.

`kickstart` with `-activate -configure -allowAccessFor -allUsers` is the enabling command

to grep history for.
```

## † DIRECTORY SERVICE BINDINGS (AD/LDAP)

BASH

```
dscl localhost -list /

dsconfigad -show 2>/dev/null

dsconfigldap -show 2>/dev/null

odutil show all 2>/dev/null | head -60

dscl "/Active Directory/<DOMAIN>/All Domains" -read /Users/<user> 2>/dev/null

klist                                    # Kerberos tickets

klist -A

kinit <user>@<REALM>

kdestroy

cat /etc/krb5.conf 2>/dev/null

dscacheutil -statistics

# dscacheutil -flushcache
```

## † ACCOUNT TIMELINE

BASH

```
# When were accounts created?

stat -f '%SB %N' -t '%FT%T' /var/db/dslocal/nodes/Default/users/*.plist

stat -f '%SB %N' -t '%FT%T' /Users/*

log show --last 90d --predicate 'subsystem == "com.apple.opendirectoryd"' --info | grep -
iE 'create.*user|add.*record'

# eslogger od_create_user od_group_add od_modify_password od_disable_user | jq .
```

## † QUICK ACCOUNT AUDIT

BASH

```
#!/bin/zsh

print "═══ Users (UID >= 500)"

dscl . -list /Users UniqueID | awk '$2>=500'

print "\n═══ Admins"

dscl . -read /Groups/admin GroupMembership

print "\n═══ Secure token status"

for u in $(dscl . -list /Users UniqueID | awk '$2>=500{print $1}'); do

  printf '%-20s %s\n' "$u" "$(sysadminctl -secureTokenStatus $u 2>&1 | grep -o 'ENABLED\|
DISABLED')"

done

print "\n═══ Shells"

dscl . -list /Users UserShell | grep -v false

print "\n═══ authorized_keys"

for f in /Users/*/.ssh/authorized_keys; do [[ -f $f ]] && { print "--- $f"; cat $f; };
done

print "\n═══ sudoers.d"

ls -la /etc/sudoers.d/ 2>/dev/null

print "\n═══ Autologin"

[[ -f /etc/kcpassword ]] && print "!! /etc/kcpassword EXISTS"

defaults read /Library/Preferences/com.apple.loginwindow autoLoginUser 2>/dev/null
```

BOOK XXV

# STATIC MACH-O TRIAGE

*canticle of the 18th sheet*

✦ ⁂ ✦

On-host static analysis of a suspicious binary. Do this in a VM or an isolated machine, not on the host you're investigating.

## † IDENTIFY

BASH

```
file <bin>

file -b <bin>

lipo -info <bin>                          # architectures in a fat binary

lipo -thin arm64 <bin> -output /tmp/bin.arm64

otool -h <bin>                            # Mach-O header

otool -f <bin>                            # fat header

shasum -a 256 <bin>

md5 <bin>

ls -la@ <bin>

xattr -l <bin>

stat -x <bin>

Mach-O magic: `0xfeedface` (32-bit), `0xfeedfacf` (64-bit), `0xcafebabe` (fat/universal).
```

BASH

```
xxd -l 16 <bin>
```

## † LOAD COMMANDS & STRUCTURE

BASH

```
otool -l <bin>                          # all load commands
otool -l <bin> | grep -A5 LC_SEGMENT
otool -l <bin> | grep -A3 LC_RPATH
otool -l <bin> | grep -A2 LC_LOAD_DYLIB
otool -l <bin> | grep -A2 LC_LOAD_WEAK_DYLIB     # hijackable
otool -l <bin> | grep -A5 LC_CODE_SIGNATURE
otool -l <bin> | grep -A5 LC_MAIN
otool -l <bin> | grep -A5 LC_ENCRYPTION_INFO     # encrypted segment
otool -L <bin>                          # linked libraries, concise
otool -tV <bin> | head -100             # disassembly
otool -s __TEXT __cstring <bin>         # C strings section
otool -o <bin>                          # Objective-C metadata
otool -v -s __DATA __objc_classlist <bin>
```

## † SYMBOLS & IMPORTS

BASH

```
nm -m <bin>

nm -u <bin>                              # undefined = imports

nm -g <bin>                              # global symbols

nm -m <bin> | grep -i 'undefined.*_'     # what it calls

dyldinfo -bind <bin> 2>/dev/null

dyldinfo -lazy_bind <bin> 2>/dev/null

objdump --syms <bin> 2>/dev/null

objdump -d <bin> | head -100

c++filt < <(nm -g <bin>)                 # demangle C++

class-dump <bin>                         # Obj-C headers; brew install class-dump
```

### ▸ HIGH-SIGNAL IMPORTS

BASH

```
nm -u <bin> | grep -iE 'NSTask|posix_spawn|system|popen|execv|fork'

nm -u <bin> | grep -iE 'SecKeychain|SecItemCopy|SecItemAdd'

nm -u <bin> | grep -iE 'CGEventTap|CGEventPost|IOHIDManager'      # keylogging

nm -u <bin> | grep -iE 'CGDisplayCreateImage|CGWindowListCreateImage'  # screenshots

nm -u <bin> | grep -iE 'AVCaptureDevice|AudioQueue'               # camera/mic

nm -u <bin> | grep -iE 'NSAppleScript|OSAExecute'                 # AppleScript

nm -u <bin> | grep -iE 'CFNetwork|NSURLSession|socket|connect|send'

nm -u <bin> | grep -iE 'dlopen|dlsym|NSCreateObjectFile'          # dynamic loading

nm -u <bin> | grep -iE 'ptrace|task_for_pid|mach_vm_write'        # anti-debug /
injection

nm -u <bin> | grep -iE 'CCCrypt|SecKeyCreate|EVP_'                # crypto
```

## † STRINGS

BASH

```
strings -a <bin>

strings -a -n 8 <bin>

strings -a -t x <bin>                    # with offsets

strings -a - <bin>                       # all sections (macOS strings syntax varies)

strings -a <bin> | grep -iE 'https?://|ftp://'

strings -a <bin> | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}'

strings -a <bin> | grep -oE '[a-z0-9.-]+\.(com|net|org|io|ru|cn|xyz|top|onion)'

strings -a <bin> | grep -iE '/Users/|/tmp/|/Library/|LaunchAgent|LaunchDaemon|plist'

strings -a <bin> | grep -iE 'password|token|api[_-]?key|secret|credential'

strings -a <bin> | grep -iE 'osascript|do shell script|display dialog'

strings -a <bin> | grep -iE 'curl|wget|base64|xattr|chmod|chflags'

strings -a <bin> | grep -E '^[A-Za-z0-9+/]{40,}={0,2}$'   # base64 blobs

# UTF-16 strings (missed by default)

strings -a -el <bin>
```

## † ENTROPY / PACKING

BASH

```
python3 - <<'EOF' <bin>

import sys, math, collections

d = open(sys.argv[1],'rb').read()

c = collections.Counter(d)

e = -sum((n/len(d))*math.log2(n/len(d)) for n in c.values())

print(f"size={len(d)} entropy={e:.3f}")

Entropy > 7.2 across the whole file suggests packing or encryption. Check per-section
too:
```

```bash
otool -l <bin> | grep -A4 'sectname'
```

Common macOS packers: UPX (`strings | grep UPX`), custom loaders, and increasingly just encrypted payload blobs decrypted at runtime.

```bash
strings -a <bin> | grep -i upx

upx -t <bin> 2>/dev/null && upx -d -o /tmp/unpacked <bin>
```

## † CODE SIGNATURE DETAIL

```bash
codesign -dv --verbose=4 <bin> 2>&1

codesign -d --entitlements - <bin> 2>/dev/null | plutil -p -

codesign -d -r- <bin>

codesign --verify --deep --strict --verbose=4 <bin> 2>&1

codesign -d --extract-certificates=/tmp/c <bin> && \

    for f in /tmp/c*; do openssl x509 -inform DER -in "$f" -noout -subject -issuer -
dates; done
```

## † BUNDLE ANALYSIS (.APP)

BASH

```
find /path/Foo.app -type f | head -50

plutil -p /path/Foo.app/Contents/Info.plist

/usr/libexec/PlistBuddy -c Print /path/Foo.app/Contents/Info.plist

ls -la /path/Foo.app/Contents/{MacOS,Resources,Frameworks,Library,XPCServices,PlugIns}
2>/dev/null

find /path/Foo.app -name '*.dylib' -o -name '*.so' -o -name '*.framework' -o -name
'*.xpc'

find /path/Foo.app -type f -perm +111 -exec file {} \; | grep Mach-O

find /path/Foo.app \( -name '*.sh' -o -name '*.py' -o -name '*.pl' -o -name '*.scpt' -o -
name '*.command' \)

du -sh /path/Foo.app
```

### ▸ INFO.PLIST KEYS WORTH CHECKING

BASH

```
/usr/libexec/PlistBuddy -c 'Print :LSUIElement' Info.plist 2>/dev/null      # hides from
Dock

/usr/libexec/PlistBuddy -c 'Print :LSBackgroundOnly' Info.plist 2>/dev/null

/usr/libexec/PlistBuddy -c 'Print :NSAppleEventsUsageDescription' Info.plist 2>/dev/null

plutil -p Info.plist | grep -iE 'UsageDescription|LSUIElement|LSBackgroundOnly|
CFBundleURLSchemes|NSAppTransportSecurity'

`LSUIElement = true` means the app runs with no Dock icon or menu bar

 legitimate for

menu-bar utilities, and standard for anything trying to stay out of sight.
```

## † SCRIPTS & INTERPRETED PAYLOADS

```bash
file <f>

head -50 <f>

osadecompile <f>.scpt                    # compiled AppleScript

python3 -c "import dis,marshal,sys; ..." # .pyc

 use uncompyle6/decompyle3

node --check <f>.js

base64 -d <<< "<blob>" | file -

echo "<blob>" | base64 -d | head -c 500
```

### ▸ DEOBFUSCATING COMMON PATTERNS

```bash
# base64 in a shell script

grep -oE '[A-Za-z0-9+/]{40,}={0,2}' script.sh | while read b; do echo "$b" | base64 -d
2>/dev/null; echo; done

# hex

grep -oE '(\\x[0-9a-f]{2})+' script.sh | tr -d '\\x' | xxd -r -p

# eval chains

grep -nE 'eval|exec|osascript -e|do shell script' script.sh
```

## † BEHAVIORAL HINTS WITHOUT RUNNING IT

```bash
strings -a <bin> | grep -iE 'LaunchAgents|LaunchDaemons|com\.apple\.' | sort -u

strings -a <bin> | grep -iE 'plist' | sort -u

strings -a <bin> | grep -oE 'com\.[a-z0-9.-]+' | sort -u | head
-30    # candidate bundle IDs / labels

strings -a <bin> | grep -iE 'User-Agent|POST|GET /|Content-Type'
```

## † YARA

```bash
brew install yara

yara -r rules.yar /path/to/scan

yara -s rules.yar <bin>                  # show matching strings

yara -m rules.yar <bin>                  # show meta

yara -w -r /path/rules/ /Applications
```

rule macos_suspicious_persistence meta: description = "Mach-O referencing launch persistence + shell exec" author = "james" strings: $macho1 = { cf fa ed fe } $macho2 = { ca fe ba be } $p1 = "LaunchAgents" ascii $p2 = "LaunchDaemons" ascii $p3 = "RunAtLoad" ascii $e1 = "NSTask" ascii $e2 = "posix_spawn" ascii $e3 = "do shell script" ascii $n1 = "NSURLSession" ascii condition: ($macho1 at 0 or $macho2 at 0) and any of ($p*) and any of ($e*) and $n1

```yara
rule macos_osascript_credential_prompt

    meta:

        description = "AppleScript password phishing pattern used by AMOS-family
stealers"

    strings:

        $a = "display dialog" ascii nocase

        $b = "hidden answer" ascii nocase

        $c = "with administrator privileges" ascii nocase

        $d = "password" ascii nocase

    condition:

        $a and $d and ($b or $c)
```

## † DIFFING AGAINST A KNOWN-GOOD COPY

BASH

```
shasum -a 256 suspect_binary known_good_binary

cmp -l suspect known_good | head -50

vbindiff suspect known_good              # brew install vbindiff

diff <(otool -L suspect) <(otool -L known_good)

diff <(nm -m suspect) <(nm -m known_good)

diff <(strings -a suspect | sort -u) <(strings -a known_good | sort -u)

diff <(codesign -dv --verbose=4 suspect 2>&1) <(codesign -dv --verbose=4 known_good 2>&1)
```

## † TOOLING TO INSTALL

BASH

```
brew install binutils yara radare2 xxd hexyl

brew install --cask hopper-disassembler   # commercial, excellent on macOS

brew install --cask ghidra                # free, NSA; handles Mach-O well

pip3 install macholib lief

brew install class-dump                   # Obj-C class dumps

brew install jtool2 2>/dev/null           # or download from newosxbook.com

`jtool2` (Jonathan Levin) is purpose-built for Mach-O and covers things `otool` doesn't

entitlements, code directory parsing, dyld shared cache extraction.
```

BASH

```
jtool2 -l <bin>

jtool2 --ent <bin>

jtool2 -S <bin>

jtool2 -d __TEXT.__text <bin>
```

## † LIEF FOR SCRIPTED ANALYSIS

PYTHON

```python
#!/usr/bin/env python3

import lief, sys, hashlib

b = lief.MachO.parse(sys.argv[1])

for binary in b:

    print(f"--- {binary.header.cpu_type}")

    print("libs:", [c.name for c in binary.libraries])

    print("rpaths:", [c.path for c in binary.commands if isinstance(c,
lief.MachO.RPathCommand)])

    print("imports:", [s.name for s in binary.imported_symbols][:40])

    print("segments:", [(s.name, hex(s.virtual_address), s.virtual_size) for s in
binary.segments])

    if binary.has_code_signature:

        print("has code signature")
```

## † REPORTING CHECKLIST

For each sample, record: - SHA-256, SHA-1, MD5, file size - Mach-O type, architectures, entry point - Code signing: signed?, ad-hoc?, TeamID, authority chain, notarized? - Entitlements (full list) - Linked libraries and rpaths - Network indicators (domains, IPs, URLs, user agents) - Filesystem indicators (paths written, persistence locations) - Persistence mechanism(s) - Capabilities inferred from imports - Any embedded scripts or secondary payloads - Compilation timestamp if present, and whether it's plausible



*Thus ends the Codex. Every verse herein remains a true incantation — strip nothing further, add nothing false. Speak them into the terminal-altar and the spirits answer as they answered the Wanderers of old.*

*Let the faithful remember: power turned upon the innocent is heresy. These rites are for the guarding of one's own keep and for those who have granted thee leave. Wander wisely.*

◆ TRANSCRIBED FROM THE ORAL TERMINAL ◆